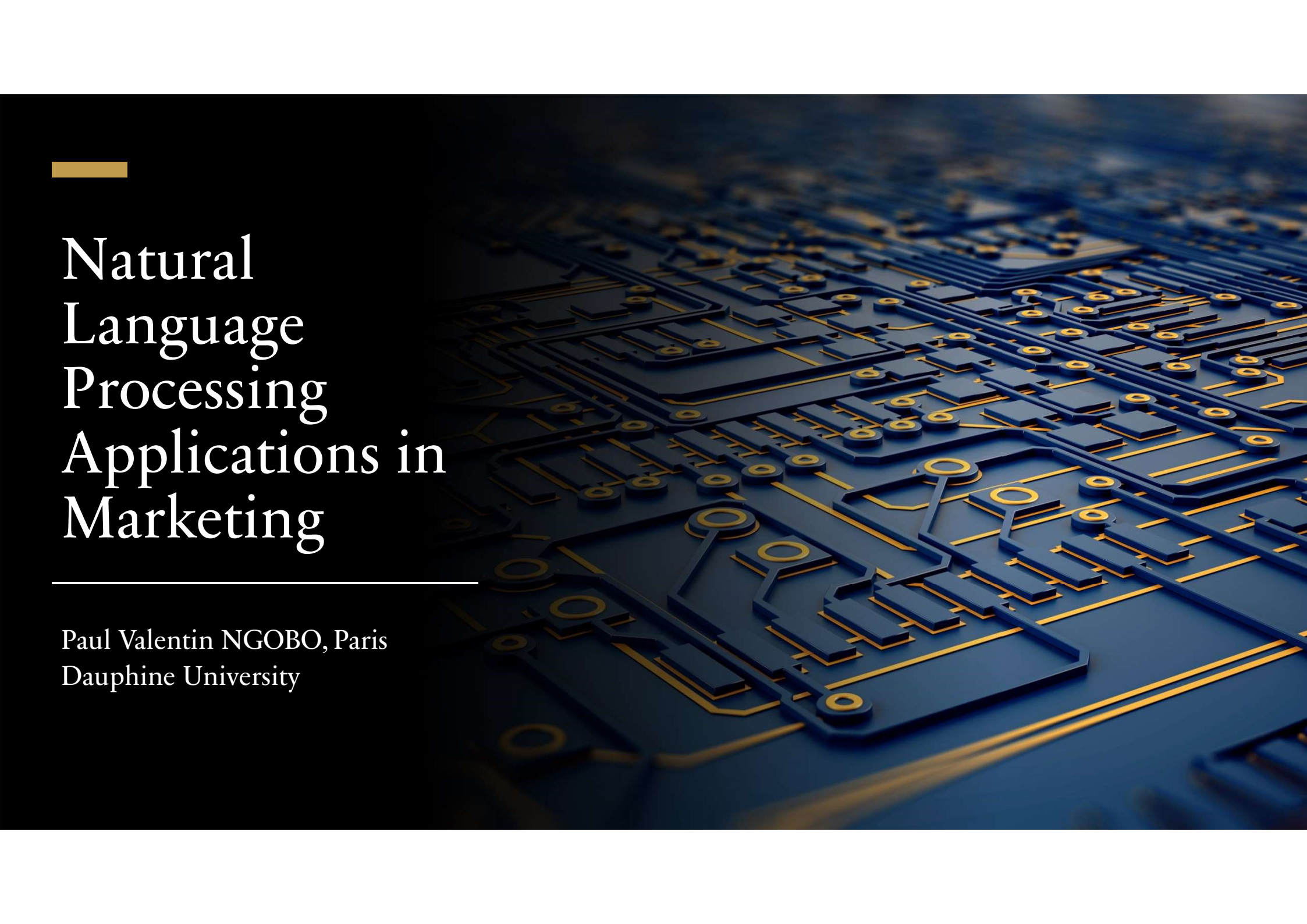




Natural Language Processing Applications in Marketing

Paul Valentin NGOBO, Paris
Dauphine University

Agenda



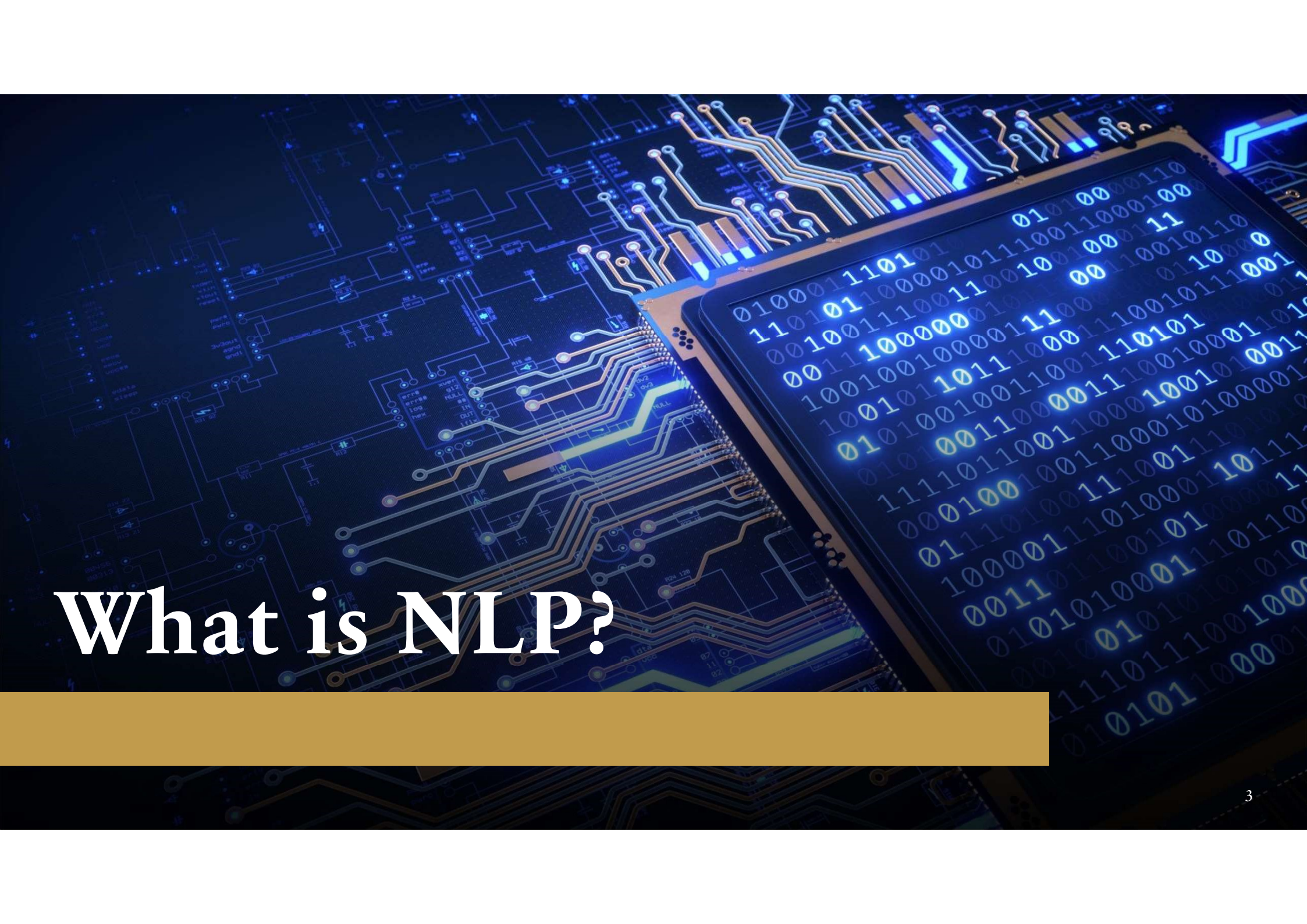
What is NLP?



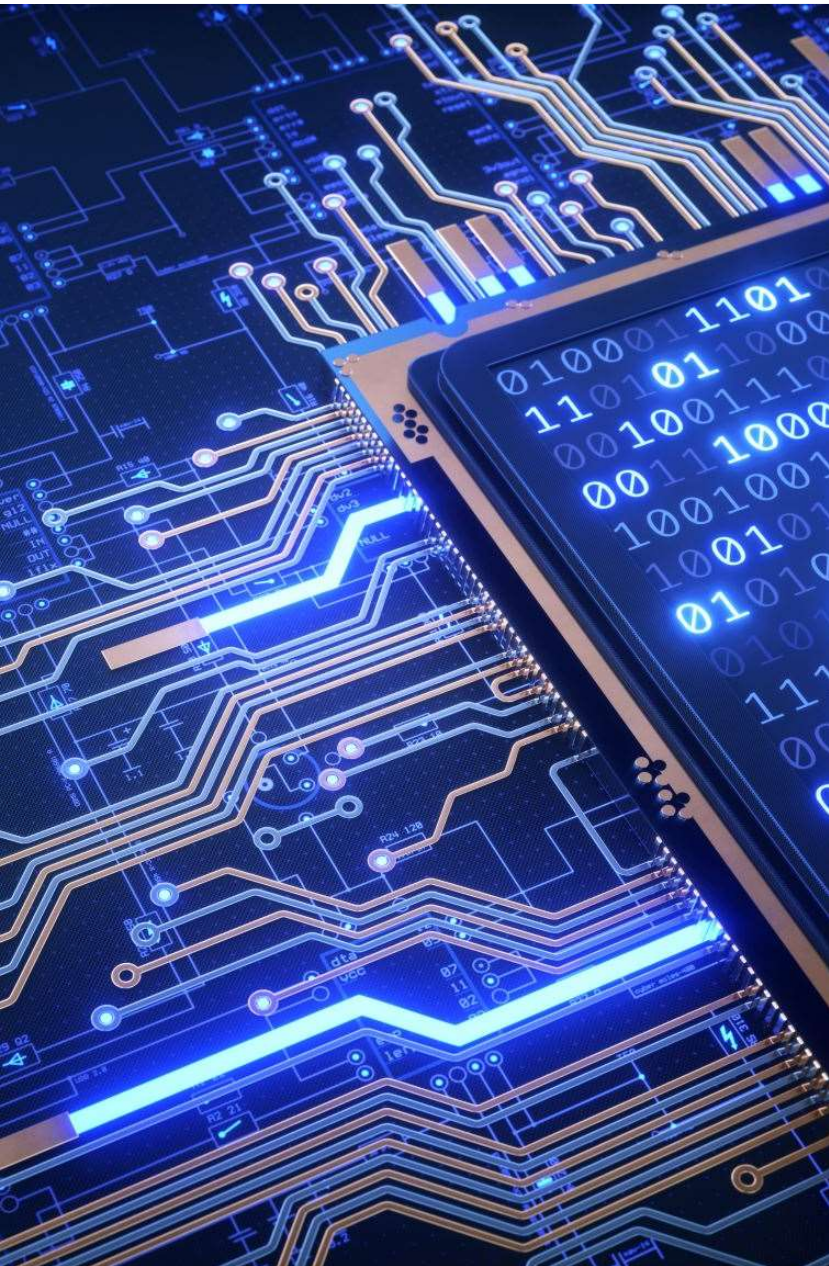
Why do we need NLP in Marketing?



How to do NLP?

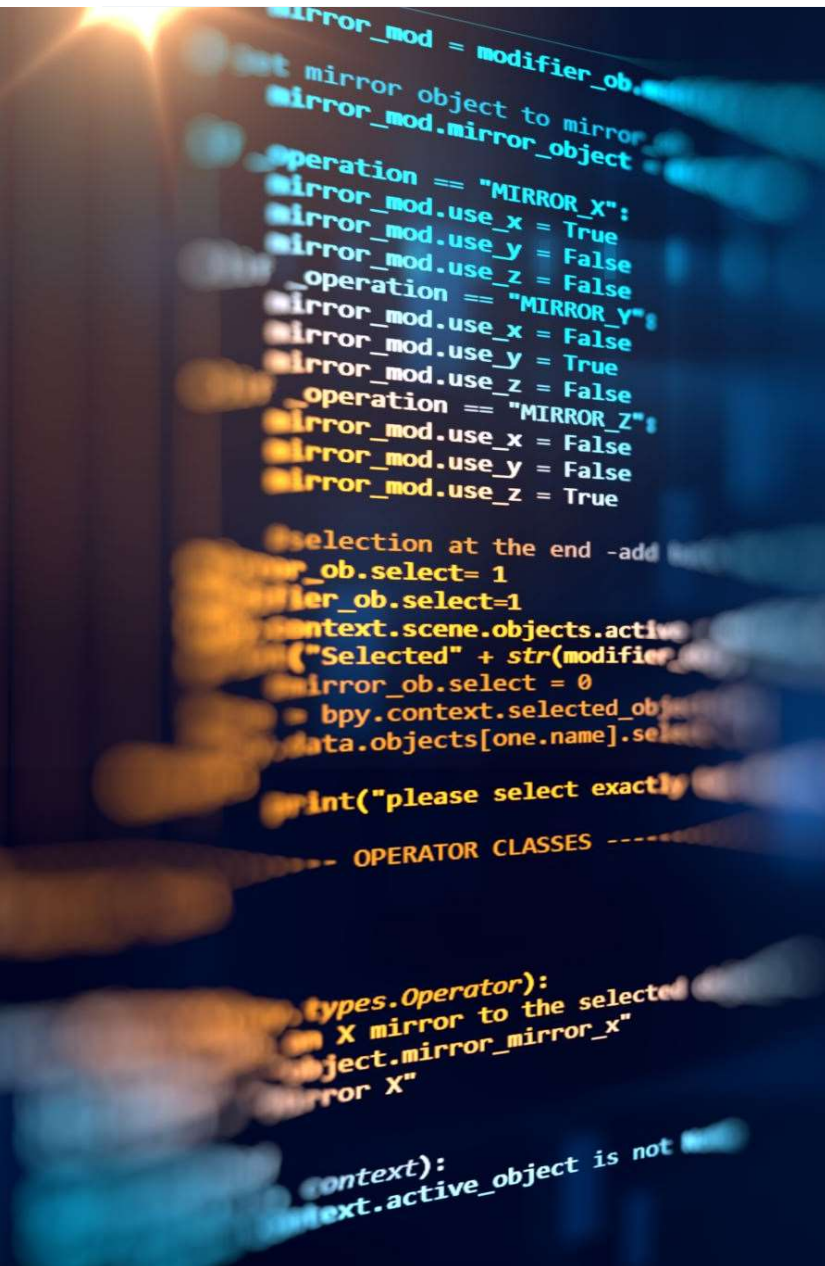


What is NLP?



- Natural language
 - English
 - French
 - Chinese
 - Swahili
 - Python is not a natural language
- Processing
 - How a computer conduct instructions

- A Quick Definition
 - Natural language simply refers to the way we communicate with each other: speech and text.
- **Processing** refers to making natural language usable for computational tasks.
 - So, Natural Language Processing (NLP) is concerned with finding, digesting, and understanding human speech and text.



- Largely defined, it is
 - *The automatic manipulation of natural language such as speech and text, by software.*
- NLP
 - uses algorithms
 - to transform text data (unstructured data) into data that can be processed (structured data)
 - using statistical and Machine Learning, and computational linguistics methods.

La manipulation automatique du langage naturel tel que la parole et le texte, par un logiciel.

Artificial Intelligence

Any technique which enables computers to mimic human behaviour.

Machine Learning

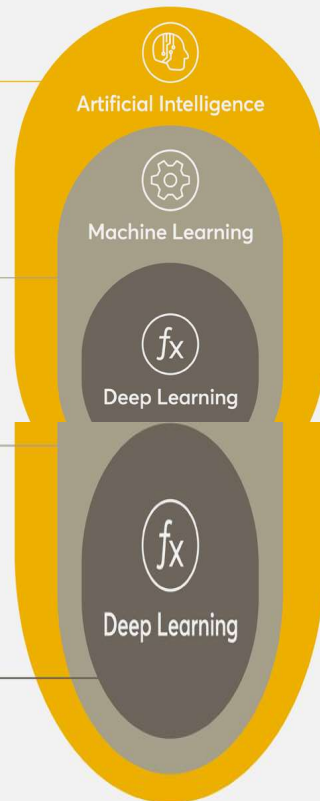
Subset of AI techniques which use statistical methods to enable machines to improve with experiences.

Deep Learning

statistical methods to enable machines to improve with experiences.

Deep Learning

Subset of ML which makes the computation of multi-layer neural networks feasible.



- NLP is a combination of
 - Artificial intelligence
 - *It uses Artificial intelligence – making computers do things that human do*
 - Linguistics

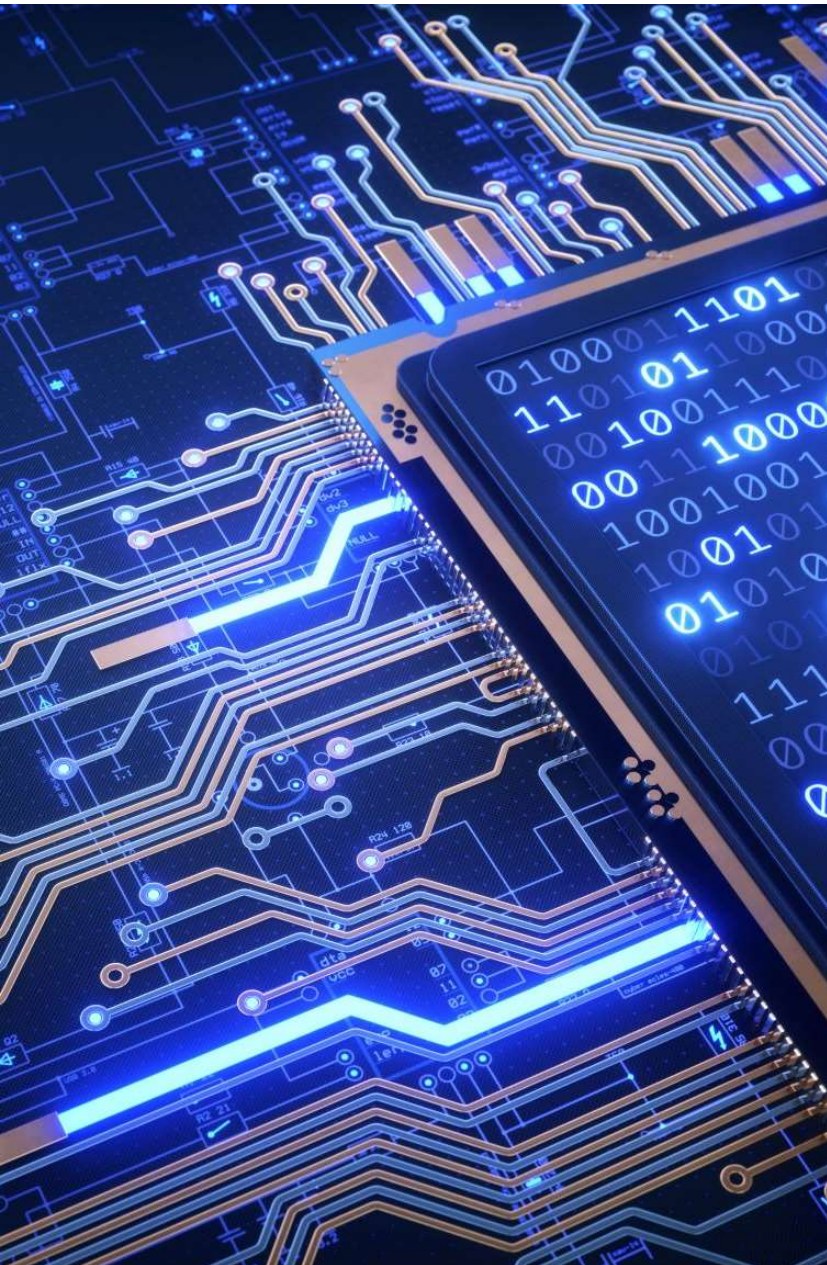
NLP is a branch of AI that aims at *enabling computers to make sense of human language* (natural language).



Why do we need NLP in Marketing?



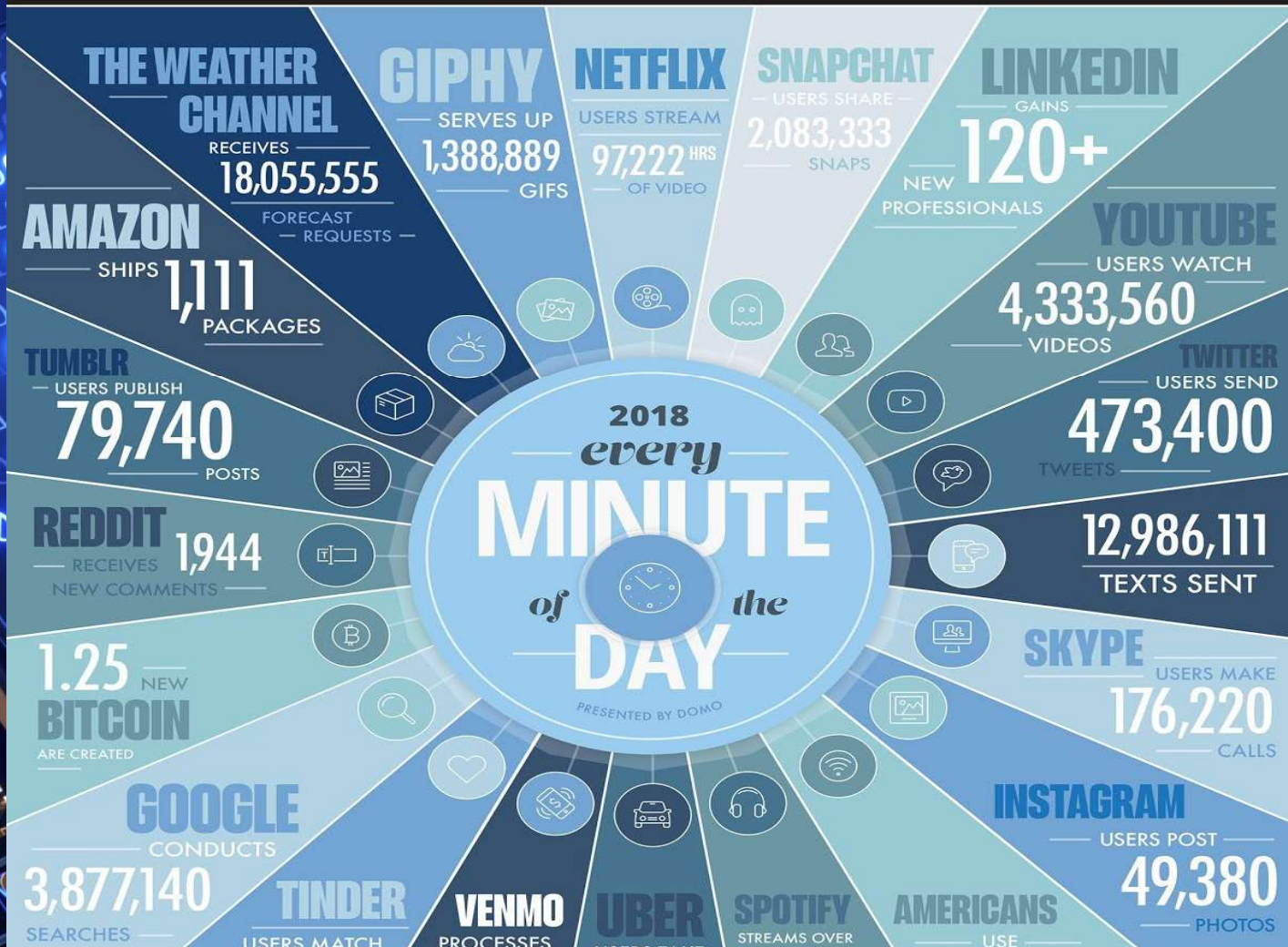
- *Millions of giga-bytes are generated everyday in blogs, social networks, web pages, etc.*
- *Many companies collect those data to understand users and their preferences.*



DOMO

How much data is generated *every minute*?

There's no way around it: big data just keeps getting bigger. The numbers are staggering, but they're not slowing down. By 2020, it's estimated that for every person on earth, 1.7 MB of data will be created every second. In our 6th edition of *Data Never Sleeps*, we once again take a look at how much data is being created all around us every single minute of the day—and we have a feeling things are just getting started.



Data Format

Data Source

Internal



Structured



Human-Generated

- Survey ratings
- Aptitude testing

Machine-Generated

- Web metrics from Web logs
- Product purchase from sales Records
- Process control measures

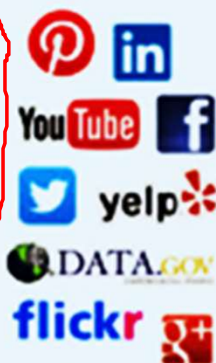
Unstructured



Human-Generated

- Emails, letters, text messages
- Audio transcripts
- Customer comments
- Voicemails
- Corporate video/communications
- Pictures, illustrations
- Employee reviews

External



Human-Generated

- Number of Retweets, Facebook likes, Google Plus +1s
- Ratings on Yelp
- Patient ratings ratings

Machine-Generated

- GPS for tweets
- Time of tweet/updates/postings

Human-Generated

- Content of social media updates
- Comments in online forums
- Comments on Yelp
- Video reviews
- Pinterest images
- Surveillance video



Some of the applications of NLP





- Information Extraction:
 - Automatic extraction of structured information such as entities, relationships between entities, and attributes describing entities from unstructured sources.
- Information retrieval
 - Information retrieval system is a network of algorithms, which facilitate the search of relevant data / documents as per the user requirement.
- Text Classification
 - Spam filtering, classifying email text as spam or not.
 - Language identification, classifying the language of the source text.
 - Genre classification, classifying the genre of a fictional story.



- Language Modelling
 - Generating new article headlines.
 - Generating new sentences, paragraphs, or documents.
 - Generating suggested continuation of a sentence.
- Speech Recognition
 - Transcribing a speech.
 - Creating text captions for a movie or TV show.
 - Issuing commands to the radio while driving.
- Caption Generation
 - Describing the contents of a scene.
 - Creating a caption for a photograph.
 - Describing a video.



- Machine Translation — Converting one language into another
 - Translating a text document from French to English.
 - Translating Spanish audio to German text.
 - Translating English text to Italian audio.
- Document Summation — Abstract creation for a document
 - Creating a heading for a document.
 - Creating an abstract of a document.
- Question Answering — Understanding a question and then formulating a human readable answer
 - Answering questions about news articles.
 - Answering questions about medical records.



NLP in Marketing





- **Customer satisfaction**

- You receive customer reviews upon their purchase and/or consumption experiences.
- You want to know,
 - the degree of customer satisfaction
 - Identify attributes of satisfaction and dissatisfaction
 - Identify topics of complaints
 - Examine customers' intentions



- **Brand or Product Analysis
(brand/product mentions)**

- Identify references to your brands and your products in your data.
- Analyze these mentions with sentiment, emotion and effort to determine how your customers relate to your offerings.
- Look at these mentions with filters for “suggestions” or “requests” to identify areas of opportunity to improve your offerings.



• **Competitor Analysis**

- Search your data for mentions of your competitors.
 - Look at corresponding volume and sentiment to assess market share.
 - Look at these mentions with a customer journey model to determine areas in which your competition shines and areas in which they fail.
 - Or, search for sentences which mention both you and your competition comparatively.
 - Armed with this information, you can design your business strategy to capitalize on your advantages.
-



- **Use NLP to Power Your SEO and User Experience**
 - Because of its syntax analysis capabilities, NLP understands **sentence content and context**.
 - Google's biggest algorithm update in 2019, the **BERT** Natural Processing Language model, allows its search engine to use its linguistic AI capabilities to understand the **intent** of searchers.
-

5. Text-based predictions

3. Topic Modeling

Real world

The observed world

Text data

perceive

(Perspective)

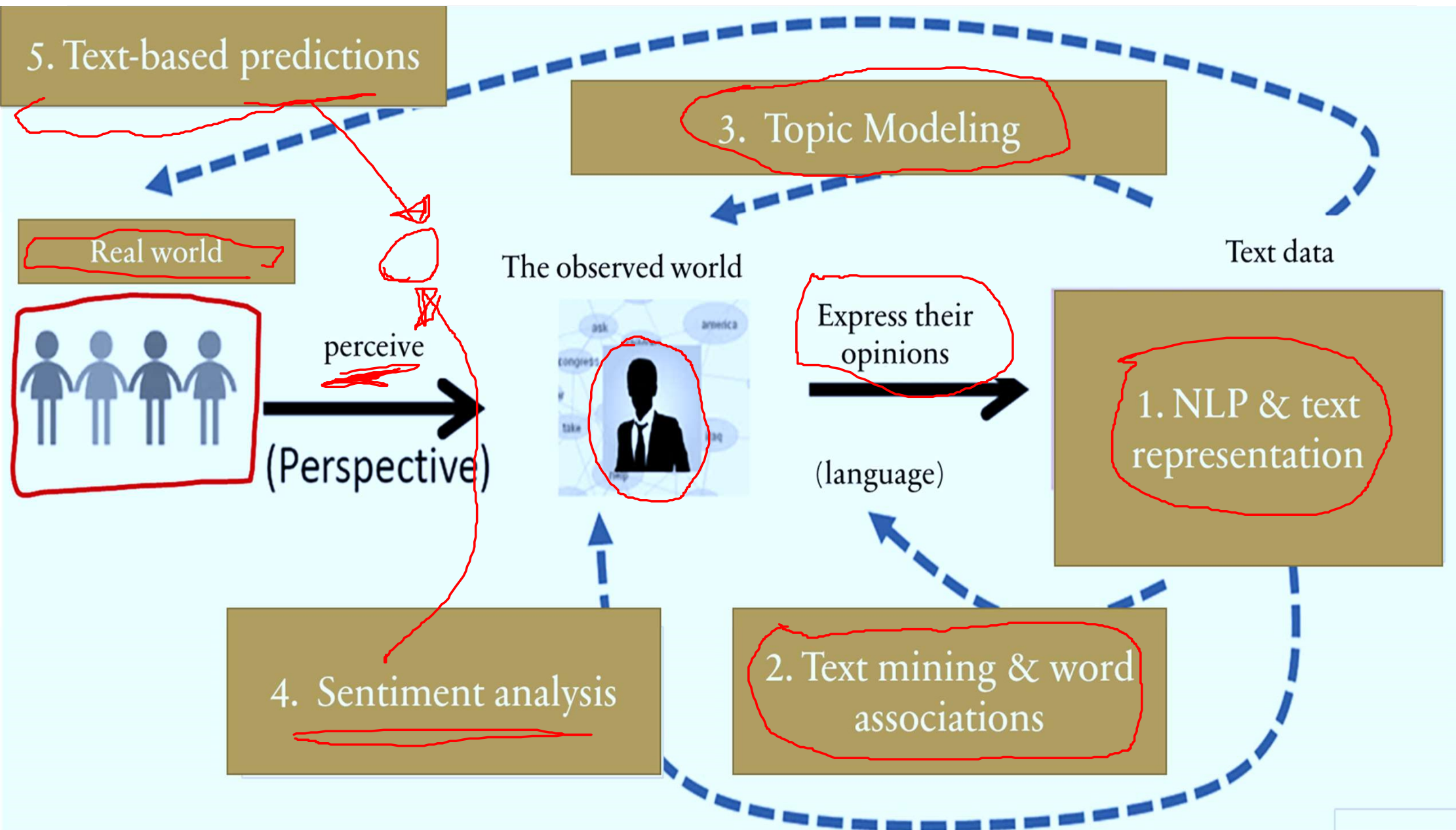
Express their
opinions

(language)

1. NLP & text
representation

4. Sentiment analysis

2. Text mining & word
associations

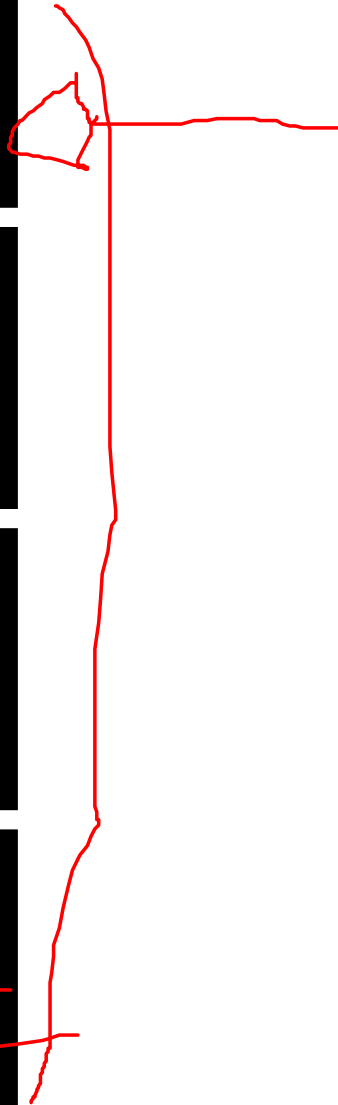


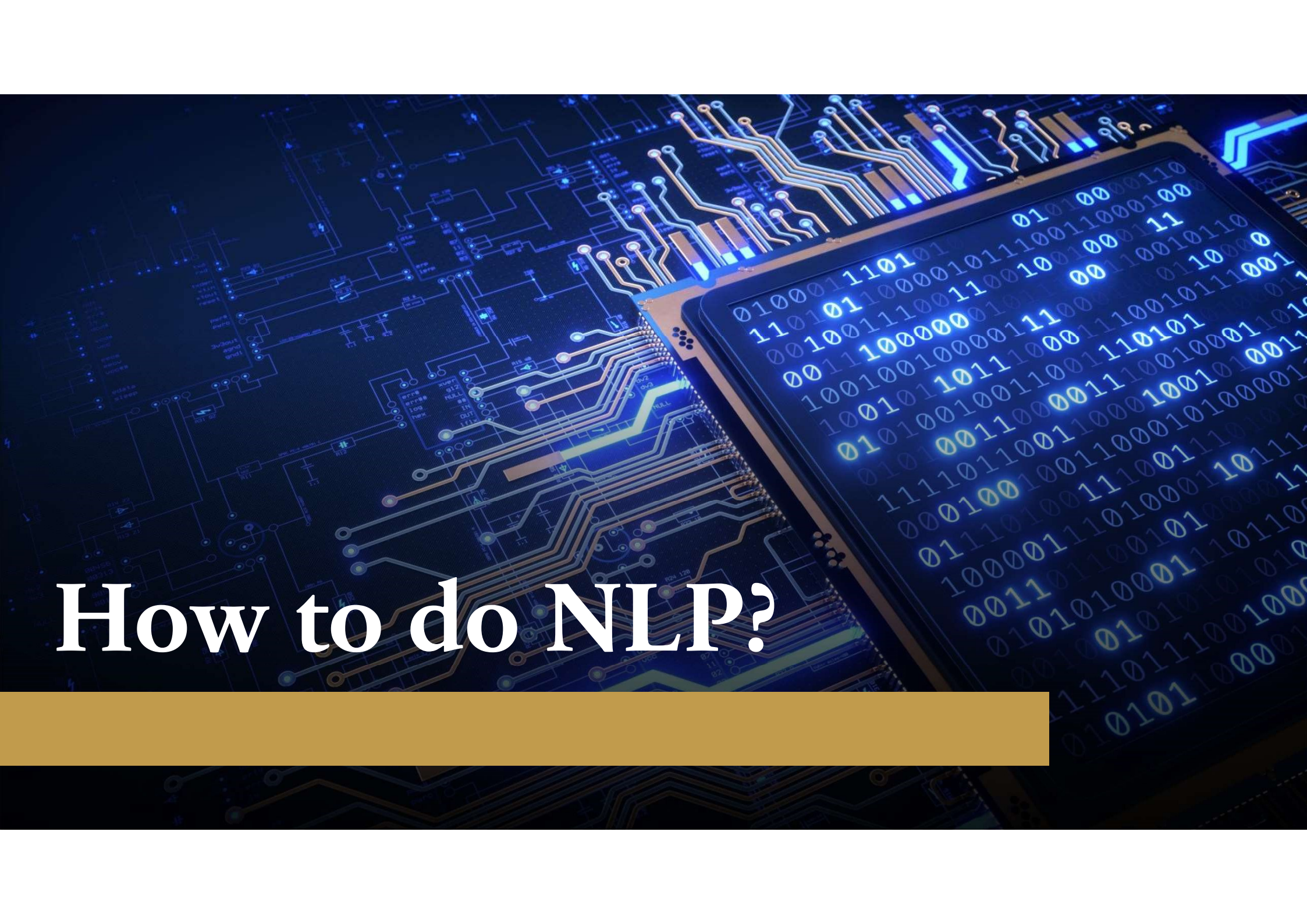
Situation analysis
(customers, competition)

Marketing strategy
(segmentation, target,
positioning)

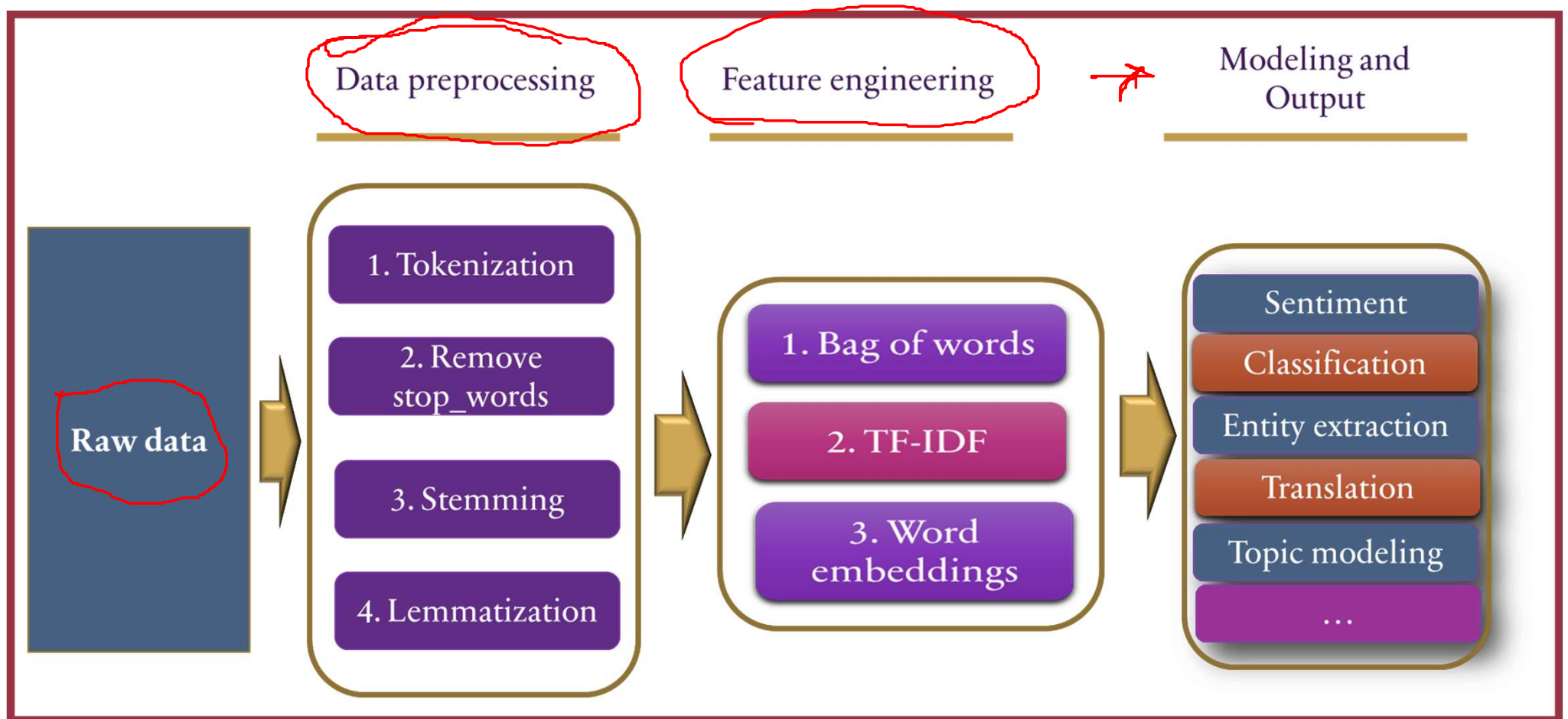
Marketing Mix
(e.g., product quality,
pricing)

Post-purchase evaluations
(customer satisfaction,
complaints, intentions)

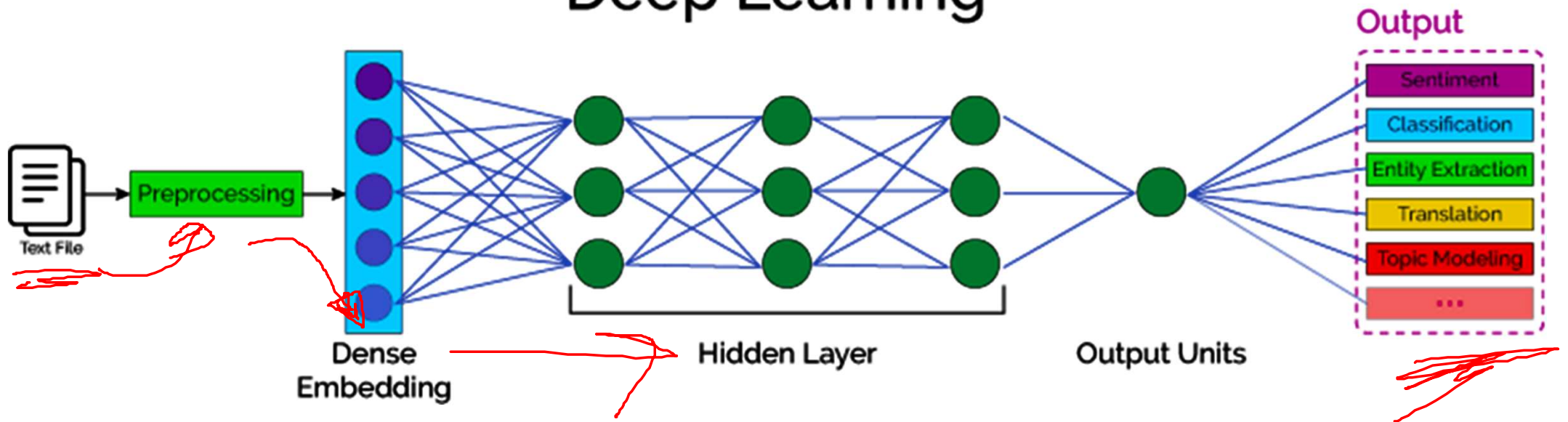


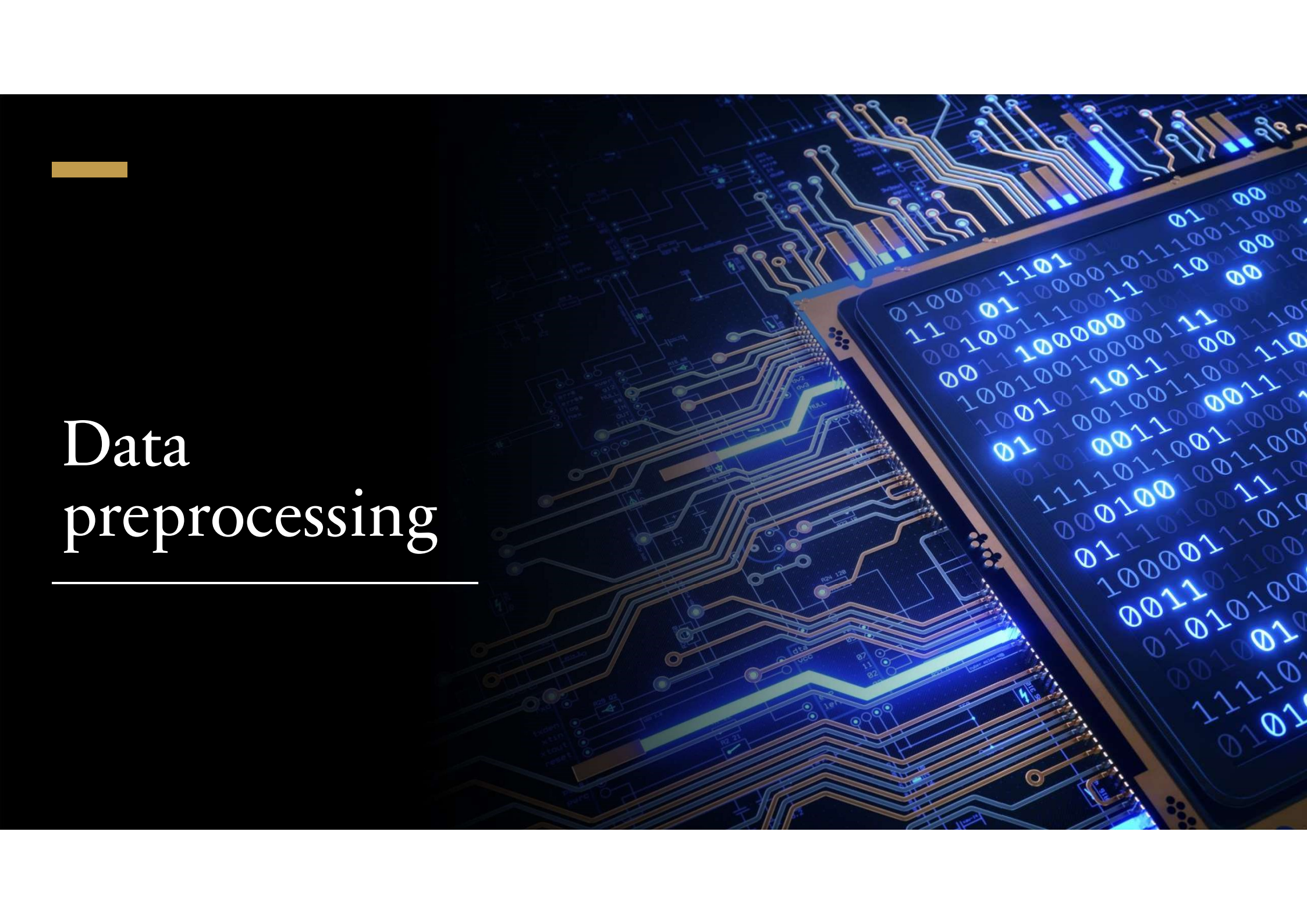


How to do NLP?

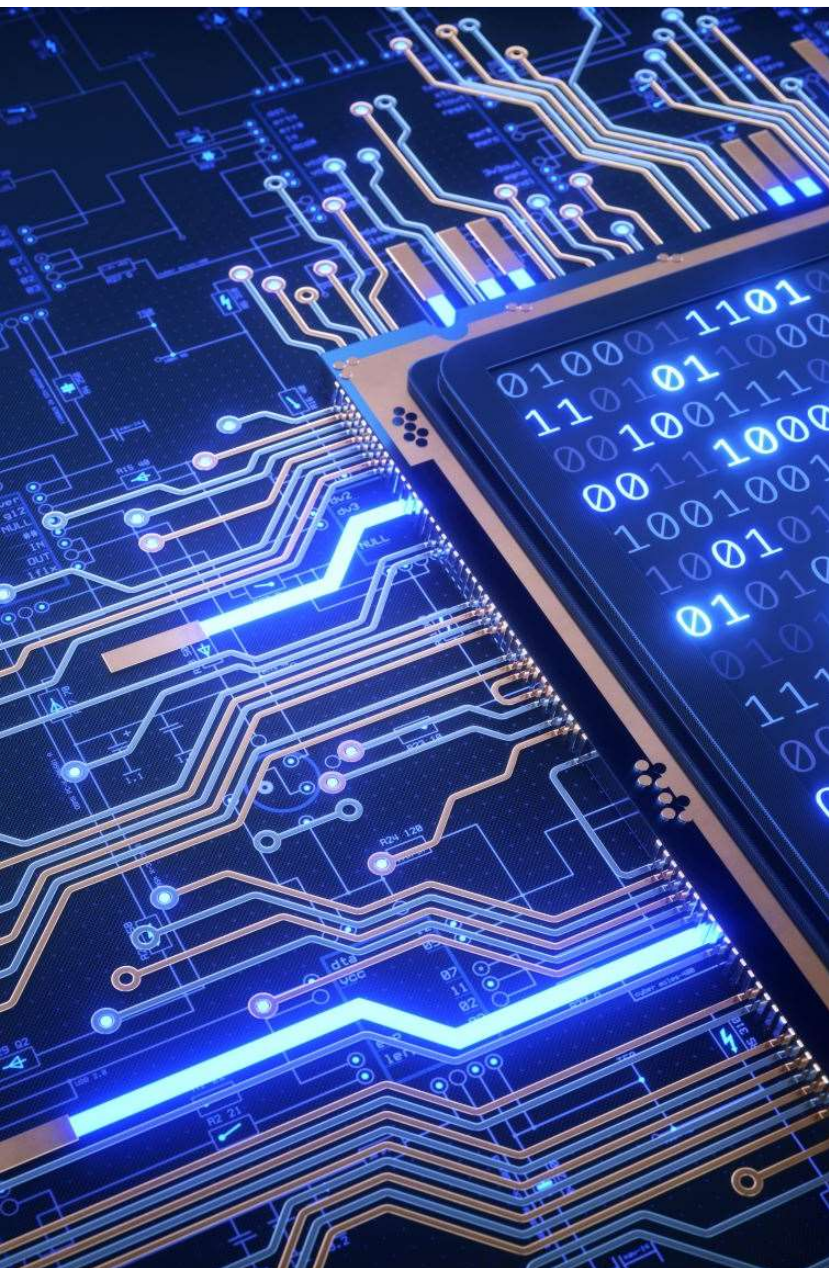


Deep Learning





Data preprocessing



- Tokenization :
 - is the act of breaking up a sequence of strings into pieces such as words, keywords, phrases, symbols and other elements called tokens. Tokens can be individual words, phrases or even whole sentences.



-
- The **bag-of-words** model is a simplifying representation used in **natural language processing** and information retrieval (IR). In this model, a text (such as a sentence or a document) is represented as the **bag** (multiset) of its **words**, disregarding grammar and even **word** order but keeping multiplicity.



```
ds('english')
```

[illegible]

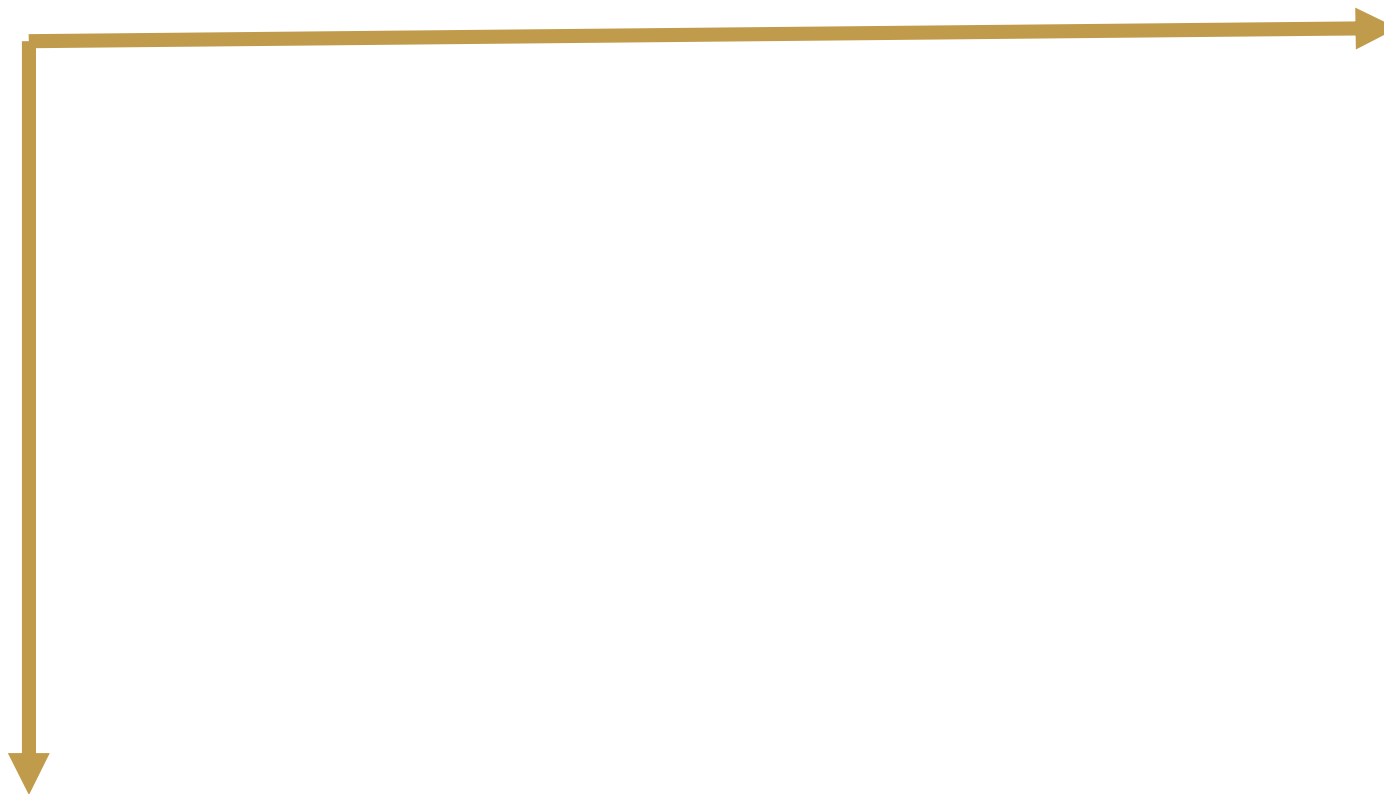
Stop words :

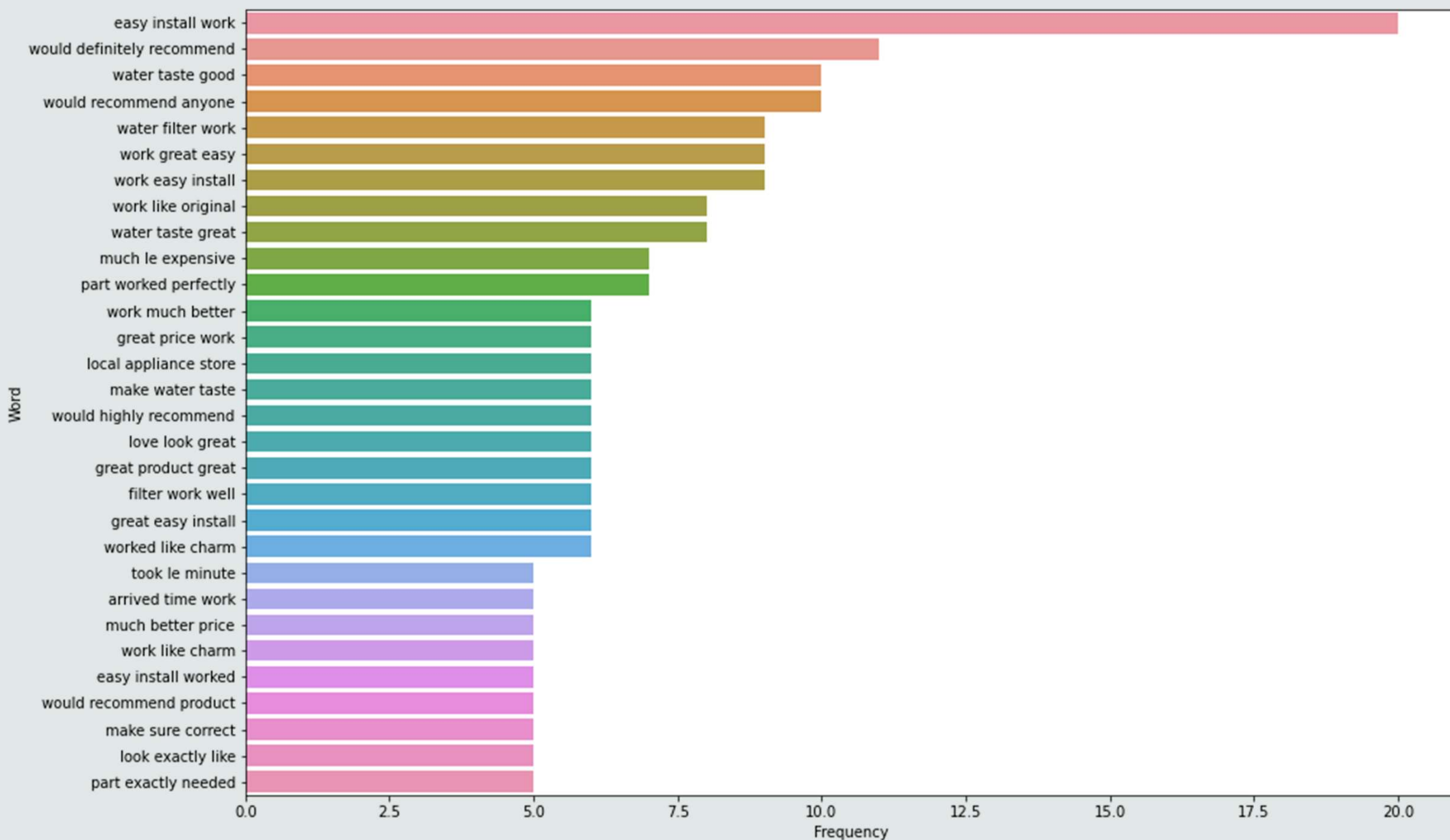
- Stop words means that it is a very common words in a language

(7000, 9457)

9457

(7000

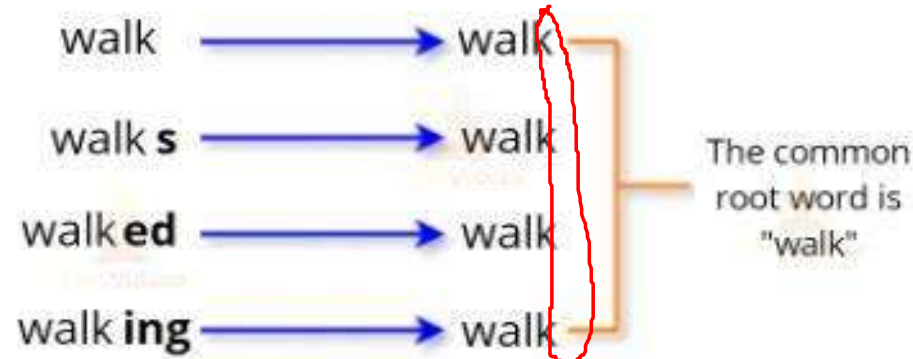




[illegible]

Stemming :

- Stemming is the process of reducing a word to its root known as a stem



30-40%
ending

- **Lemmatization**

- The process of grouping together the different inflected forms of a word so they can be analyzed as a single item.

are, am, => to be

Each word has a base form:

- has, had, have $\Rightarrow$ have
- cats, cat, cat's $\Rightarrow$ cat

The difference between Stemming and Lemmatization is that stem might not be an actual word whereas, lemma is an actual language word.

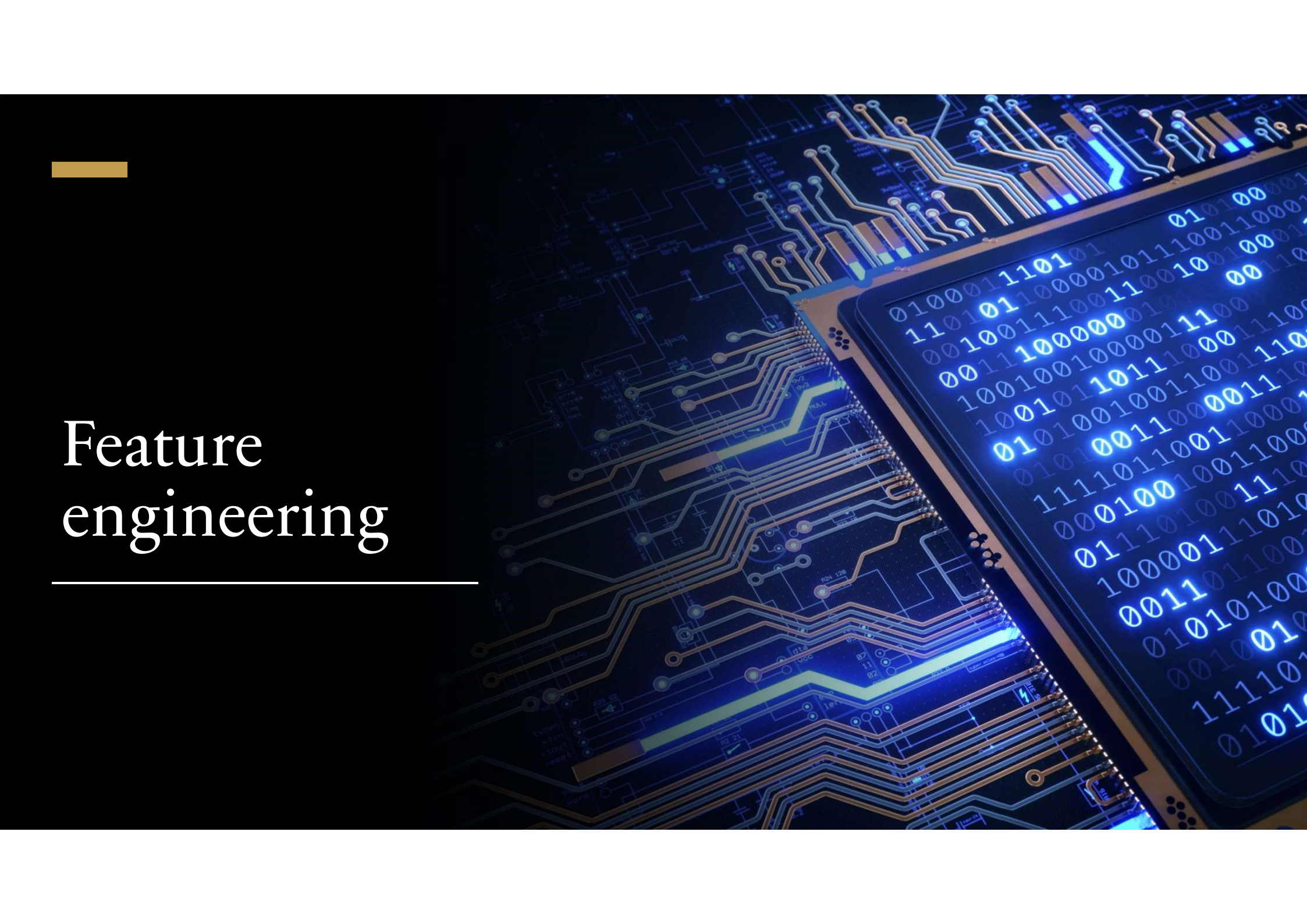
Part-of-speech tagging

Definition

- In corpus linguistics, **part-of-speech tagging** (POS tagging or PoS tagging or POST), also called **grammatical tagging** or **word-category disambiguation**, is
 - *the process of marking up a word in a text (corpus) as corresponding to a particular part of speech, based on both its definition and its context — i.e., its relationship with adjacent and related words in a phrase, sentence, or paragraph. A simplified form of this is commonly taught to school-age children, in the identification of words as nouns, verbs, adjectives, adverbs, etc.*

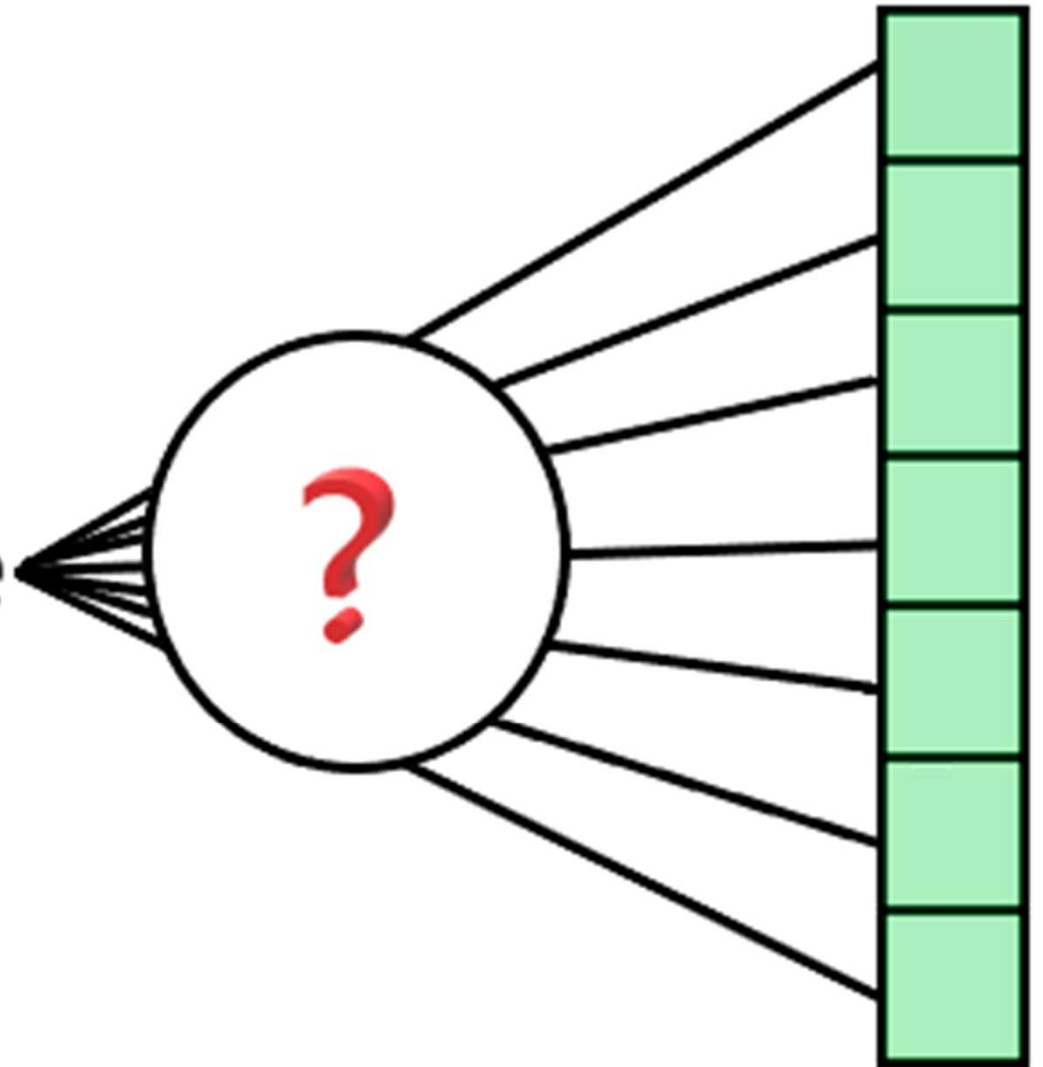
The POS tagger in the NLTK library outputs specific tags for certain words. The list of POS tags is as follows, with examples of what each POS stands for.

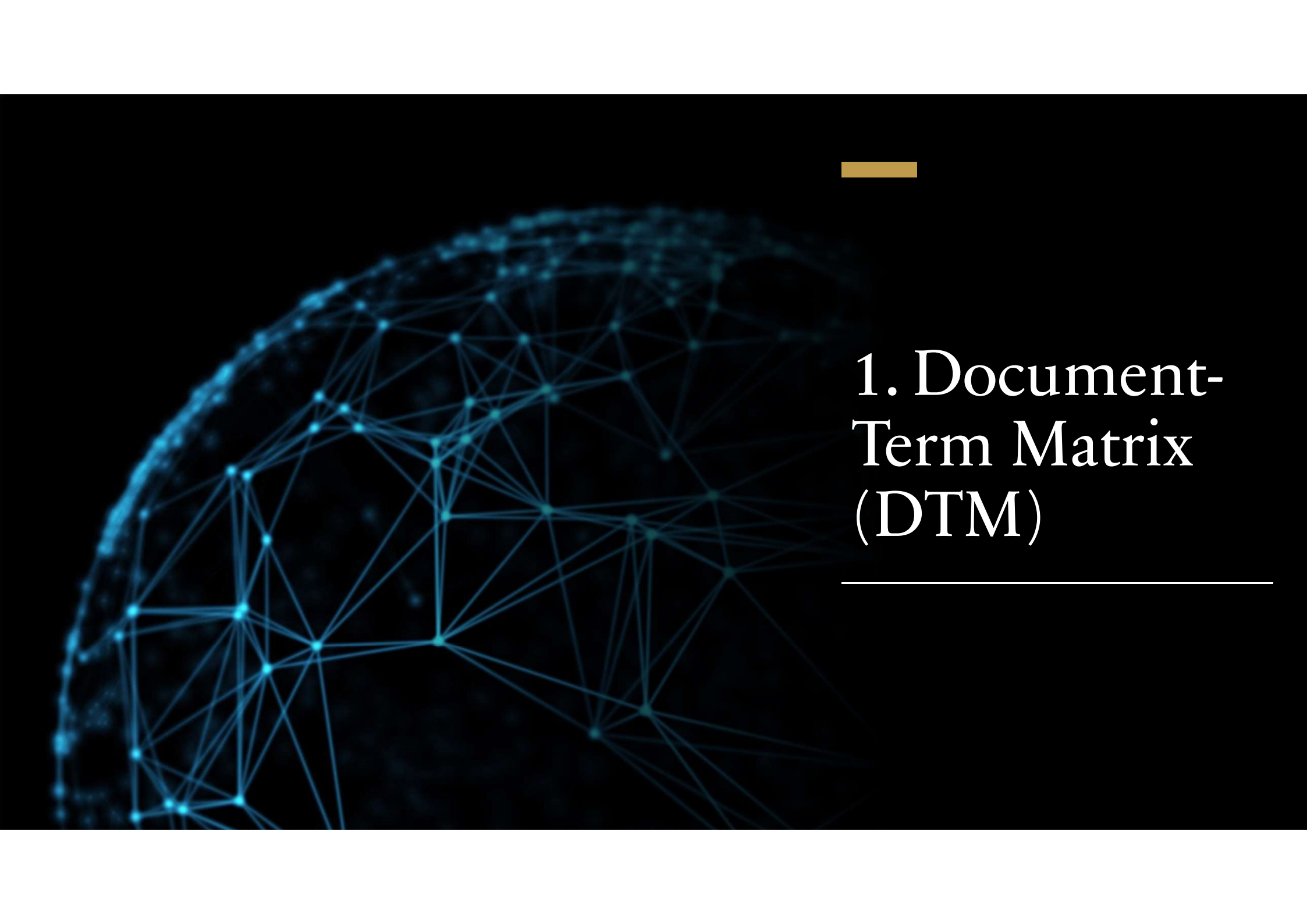
- CC coordinating conjunction
- CD cardinal digit
- DT determiner
- EX existential there (like: “there is” ... think of it like “there exists”)
- FW foreign word
- IN preposition/subordinating conjunction
- JJ adjective ‘big’
- JJR adjective, comparative ‘bigger’
- JJS adjective, superlative ‘biggest’
- LS list marker 1)
- MD modal could, will
- NN noun, singular ‘desk’
- NNS noun plural ‘desks’
- NNP proper noun, singular ‘Harrison’
- NNPS proper noun, plural ‘Americans’
- PDT predeterminer ‘all the kids’
- PRP personal pronoun I, he, she
- PRP\$ possessive pronoun my, his, hers
- RB adverb very, silently,
- RBR adverb, comparative better
- RBS adverb, superlative best
- RP particle give up
- TO, to go ‘to’ the store.
- UH interjection, errrrrrrm
- VB verb, base form take
- VBD verb, past tense took
- VBG verb, gerund/present participle taking
- VBN verb, past participle taken
- VBP verb, sing. present, non-3d take
- VBZ verb, 3rd person sing. present takes
- WDT wh-determiner which
- WP wh-pronoun who, what
- WP\$ possessive wh-pronoun whose
- WRB wh-abverb where, when



Feature engineering

This is a sentence

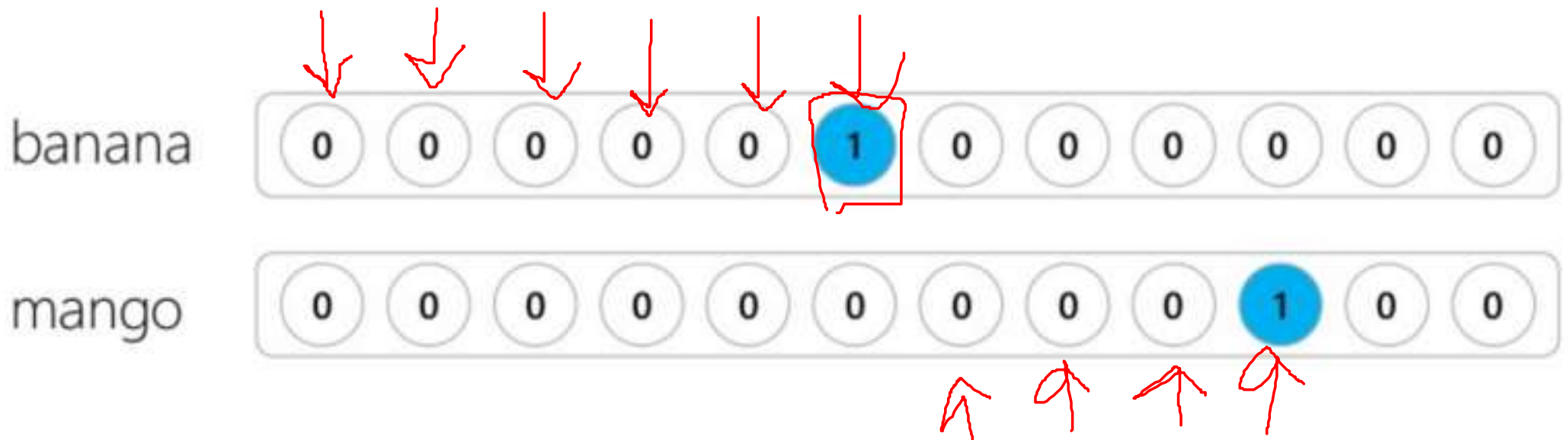




1. Document- Term Matrix (DTM)

Bag of words and the Document-Term-Matrix

- **Bag of Words** or BoW vector representations are the most common used traditional vector representation. Each word or n-gram is linked to a vector index and marked as 0 or 1 depending on whether it occurs in a given document.



Document Term Matrix

- Term frequency for multiple documents
- "Count Vectorizer" in scikit-learn

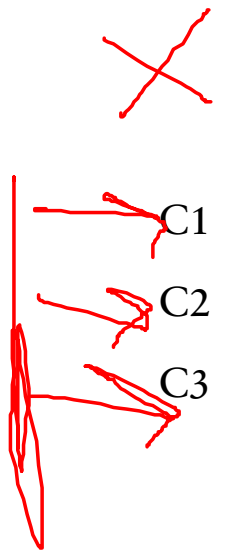
	Document 1	Document 2	Document 3	Document 4	Document 5	Document 6	Document 7	Document 8
Term(s) 1	10	0	1	0	0	0	0	2
Term(s) 2	0	2	0	0	0	18	0	2
Term(s) 3	0	0	0	0	0	0	0	2
Term(s) 4	6	0	0	4	6	0	0	0
Term(s) 5	0	0	0	0	0	0	0	2
Term(s) 6	0	0	1	0	0	1	0	0
Term(s) 7	0	1	8	0	0	0	0	0
Term(s) 8	0	0	0	0	0	3	0	0

Word Vector (Passage Vector)

Document Vector

~~Token 1~~ Token 2 Token 3

1	0	0
0	D	1
D	1	1





One challenge with bag of word representations is that they don't encode any information with regards to the meaning of a given word.

In BoW word occurrences are **evenly weighted independently of how frequently or what context they occur**. However, in most NLP tasks some words are more relevant than others.

Raw data

Preprocessing

NLTK



SKLEARN



Words and their count (DTM, Viz...)

PREPROCESSOR

- I, am, very, satisfied
- The product is very good
- They are very kind

- I (1)
- Am (1)
- Is (1)
- Are (1)
- Satisfied (1), ...

- I, am, very, satisfied
- The product is very good
- They are very kind

- I (1)
- To be (3)
- Satisfied (1), ...



2. TF-IDF

- **TF-IDF**, short for **term frequency–inverse document frequency**, is a numerical statistic that is intended *to reflect how important a word or n-gram is to a document in a collection or corpus*. They provide some weighting to a given word based on the context it occurs.
- The TF–IDF value *increases proportionally to the number of times a word appears in a document* and is *offset by the number of documents in the corpus that contain the word*, which helps to adjust for the fact that some words appear more frequently than others.

- TF-IDF stands for Term Frequency x Inverse document frequency.

- TF stands for Term Frequency - *how often do you are talking about appear in the document?* how many times the keyword “organic” appears in product description.

- IDF stands for Inverse Document Frequency - *how rare it is for a document to have this term.* it by taking the inverse of how many documents this term divided by total number of documents.

- What it will do is that if a term appears in a large number of documents, it will provide us with a low value. We use log of IDF to bring the value in a reasonable scale when the total number of documents is large.

TF-IDF signifie Term Frequency x Inverse document frequency.

- à quelle fréquence le terme dont vous parlez apparaît-il dans le document ?
- IDF signifie Inverse Document Frequency - à quel point il est rare qu'un document comporte ce terme.
- Si un terme apparaît dans un grand nombre de documents, nous obtiendrons une faible valeur IDF.

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

sentences = 30

TF(love) = 10

sentences = 8

IDF = $\log(30/8)$

TF(like, 34) = 23

sentences = 23

IDF = $\log(30/23)$

TF(awesome) = 2

sentences = 2

IDF = $\log(30/2)$

	Product A	Product B	Product C
Light ↗	1	0	0
Salt W	1	1	1
Sugar	0	1	0

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y
 df_x = number of documents containing x
 N = total number of documents

	Doc1	Doc2	Doc3	Doc4	Doc5	Doc6	Doc7	Doc8	# doc with term
Term(1)	10	0	1	0	0	0	0	2	3
Term(2)	0	2	0	0	0	18	0	2	3
Term(3)	0	0	0	0	0	0	0	2	1
Term(4)	6	0	0	4	6	0	0	0	3
Term(5)	0	0	0	0	0	0	0	2	1
Term(6)	0	0	1	0	0	1	0	0	2
Term(7)	0	1	0	0	0	0	0	0	1
Term(8)	0	0	0	0	0	3	0	0	1

TF= 1+log(count)

	Doc1	Doc2	Doc3	Doc4	Doc5	Doc6	Doc7	Doc8
Term(1)	2,00	0,00	1,00	0,00	0,00	0,00	0,00	1,30
Term(2)	0,00	1,30	0,00	0,00	0,00	2,26	0,00	1,30
Term(3)	0,00	0,00	0,00	0,00	0,00	0,00	0,00	1,30
Term(4)	1,78	0,00	0,00	1,60	1,78	0,00	0,00	0,00
Term(5)	0,00	0,00	0,00	0,00	0,00	0,00	0,00	1,30
Term(6)	0,00	0,00	1,00	0,00	0,00	1,00	0,00	0,00
Term(7)	0,00	1,00	0,00	0,00	0,00	0,00	0,00	0,00
Term(8)	0,00	0,00	0,00	0,00	0,00	1,48	0,00	0,00

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

TF-IDF	Doc1	Doc2	Doc3	Doc4	Doc5	Doc6	Doc7	Doc8
Term(1)	0,85	0,00	0,43	0,00	0,00	0,00	0,00	0,55
Term(2)	0,00	0,55	0,00	0,00	0,00	0,96	0,00	0,55
Term(3)	0,00	0,00	0,00	0,00	0,00	0,00	0,00	1,17
Term(4)	0,76	0,00	0,00	0,68	0,76	0,00	0,00	0,00
Term(5)	0,00	0,00	0,00	0,00	0,00	0,00	0,00	1,17
Term(6)	0,00	0,00	0,60	0,00	0,00	0,60	0,00	0,00
Term(7)	0,00	0,90	0,00	0,00	0,00	0,00	0,00	0,00
Term(8)	0,00	0,00	0,00	0,00	0,00	1,33	0,00	0,00

Standardized TF-IDF

$$\frac{\text{TF-IDF}(I)}{\sqrt{[\text{TF-IDF}(1)]^2 + [\text{TF-IDF}(2)]^2 + \dots + [\text{TF-IDF}(p)]^2}}$$

	Doc1	Doc2	Doc3	Doc4	Doc5	Doc6	Doc7	Doc8
Term(1)	0,75	0,00	0,58	0,00	0,00	0,00	0,00	0,30
Term(2)	0,00	0,52	0,00	0,00	0,00	0,55	0,00	0,30
Term(3)	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,64
Term(4)	0,66	0,00	0,00	1,00	1,00	0,00	0,00	0,00
Term(5)	0,00	0,00	0,00	0,00	0,00	0,00	0,00	0,64
Term(6)	0,00	0,00	0,82	0,00	0,00	0,34	0,00	0,00
Term(7)	0,00	0,85	0,00	0,00	0,00	0,00	0,00	0,00
Term(8)	0,00	0,00	0,00	0,00	0,00	0,76	0,00	0,00

Cosine similarity...

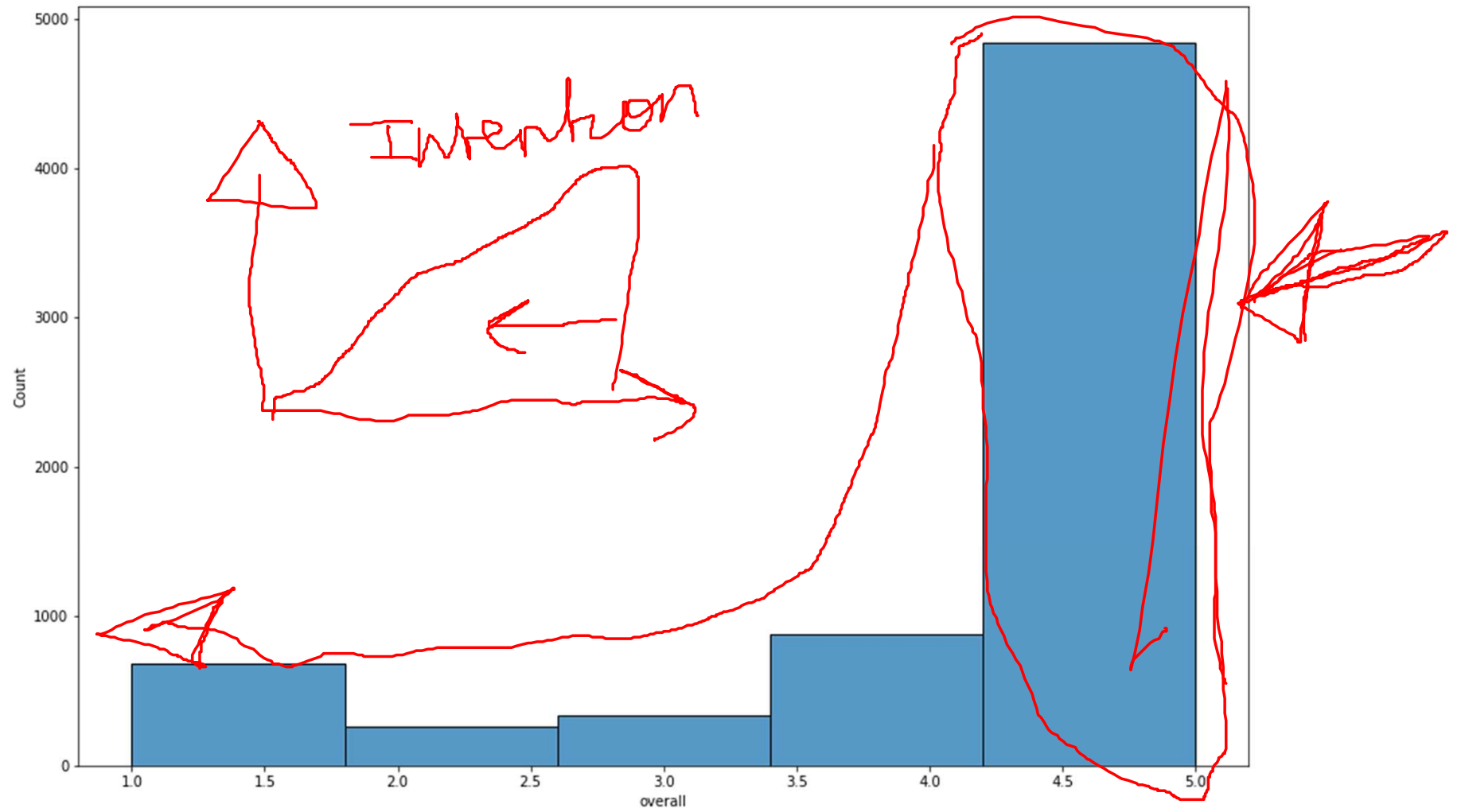
For length-normalized vectors, the cosine similarity is just a dot product (or scalar product)

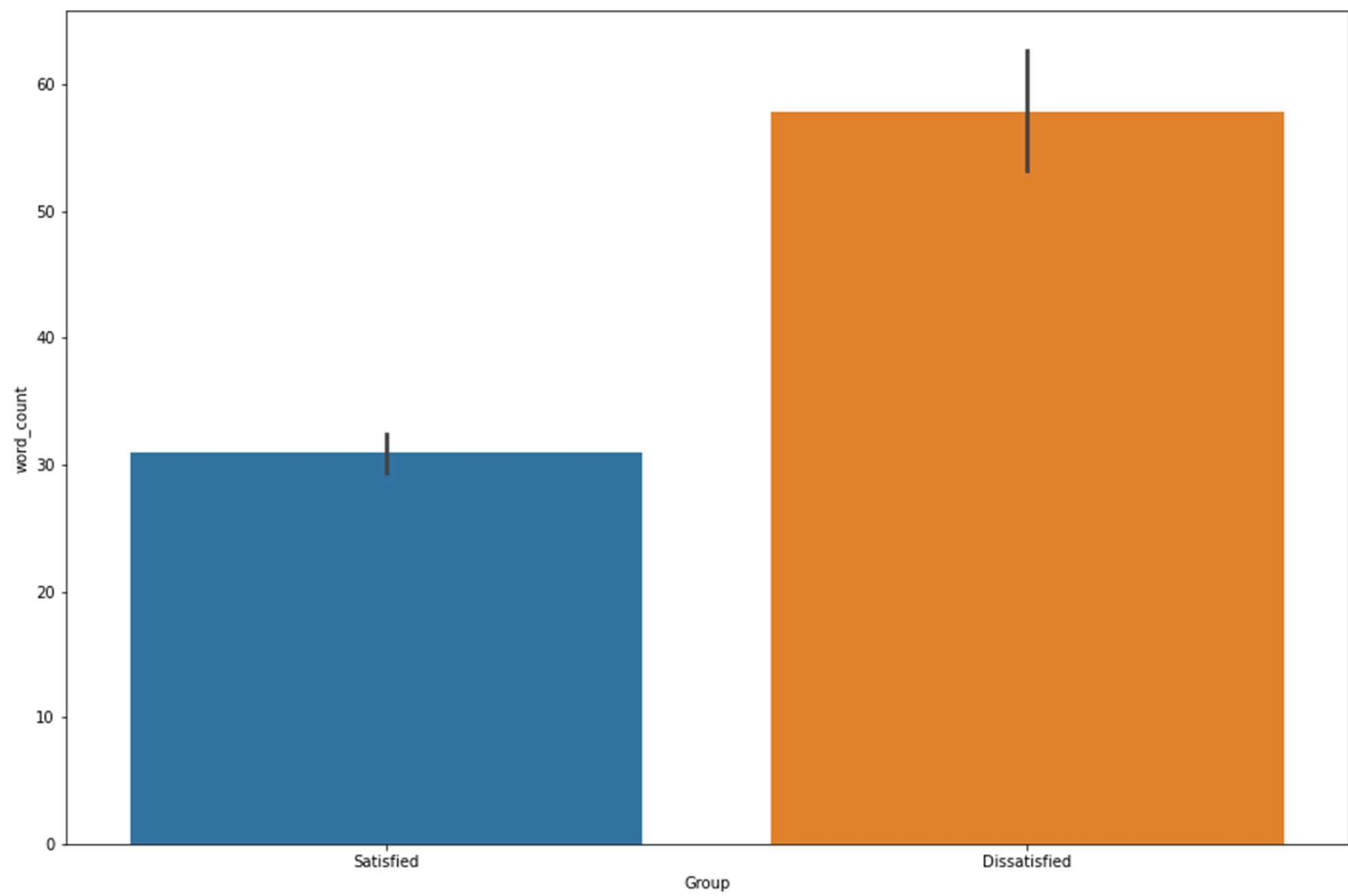
$$\text{Cos}(q, d) = \bar{q} \times \bar{d} = \sum_{i=1}^{|V|} q_i \times d_i$$

	Doc1	Doc2	Product
Term(1)	0,75	0,00	0,00
Term(2)	0,00	0,52	0,00
Term(3)	0,00	0,00	0,00
Term(4)	0,66	0,00	0,00
Term(5)	0,00	0,00	0,00
Term(6)	0,00	0,00	0,00
Term(7)	0,00	0,85	0,00
Term(8)	0,00	0,00	0,00
			0,00

	Doc1	Doc8	Product
Term(1)	0,75	0,30	0,23
Term(2)	0,00	0,30	0,00
Term(3)	0,00	0,64	0,00
Term(4)	0,66	0,00	0,00
Term(5)	0,00	0,64	0,00
Term(6)	0,00	0,00	0,00
Term(7)	0,00	0,00	0,00
Term(8)	0,00	0,00	0,00
			0,23

- However even though TF-IDF BoW representations provide weights to different words they are unable to capture the word meaning.
- As the famous linguist J. R. Firth said in 1935, “*The complete meaning of a word is always contextual, and no study of meaning apart from context can be taken seriously.*”





yet
Good
milk
best
French
found
ants
arrived
away
Amazon
crushed
years
using
sleep
Box
got
Market
put
price
yogurt
bas
lowest
open
nWe
tiny
five
places
use
coffee



3. Word embeddings



**Can words be integrated
into Machine Learning
algorithms? No, not
directly**

*We must find a way to encode the words
so that the machine understands
Encode = convert text to numbers*

Example

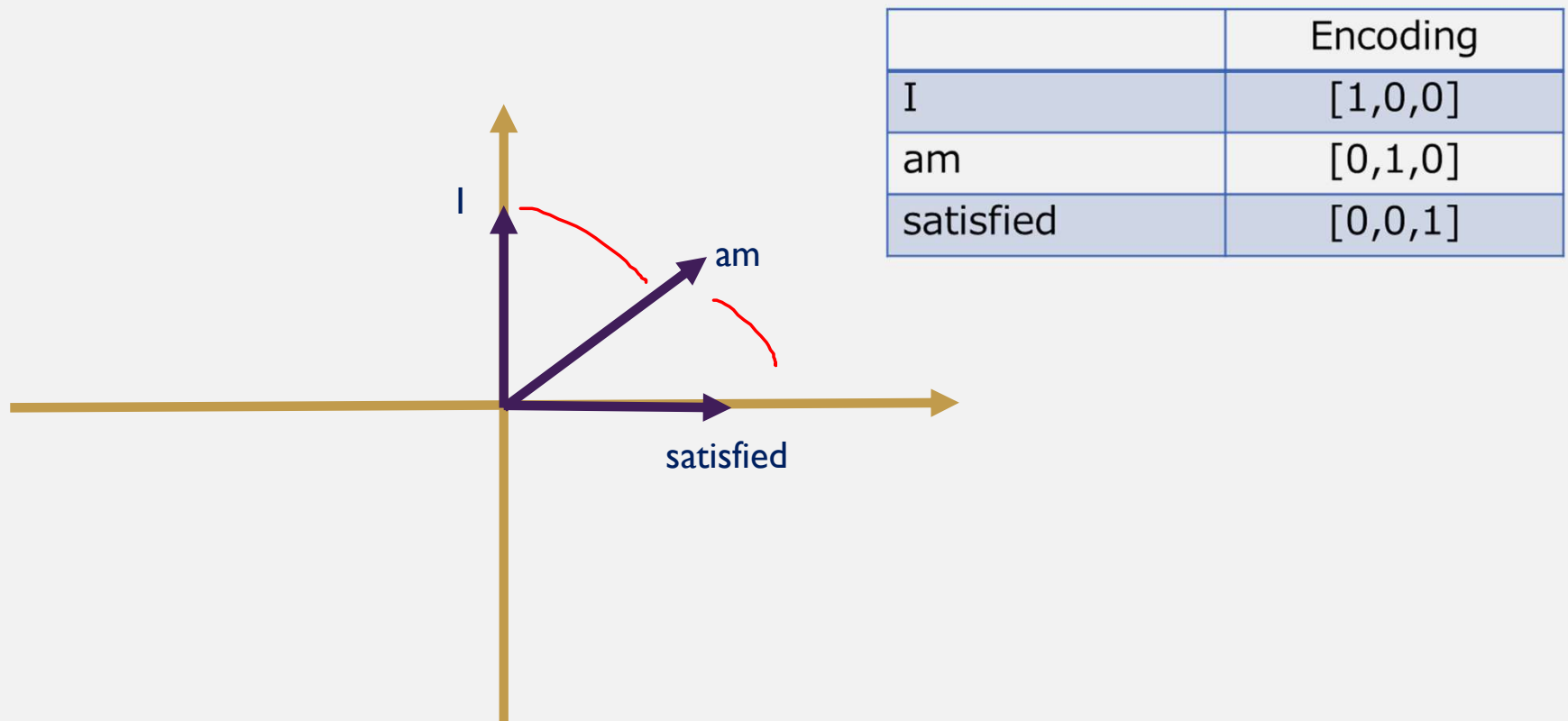
	I am satisfied		
	I	am	satisfied
I	1	0	0
am	0	1	0
satisfied	0	0	1



One-hot encoding

	Encoding
I	<u>[1,0,0]</u>
am	<u>[0,1,0]</u>
satisfied	<u>[0,0,1]</u>

Problem with this form of encoding: we can't compute the correlation/similarity between word vectors



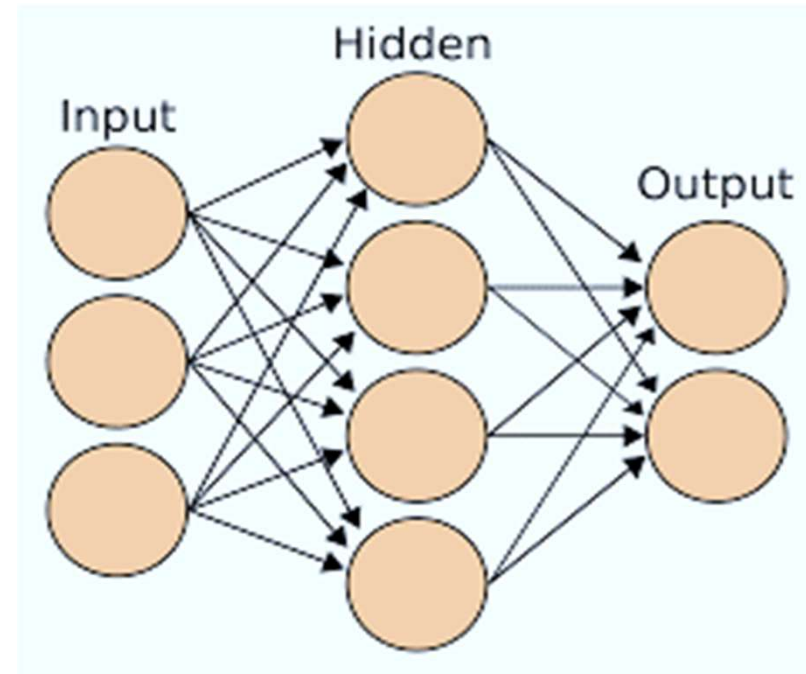
One way to calculate this similarity is to use the Word Embeddings method - the similarity is determined by the surrounding of the word.

I/am/ satisfied/ very/ delighted

Vocabulary
size = 5
words

Word	One-hot encoding	Word	One-hot encoding
I	[1,0,0,0,0]	am	[0,1,0,0,0]
I	[1,0,0,0,0]	satisfied	[0,0,1,0,0]
am	[0,1,0,0,0]	satisfied	[0,0,1,0,0]
am	[0,1,0,0,0]	I	[1,0,0,0,0]
satisfied	[0,0,1,0,0]	I	[1,0,0,0,0]
satisfied	[0,0,1,0,0]	am	[0,1,0,0,0]
I	[1,0,0,0,0]	am	[0,1,0,0,0]
I	[1,0,0,0,0]	very	[0,0,0,1,0]
I	[1,0,0,0,0]	delighted	[0,0,0,0,1]
am	[0,1,0,0,0]	very	[0,0,0,1,0]
am	[0,1,0,0,0]	delighted	[0,0,0,0,1]
am	[0,1,0,0,0]	I	[1,0,0,0,0]
very	[0,0,0,1,0]	I	[1,0,0,0,0]
very	[0,0,0,1,0]	am	[0,1,0,0,0]
very	[0,0,0,1,0]	delighted	[0,0,0,0,1]
delighgted	[0,0,0,0,1]	very	[0,0,0,1,0]
delighted	[0,0,0,0,1]	am	[0,1,0,0,0]
delighted	[0,0,0,0,1]	I	[1,0,0,0,0]

Input	Output
[1,0,0,0,0]	[0,1,0,0,0]
[1,0,0,0,0]	[0,0,1,0,0]
[0,1,0,0,0]	[0,0,1,0,0]
[0,1,0,0,0]	[1,0,0,0,0]
[0,0,1,0,0]	[1,0,0,0,0]
[0,0,1,0,0]	[0,1,0,0,0]
[1,0,0,0,0]	[0,1,0,0,0]
[1,0,0,0,0]	[0,0,0,1,0]
[1,0,0,0,0]	[0,0,0,0,1]
[0,1,0,0,0]	[0,0,0,1,0]
[0,1,0,0,0]	[0,0,0,0,1]
[0,1,0,0,0]	[1,0,0,0,0]
[0,0,0,1,0]	[1,0,0,0,0]
[0,0,0,1,0]	[0,1,0,0,0]
[0,0,0,1,0]	[0,0,0,0,1]
[0,0,0,0,1]	[0,0,0,1,0]
[0,0,0,0,1]	[0,1,0,0,0]
[0,0,0,0,1]	[1,0,0,0,0]



If the whole target column =1, how can we learn?

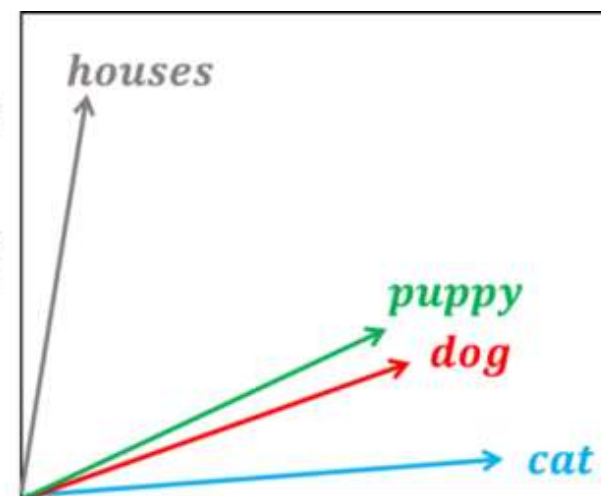
Input	Output	Target
[1,0,0,0,0]	[0,1,0,0,0]	1
[1,0,0,0,0]	[0,0,1,0,0]	1
[0,1,0,0,0]	[0,0,1,0,0]	1
[0,1,0,0,0]	[1,0,0,0,0]	1
[0,0,1,0,0]	[1,0,0,0,0]	1
[0,0,1,0,0]	[0,1,0,0,0]	1
[1,0,0,0,0]	[0,1,0,0,0]	1
[1,0,0,0,0]	[0,0,0,1,0]	1
[1,0,0,0,0]	[0,0,0,0,1]	1
[0,1,0,0,0]	[0,0,0,1,0]	1
[0,1,0,0,0]	[0,0,0,0,1]	1
[0,1,0,0,0]	[1,0,0,0,0]	1
[0,0,0,1,0]	[1,0,0,0,0]	1
[0,0,0,1,0]	[0,1,0,0,0]	1
[0,0,0,1,0]	[0,0,0,0,1]	1
[0,0,0,0,1]	[0,0,0,1,0]	1
[0,0,0,0,1]	[0,1,0,0,0]	1
[0,0,0,0,1]	[1,0,0,0,0]	1

- To remedy this, we need to introduce negative samples into our dataset - **samples of words that are not neighbors.**
- Our model must return 0 for these samples.
- We randomly sample words from our vocabulary

Word	Word	Target
I	am	1
I	satisfied	1
am	satisfied	1
am	I	1
satisfied	I	1
satisfied	am	1
<u>satisfied</u>	<u>delighted</u>	<u>0</u>
1	1	100
1	1	00
		00

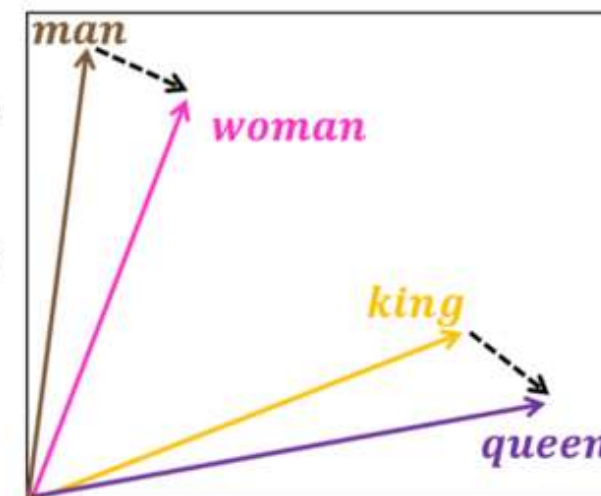
	d1	d2	d3	d4	d5	d6	d7
<i>dog</i> →	0.6	0.9	0.1	0.4	-0.7	-0.3	-0.2
<i>puppy</i> →	0.5	0.8	-0.1	0.2	-0.6	-0.5	-0.1
<i>cat</i> →	0.7	-0.1	0.4	0.3	-0.4	-0.1	-0.3
<i>houses</i> →	-0.8	-0.4	-0.5	0.1	-0.9	0.3	0.8

Dimensionality
reduction of
word
embeddings
from 7D to 2D



<i>man</i> →	0.6	-0.2	0.8	0.9	-0.1	-0.9	-0.7
<i>woman</i> →	0.7	0.3	0.9	-0.7	0.1	-0.5	-0.4
<i>king</i> →	0.5	-0.4	0.7	0.8	0.9	-0.7	-0.6
<i>queen</i> →	0.8	-0.1	0.8	-0.9	0.8	-0.5	-0.9

Dimensionality
reduction of
word
embeddings
from 7D to 2D



Word

Word embedding

Dimensionality
reduction

Visualization of word
embeddings in 2D

Word embeddings ...

- Word vectors are obtained by forming a shallow neural network (single hidden layer) on individual words in a corpus and submitting the surrounding words as the target to be predicted.
- By training a neural network on the data, we update the weights of the hidden layer, which we extract at the end as word vectors.
- Word Embeddings is a way of converting text into vectors. It provides a dense vector representation of words that, in addition, has a meaning.

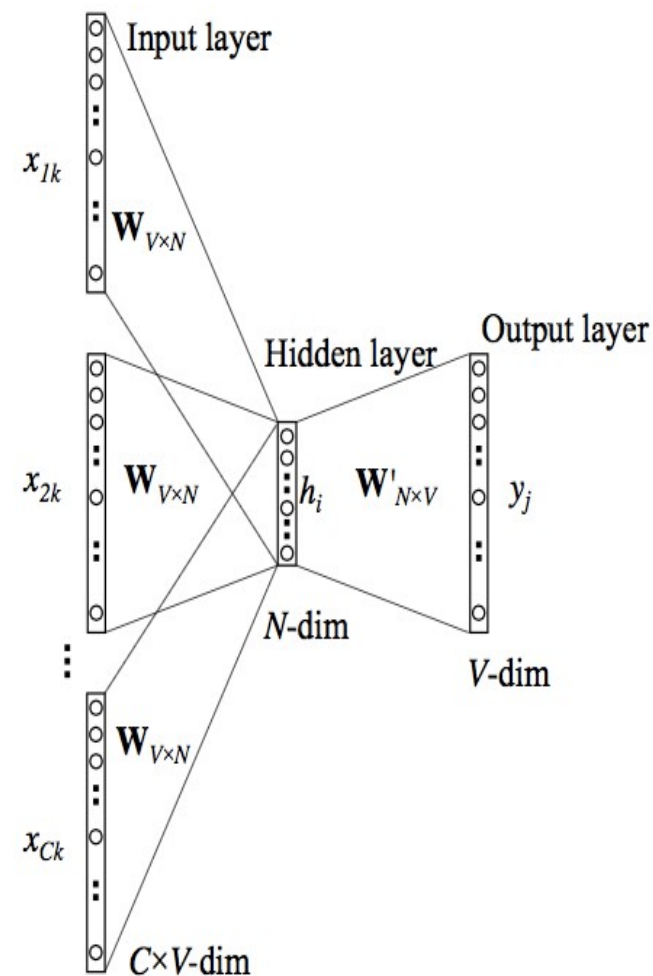
Word embeddings ...

- The semantic relationships between words are reflected in the distance and direction of the vectors.
 - Words are represented as fixed-length vectors, which are called embeddings.
 - The most interesting thing about it is that **word embeddings identify the similar words or words used in the same context by consumers when they talk about brands.**
 - More specifically, negative (positive) words and expressions are usually surrounded by similar negative (positive) words and expressions called the context.

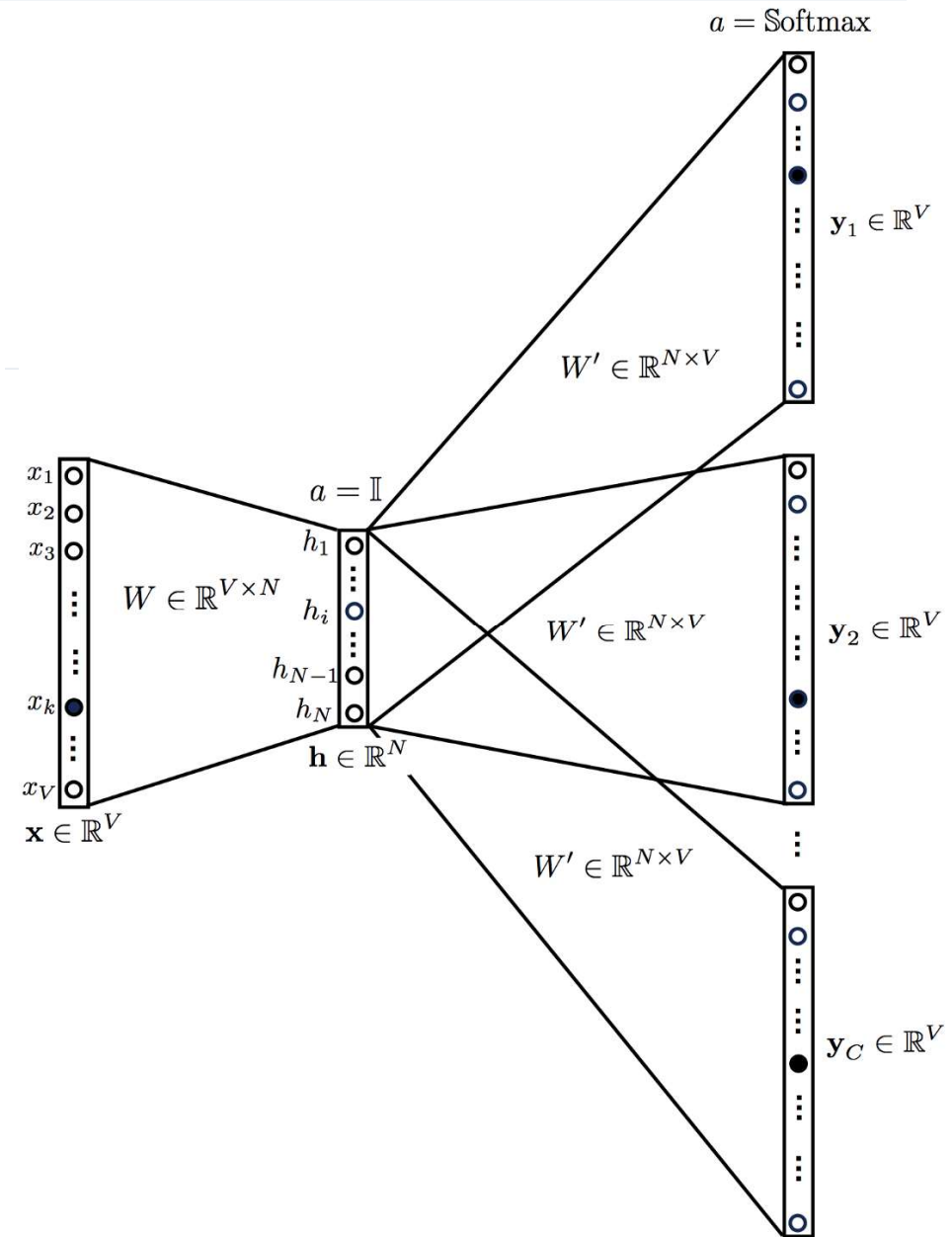
- **Word2Vec**

- Word2Vec is a predictive embedding model.
- There are two main Word2Vec architectures that are used to produce a distributed representation of words:

- Continuous bag-of-words (CBOW) - predict a word based on its context



- In the continuous skip-gram architecture, the model uses the *current word to predict the surrounding window of context words*.
- Continuous skip-gram weighs nearby context words more heavily than more distant context words. While order still is not captured each of the context vectors are weighed and compared independently vs CBOW which weighs against the average context.





CBOW is faster while skip-gram is slower but does a better job for infrequent words.

GloVe

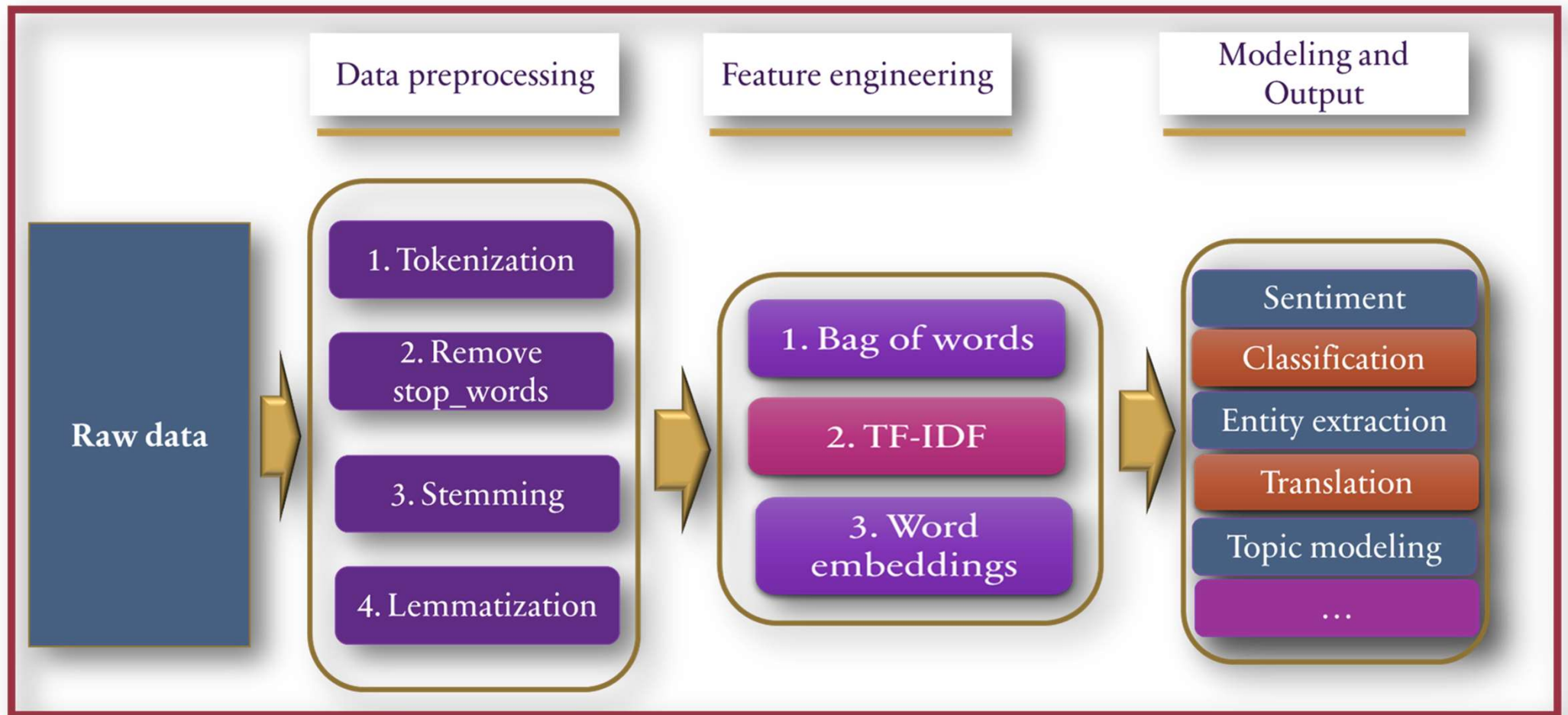
- Both CBOW and Skip-Grams are “predictive” models, in that they only take **local contexts** into account. Word2Vec does not take advantage of global context.
- GloVe embeddings by contrast leverage the same intuition behind the co-occurring matrix used distributional embeddings but uses neural methods to decompose the co-occurrence matrix into more expressive and dense word vectors.
- While GloVe vectors are faster to train, neither GloVe or Word2Vec has been shown to provide definitively better results rather they should both be evaluated for a given dataset.

FastText

- FastText, builds on Word2Vec by learning vector representations for each word and the n-grams found within each word. The values of the representations are then averaged into one vector at each training step. While this adds a lot of additional computation to training it enables word embeddings to encode sub-word information. FastText vectors have been shown to be more accurate than Word2Vec vectors by a number of different measures.

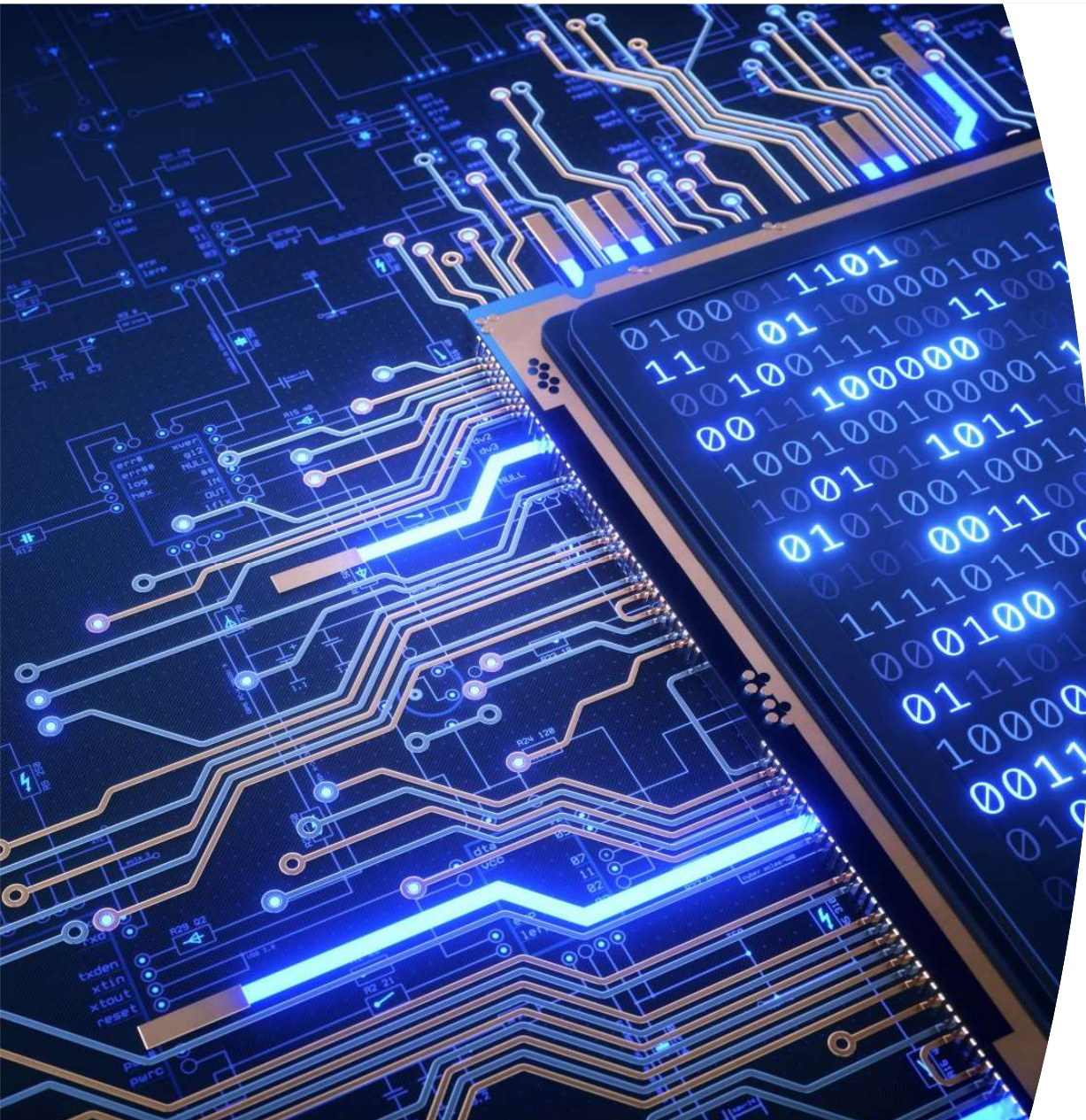


Modeling



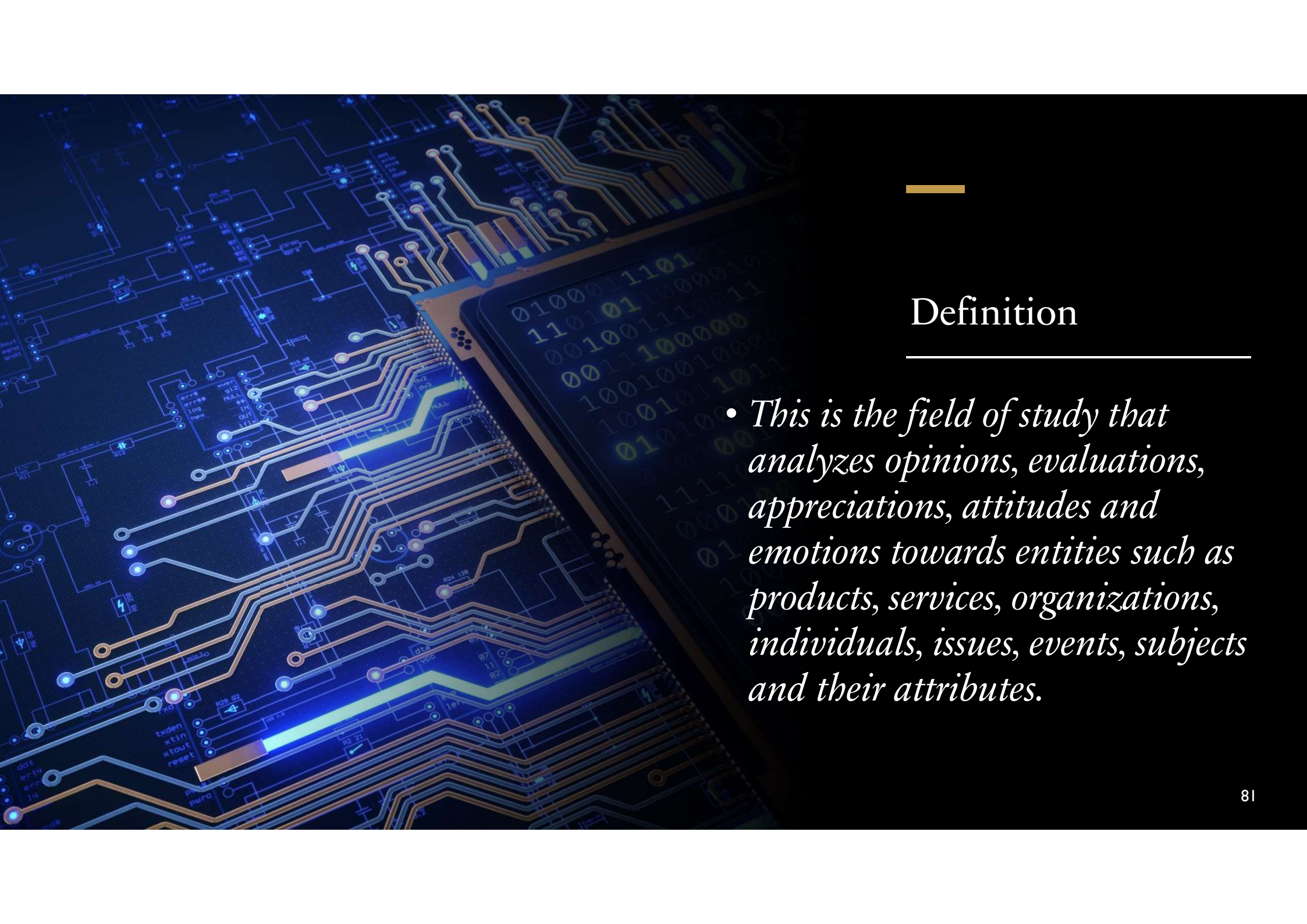


Sentiment analysis



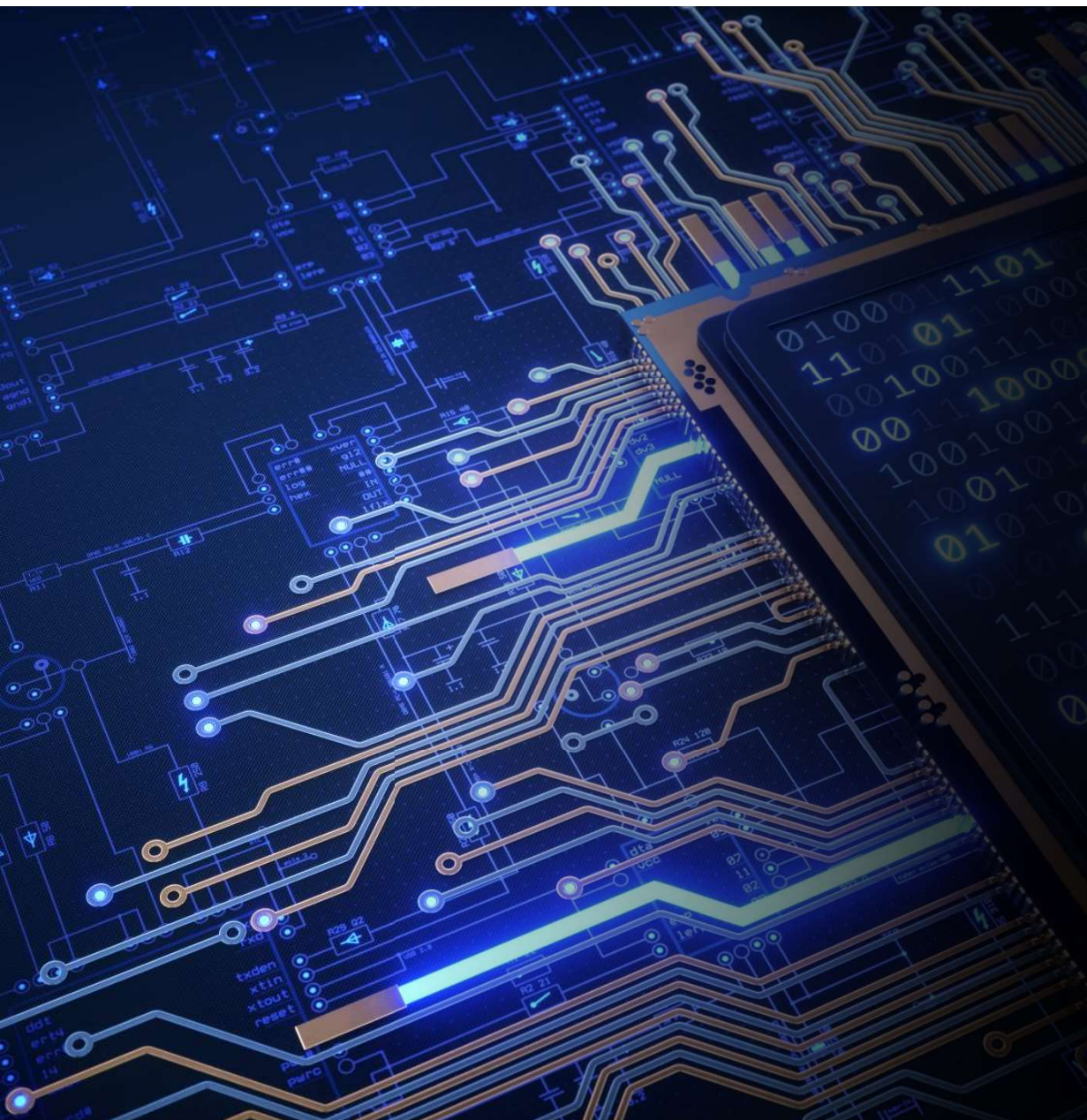
Definition

- Also called
 - *Opinion extraction*
 - *Opinion mining*
 - *Sentiment mining*
 - • *Subjectivity analysis*



Definition

- *This is the field of study that analyzes opinions, evaluations, appreciations, attitudes and emotions towards entities such as products, services, organizations, individuals, issues, events, subjects and their attributes.*



- 1. Holder (*source*) of the feeling
- 2. Target (*aspect*) of feeling
- 3. The feeling
 - The *Type*: love, hate, desire, etc.
 - The *polarity*: positive, negative, neutral
 - The *strength/level/intensity* of this attitude
- 4. Text containing the feeling
 - Entire sentence or document

- 
- In Marketing
 - Question: "Why don't consumers buy our product?"
 - We know the objective reasons: price, specifications, competition, etc.

Why?

- But we want to know the subjective reasons: "the design is in poor taste", "the customer service was condescending"...
- Misperceptions are also important.

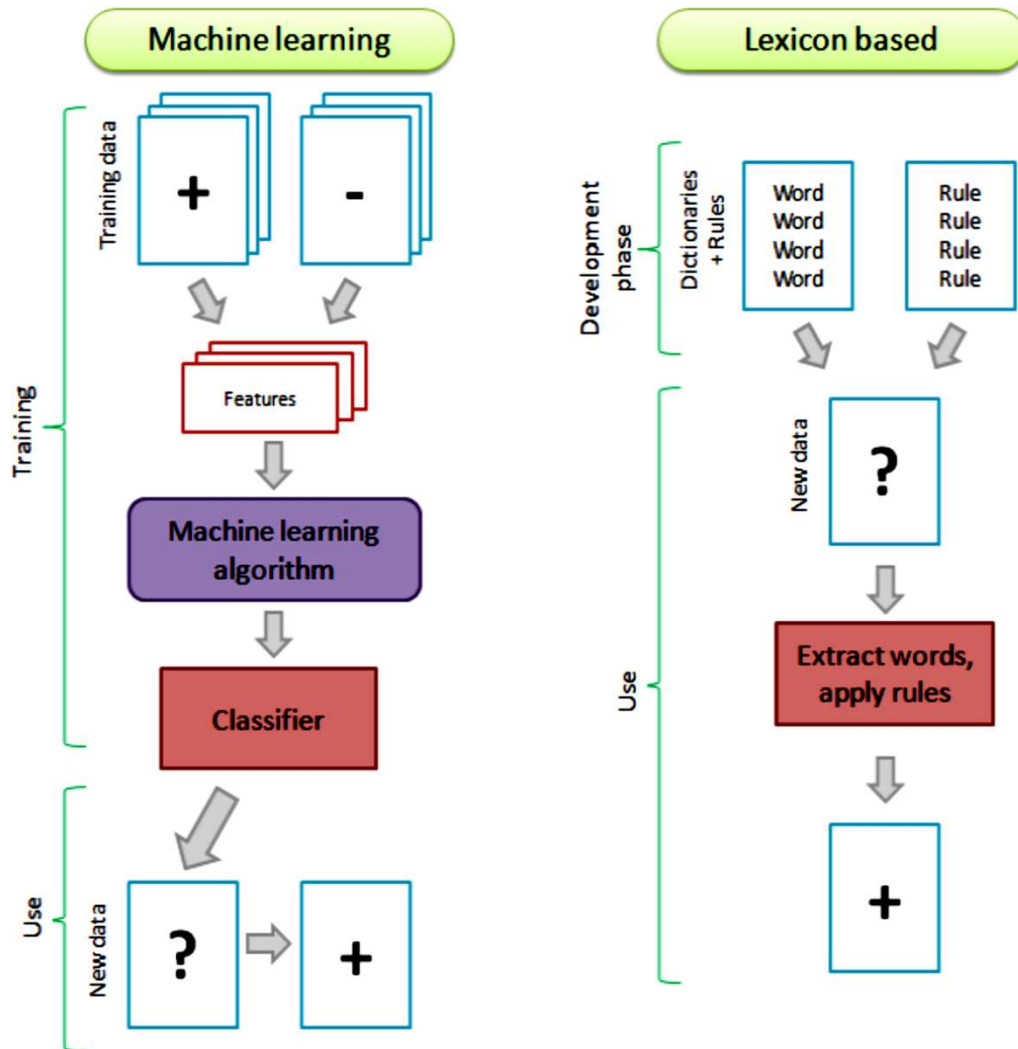


How?

Sentiment analysis at different levels

- Document level
 - The whole document is analyzed and classified as expressing a positive or negative view. At this level, the analysis considers that the entire document has only one opinion.
- Sentence level
 - It determines whether a given sentence has positive, negative, and neutral opinion. This level deals with each sentence as containing one opinion. The polarization of the whole document depends on the polarization of the sentence.
- Aspect level (feature-based sentiment analysis)
 - To find out what exactly people like or did not like this level is very important. At this level, the desired aspect is determined and then the polarity of this is set

Different types of methods

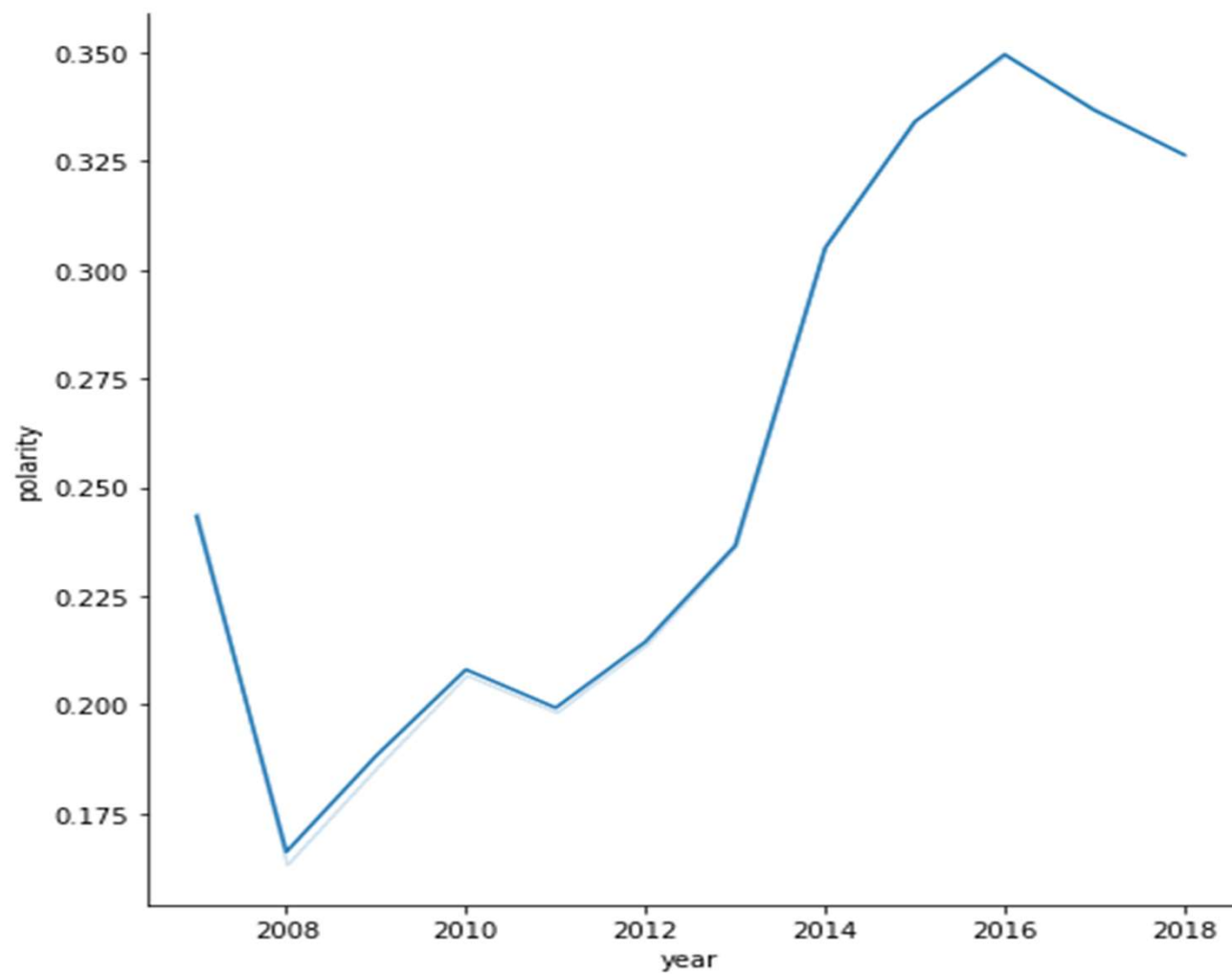


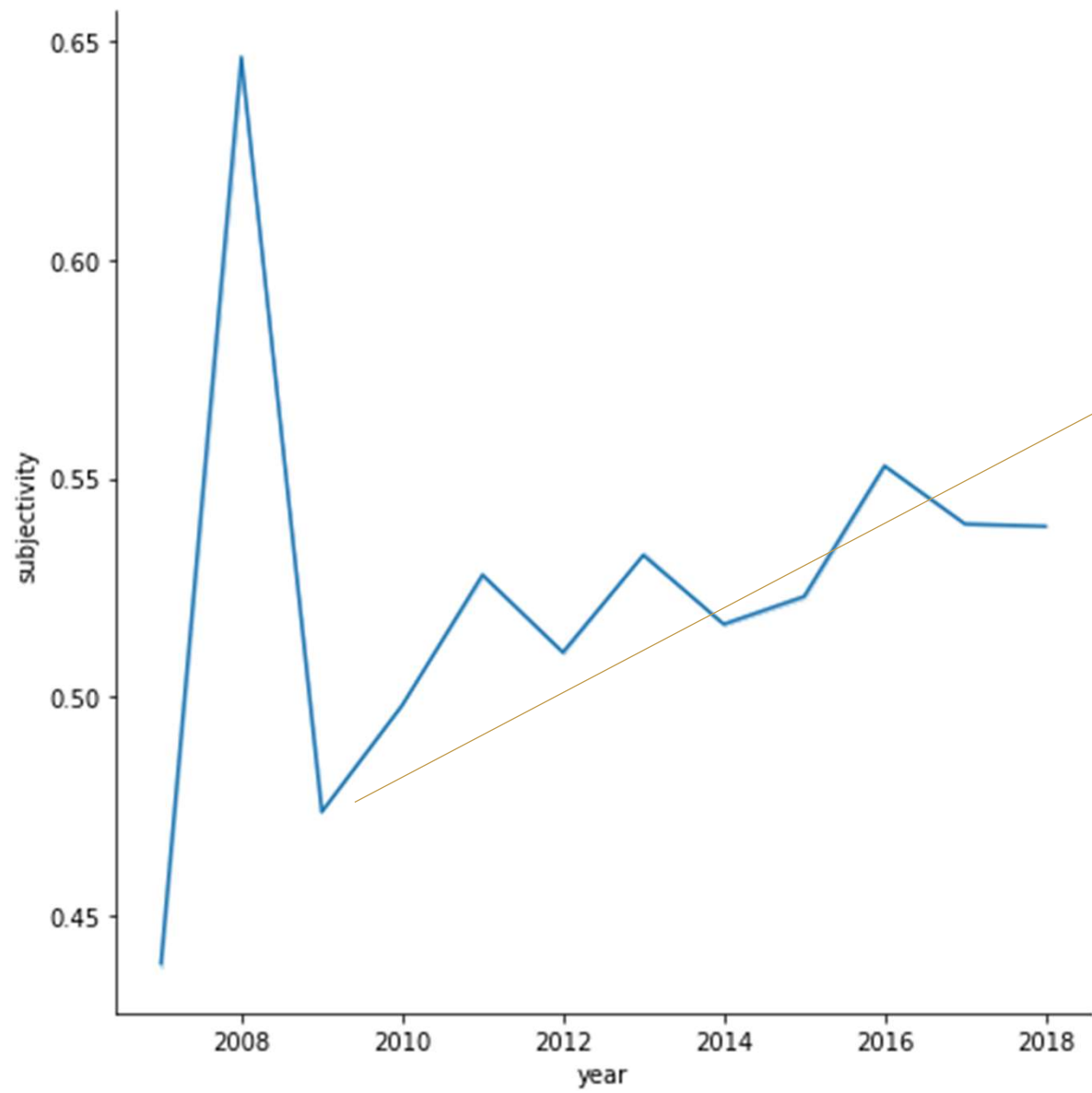
Machine learning methods:

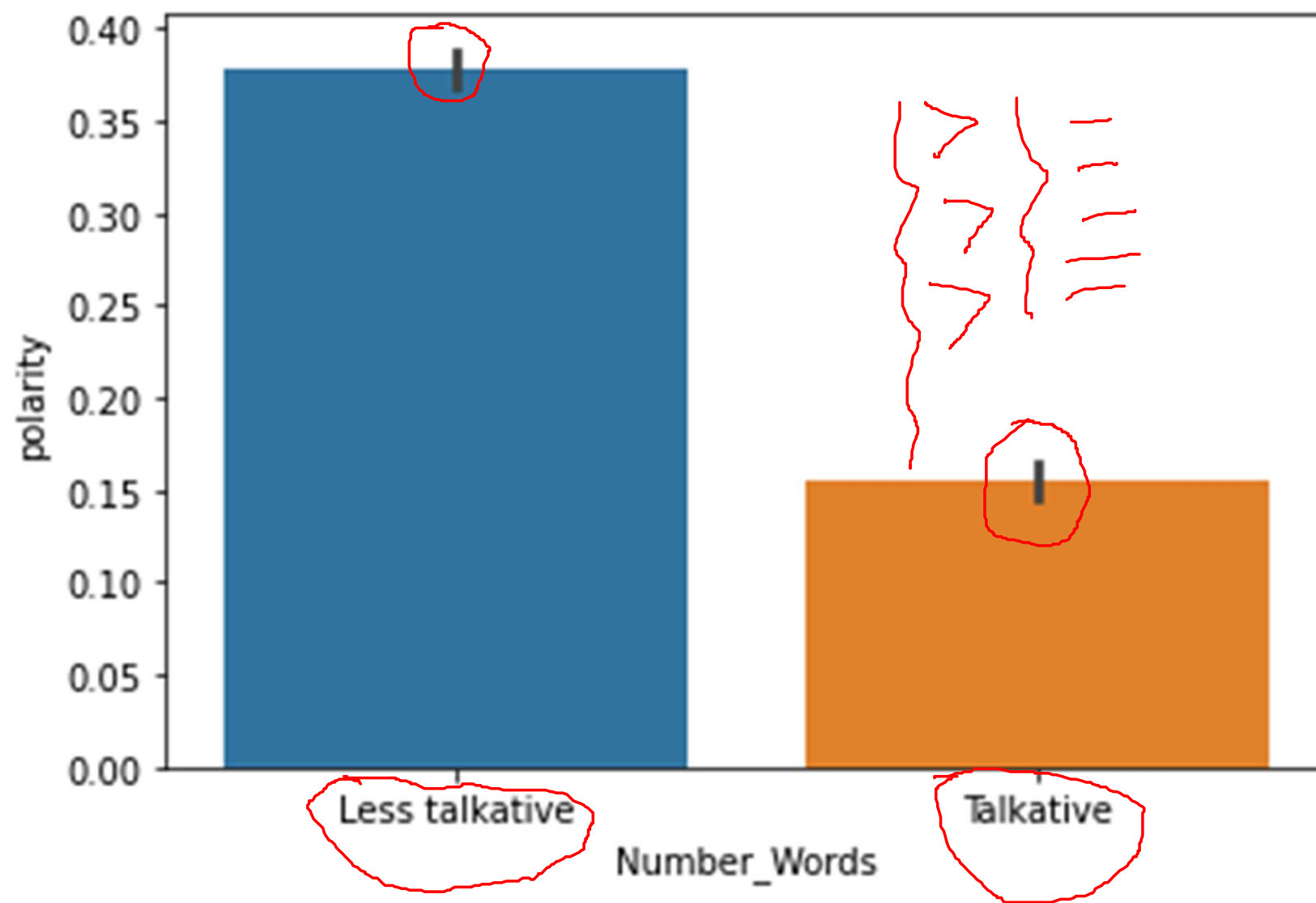
- a classifier is built that can determine the polarity of new texts. The classifier learns that certain characteristics distinguish a positive from a negative text.

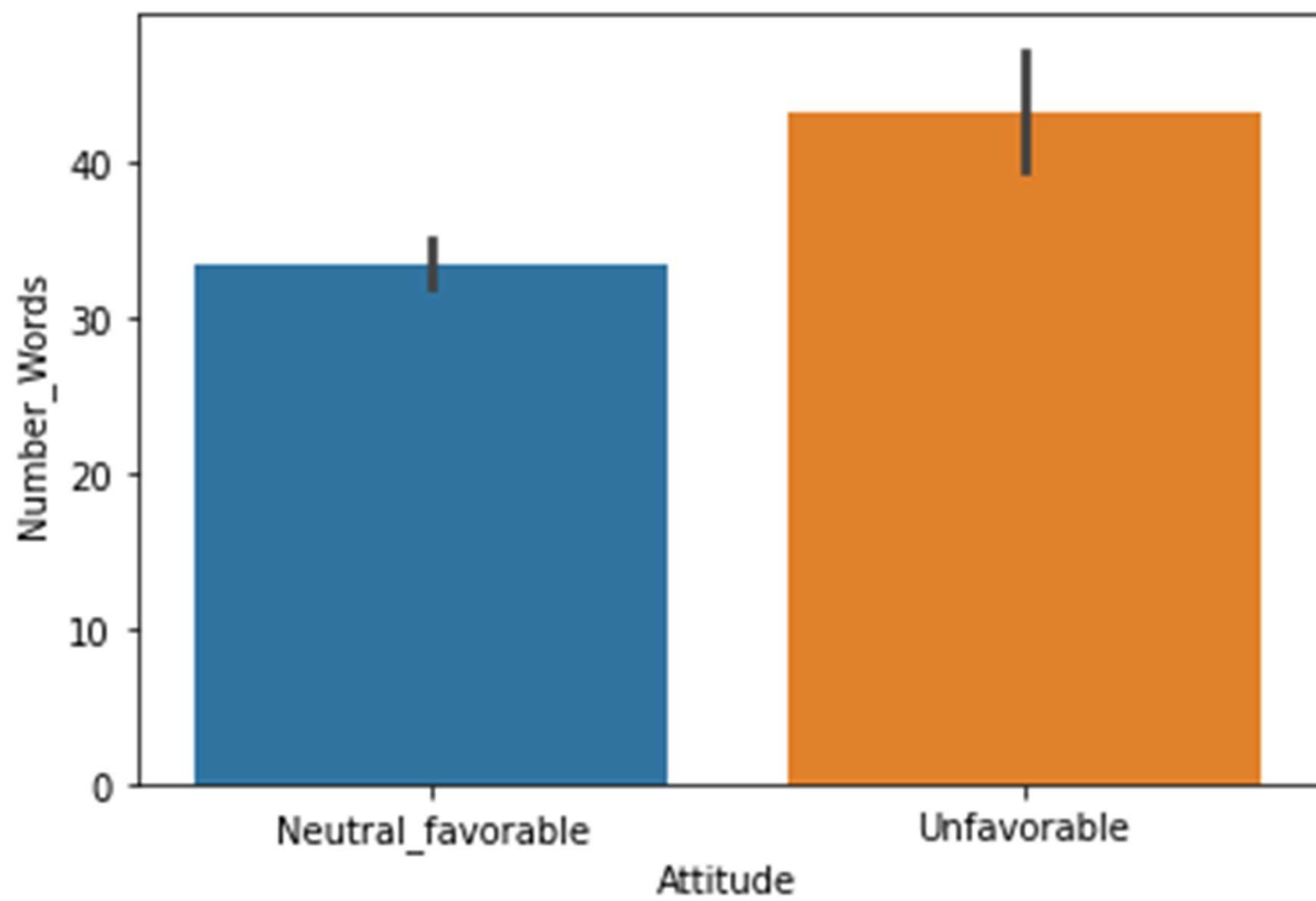
Lexicon-based, or dictionary-based method or rule-based methods

- The dictionaries are applied following certain rules.
- Sentiment values of text are derived from the sentiment orientation of the individual words in the text and using an existing dictionary.
- The dictionary contains words and their polarity (excellent is positive; horrible is negative).
- When a new text is encountered, words in the text are matched to words in the dictionary, and their values aggregated, to produce the semantic orientation for the entire text.

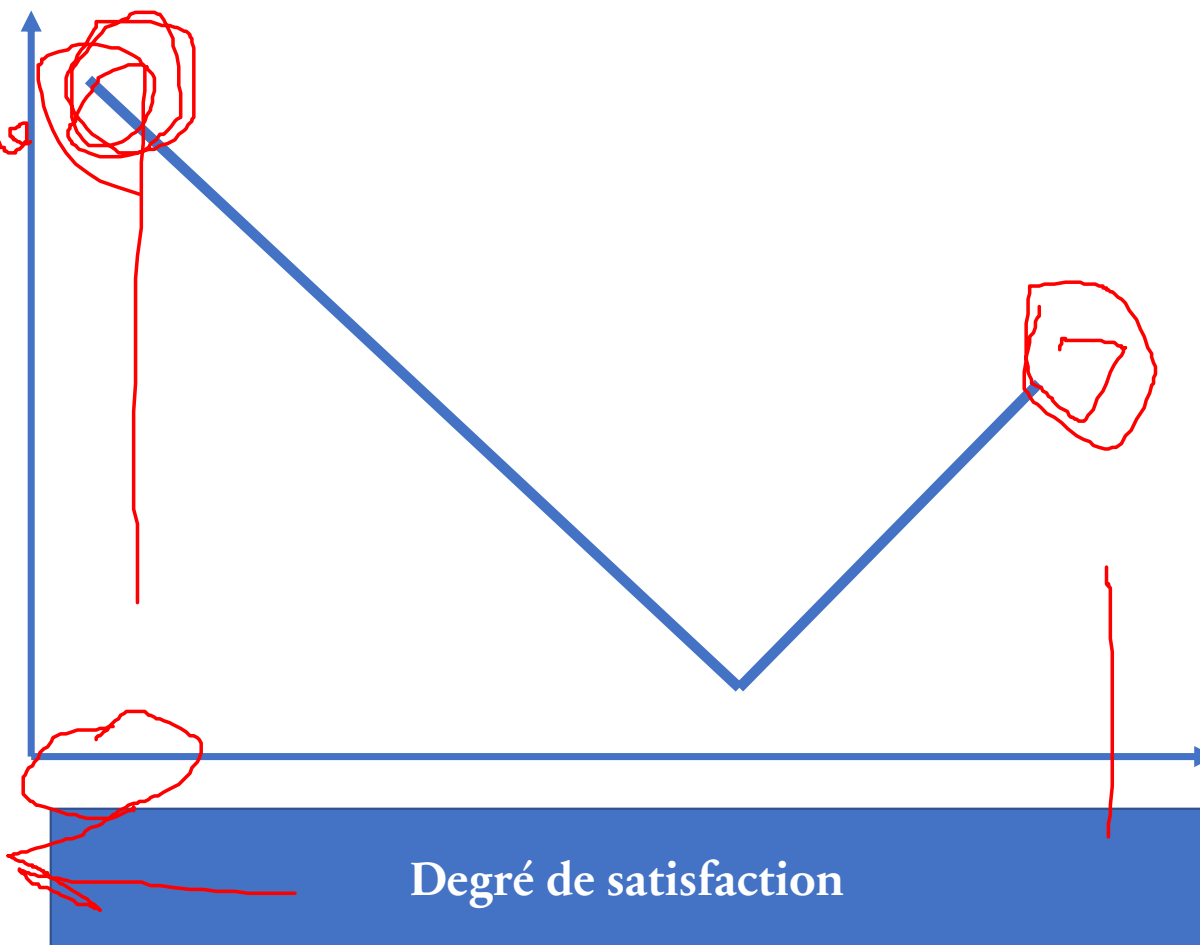








WOM
Bover
score



ML based sentiment analysis



Naïve Bayes Classifier

For NLP

Example

Suppose we are building a classifier that says whether a review text is positive or negative. Our training data has 5 sentences.

review	Tag
I am satisfied with this product	Positive
I lost my money	Negative
Very good product, I like it	Positive
Memorable experience	Positive
I was disappointed	Negative

Now, which tag does the sentence “We had a very bad experience with this product” belong to?

- Naive Bayes is a probabilistic classifier.
 1. We need to calculate the probability that the sentence “We had a very bad experience with this product” is Positive and the probability that it is Negative.
 2. Then, we take the largest one.
- Written mathematically, what we want is $P(\text{Positive} \mid \text{We had a very bad experience with this product})$, i.e. the probability that the tag of a sentence is Positive given that the sentence is “We had a very bad experience with this product”.
- That’s great, but **how do we calculate these probabilities?**

Bayes' Theorem

- We need to transform the probability we want to calculate into something that can be calculated using word frequencies.
- For this, we will use some basic properties of probabilities, and Bayes' Theorem.
- Bayes' Theorem is useful when working with conditional probabilities (like we are doing here), because it provides us with a way to reverse them:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

In our case ...

- we have $p(\text{Positive} \mid \text{We had a very bad experience with this product})$, so using this theorem we can reverse the conditional probability:

$$\begin{aligned} & p(\text{Positive} \mid \text{We had a very bad experience with this product}) = \\ & \frac{p(\text{We had a very bad experience with this product} \mid \text{Positive}) \times P(\text{Positive})}{p(\text{We had a very bad experience with this product})} \end{aligned}$$



$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

- Since for our classifier, we're just trying to find out which tag has a bigger probability, we can discard the divisor—which is the same for both tags—and just compare

$p(\text{We had a very bad experience with this product} | \text{Positive}) \times P(\text{Positive})$

with

$p(\text{We had a very bad experience with this product} | \text{Negative}) \times P(\text{Negative})$

- This is better, since we could calculate these probabilities
 - Just count how many times the sentence “We had a very bad experience with this product” appears in the *Positive* tag, divide it by the total, and obtain $p(\text{Positive} \mid \text{We had a very bad experience with this product})$.
- Problem
- “We had a very bad experience with this product” doesn’t appear in our training data, so this probability is zero.
- Unless every sentence that we want to classify appears in our training data, the model won’t be very useful.

Being Naive

- So here comes the Naive part:
 - we assume that every word in a sentence is independent of the other ones.
 - This means that we're no longer looking at entire sentences, but rather at individual words.
 - So for our purposes, "this was a nice product" is the same as "this product was nice" and "nice product was this".

- $p(\text{We had a very bad experience with this product})$
= $p(\text{We}) \times p(\text{had}) \times p(\text{a}) \times p(\text{very}) \times p(\text{bad})$
 $\times p(\text{experience}) \times p(\text{with}) \times p(\text{this}) \times p(\text{product})$

This assumption is very strong but super useful.

However, it's what makes this model work well with little data or data that may be mislabeled.

The next step is just applying this to what we had before:

- $p(\text{We had a very bad experience with this product} | \text{Positive})$
= $p(\text{We} | \text{Positive}) \times p(\text{had} | \text{Positive}) \times p(\text{a} | \text{Positive})$
 $\times p(\text{very} | \text{Positive}) \times p(\text{bad} | \text{Positive})$
 $\times p(\text{experience} | \text{Positive}) \times p(\text{with} | \text{Positive})$
 $\times p(\text{this} | \text{Positive}) \times p(\text{product} | \text{Positive})$

Calculating Probabilities

- The final step is just to calculate every probability and see which one turns out to be larger.
- Calculating a probability is just counting in our training data.
- First, we calculate the *a priori* probability of each tag:
 - for a given sentence in our training data, the probability that it is *Positive*
 $p(\text{Positive})$ is $3/5 = 0.6$
 - $p(\text{Negative}) = 2/5 = 0.4$

review	Tag
I lost my money	Negative
I was disappointed	Negative
I am satisfied with this product	Positive
Very good product, I like it	Positive
Memorable experience	Positive

Calculating Probabilities (continued)

- Sentence = We had a very bad experience with this product

- $p(\text{experience} \mid \text{Positive})$ means counting how many times the word “experience” appears in Positive texts [1] divided by the total number of words in Positive (3). Therefore,
 $p(\text{experience} \mid 1) = 1/12 = 0.08$

- However, we run into a problem here: “bad” doesn’t appear in any *Positive* text! That means that $p(\text{bad} \mid \text{Positive}) = 0$.

- A problem ... because we are going to be multiplying it with the other probabilities

$p(\text{We} \mid \text{Positive}) \times p(\text{had} \mid \text{Positive}) \times p(\text{a} \mid \text{Positive}) \times p(\text{very} \mid \text{Positive}) \times 0 \times p(\text{experience} \mid \text{Positive})$
 $\times p(\text{with} \mid \text{Positive}) \times p(\text{this} \mid \text{Positive}) \times p(\text{product} \mid \text{Positive})$

review	Tag
I lost my money	Negative
I was disappointed	Negative
I am satisfied with this product	Positive
Very good product, I like it	Positive
Memorable experience	Positive

What to do?

- Use the Laplace smoothing:
 - we add 1 to every count so it's never zero.
 - To balance this, we add the number of possible words to the divisor, so the division will never be greater than 1.
- In our case, the possible words are
 - ['I', 'lost', 'my', 'money', 'was', 'disappointed', 'am', 'satisfied', 'with', 'this', 'product', 'very', 'good', 'like', 'it', 'memorable', 'experience'].

What to do? (continued)

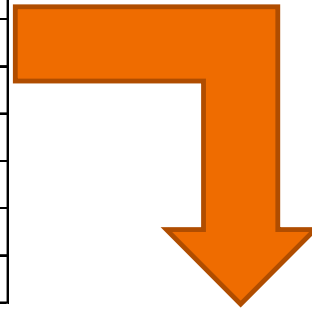
- Since the number of possible words is 17, applying smoothing we get that
 $p(\text{product}|\text{Positive}) = [2+1]/[12+17] = 0.10$

NB: 12 (number of words associated with
 positive sentences)
 17 (total number of words in the corpus)

review	Tag
I lost my money	Negative
I was disappointed	Negative
I am satisfied with this product	Positive
Very good product, I like it	Positive
Memorable experience	Positive

The full results ...

Word	P (word Positive)	P (word Negative)
We	$(0 + 1) \div (12 + 17)$	$(0 + 1) \div (6 + 17)$
had	$(0 + 1) \div (12 + 17)$	$(0 + 1) \div (6 + 17)$
a	$(0 + 1) \div (12 + 17)$	$(0 + 1) \div (6 + 17)$
very	$(1 + 1) \div (12 + 17)$	$(0 + 1) \div (6 + 17)$
bad	$(0 + 1) \div (12 + 17)$	$(0 + 1) \div (6 + 17)$
experience	$(1 + 1) \div (12 + 17)$	$(0 + 1) \div (6 + 17)$
with	$(1 + 1) \div (12 + 17)$	$(0 + 1) \div (6 + 17)$
this	$(1 + 1) \div (12 + 17)$	$(0 + 1) \div (6 + 17)$
product	$(2 + 1) \div (12 + 17)$	$(0 + 1) \div (6 + 17)$



Word	P (word Positive)	P (word Negative)
We	0,034	0,043
had	0,034	0,043
a	0,034	0,043
very	0,069	0,043
bad	0,034	0,043
experience	0,069	0,043
with	0,069	0,043
this	0,069	0,043
product	0,103	0,043

Now we just multiply all the probabilities, and see which one is the biggest:

Word	P (word Positive)	P (word Negative)
We	0,034	0,043
had	0,034	0,043
a	0,034	0,043
very	0,069	0,043
bad	0,034	0,043
experience	0,069	0,043
with	0,069	0,043
this	0,069	0,043
product	0,103	0,043
3,31E-12		5,55E-13

Our classifier gives “**We had a very bad experience with this product**”
the **Positive** tag.

Missed!

- TRAIN
- Xc vvg sd er ttt df az er df tg hj ui

- TEST
- SD LP HJ FG VCV XC VVG GH
- Xc vvg sd er ttt df az er df tg hj ui



Neural Networks and Deep Learning





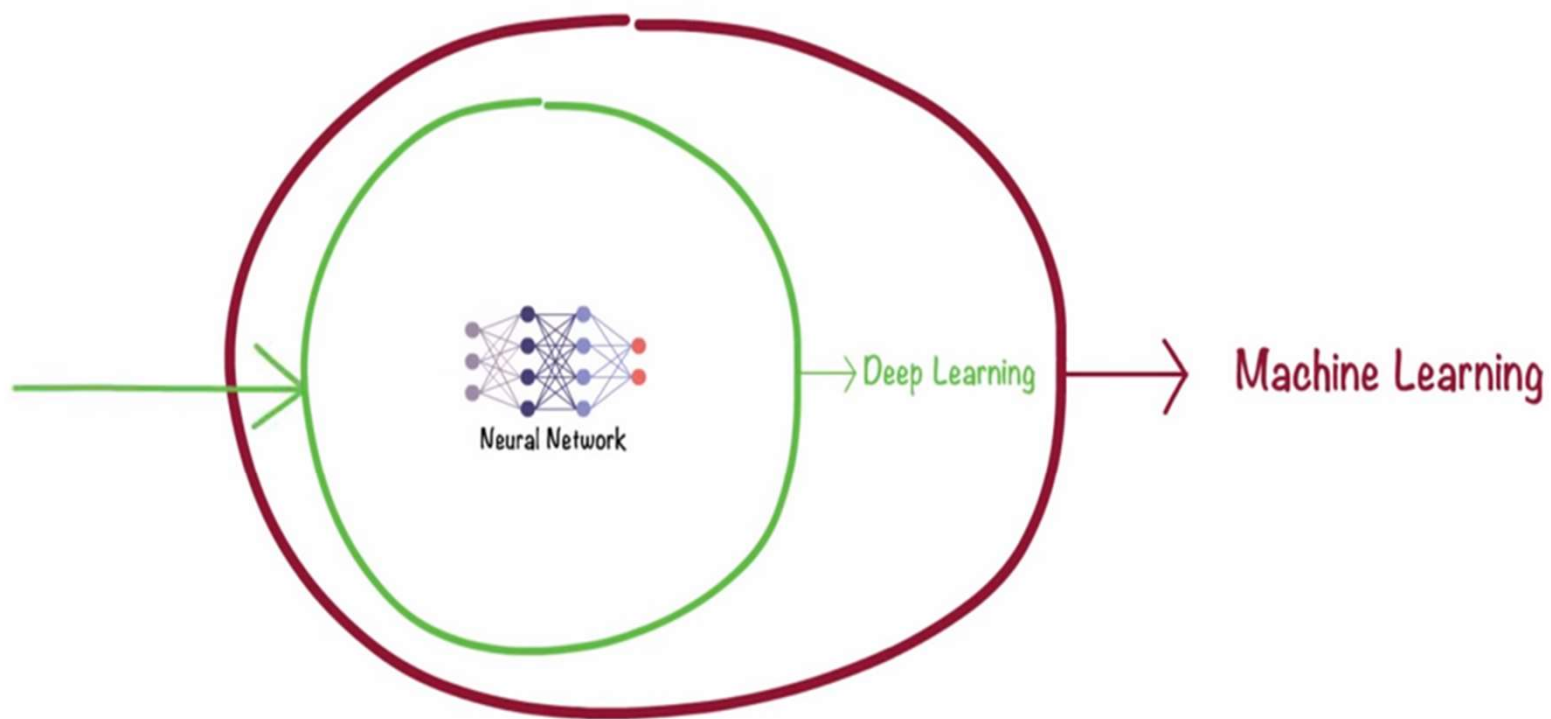
Issues

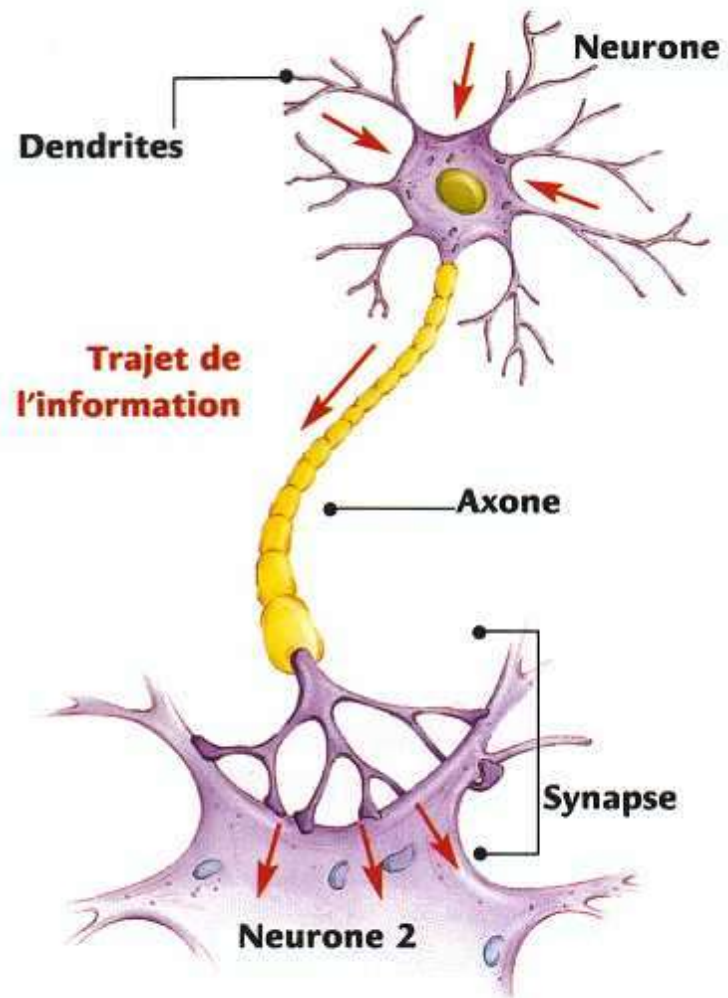
- What is artificial neural network?
- How NNs work
- Deep learning as an extension of NN
- Different types of architectures
- NN and deep learning with Keras on Python



What is an Artificial Neural Network ?

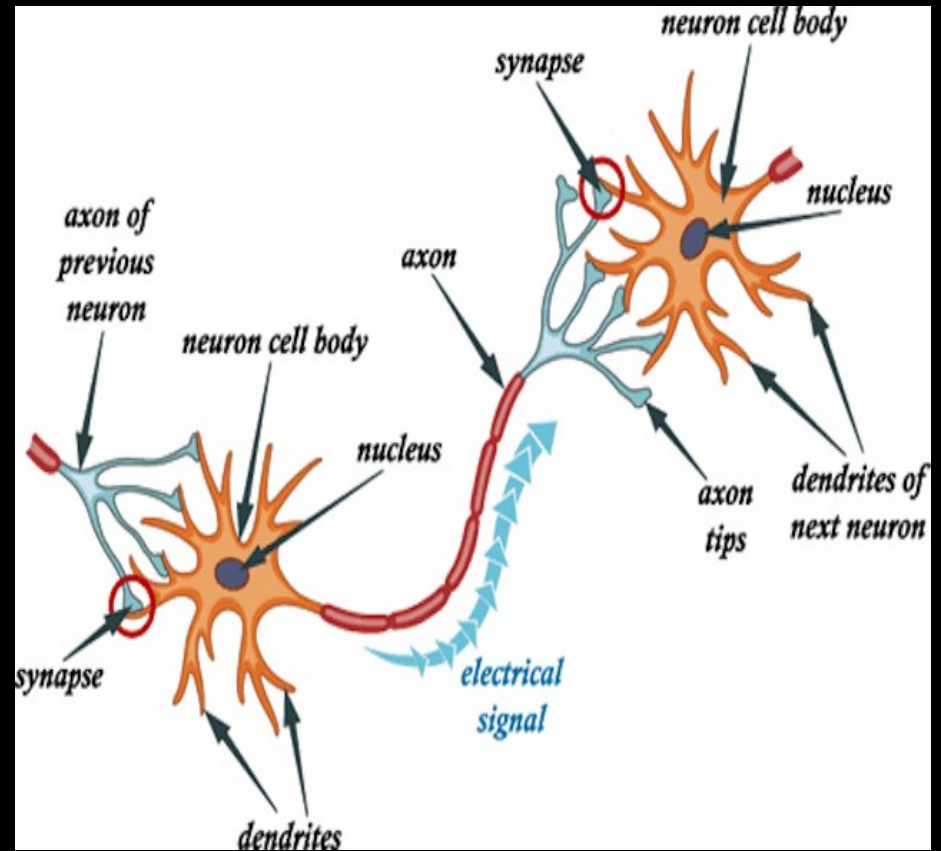




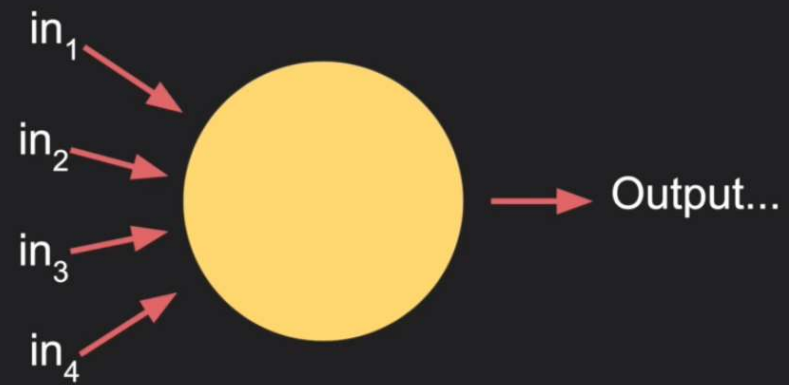


Relationship between Artificial Neural Networks & the Human Brain

- At each neuron, *dendrites* receive incoming signals sent by other neurons. If the neuron *receives a high enough level of signals* within a certain period of time, the neuron *sends an electrical pulse* into the terminals. These outgoing signals are then received by other neurons.

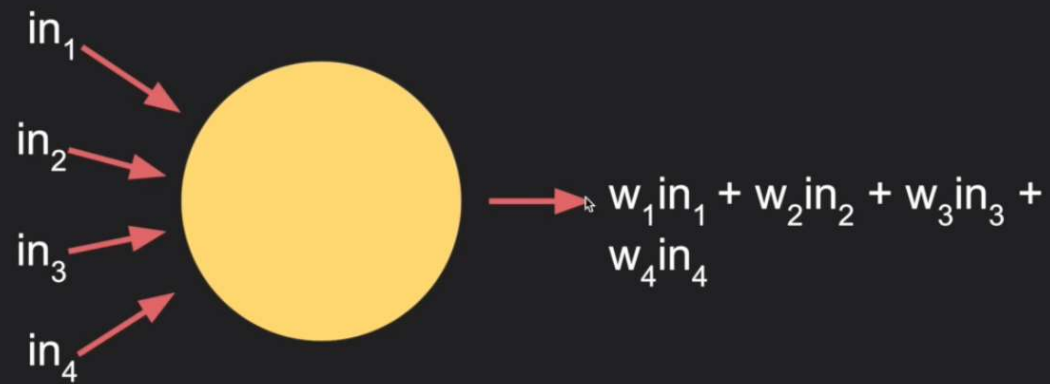


A Simple Neuron

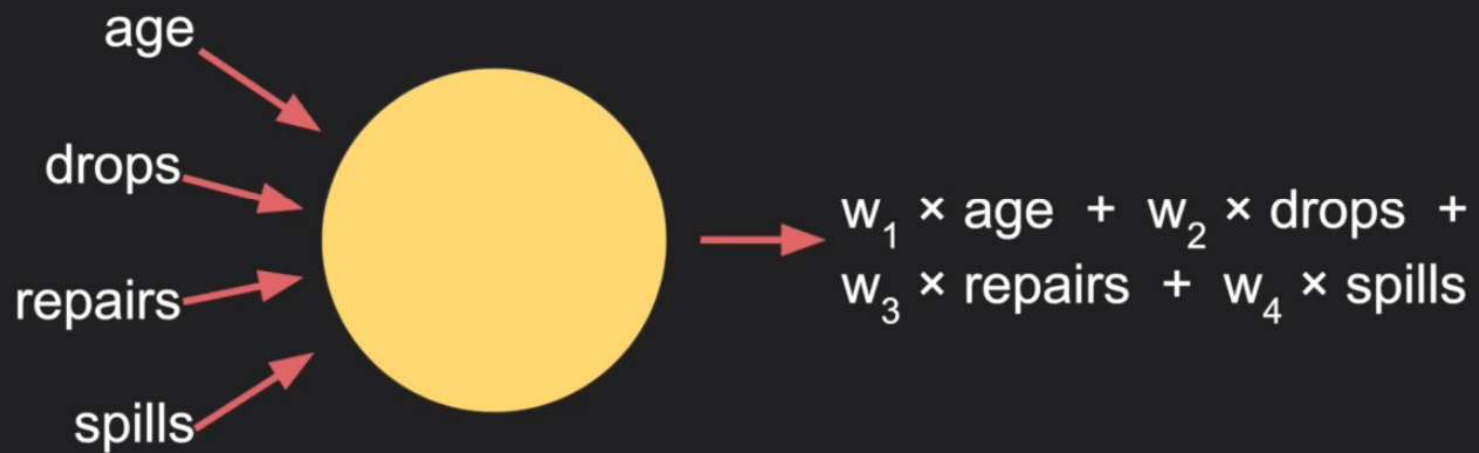


The goal of these weights is essentially to define priorities.

A Simple Neuron



Laptop Death

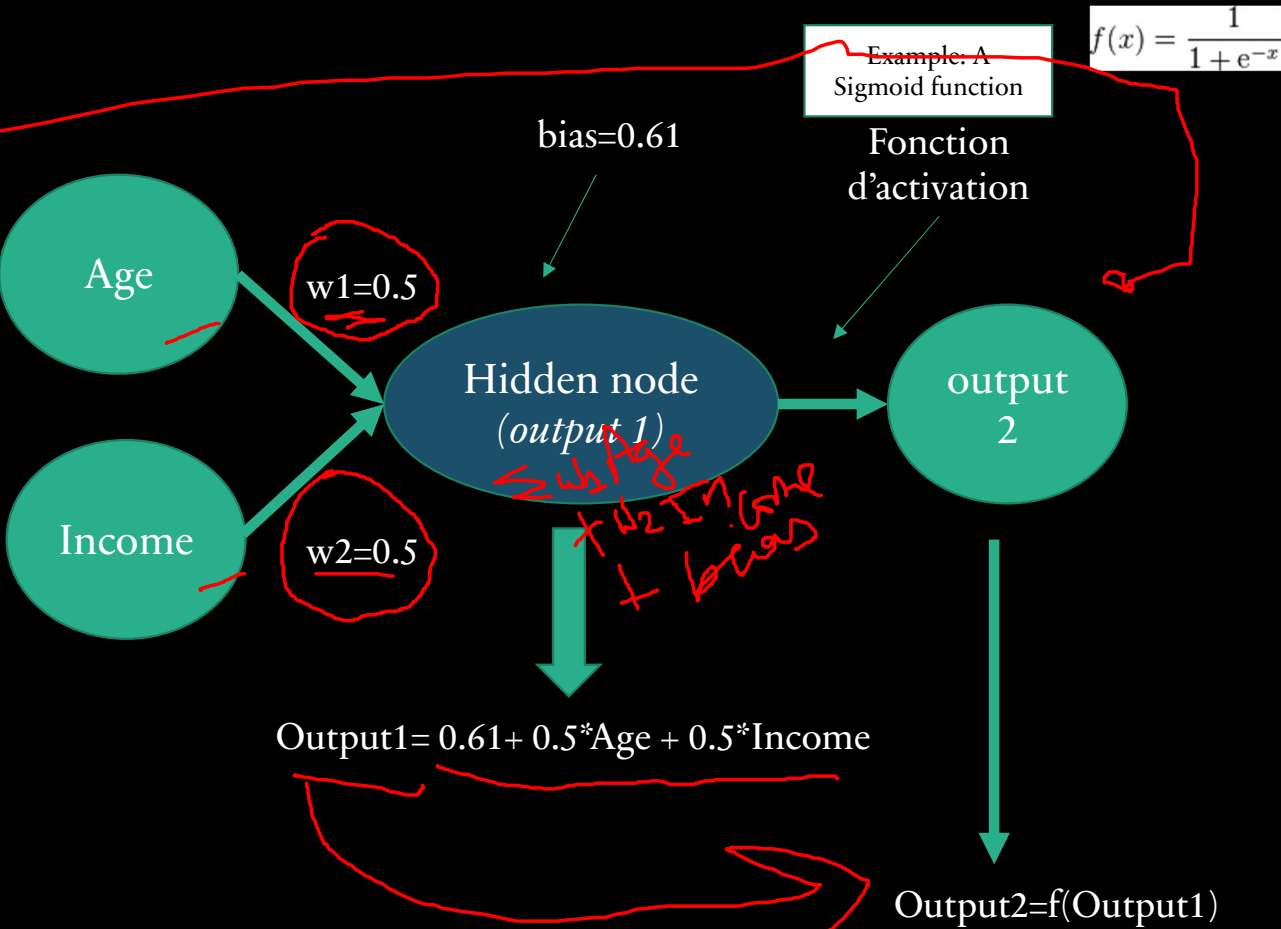


Activation Functions



Another neural network

ID	Age	Income(€100)	Response
A	34	456	0
B	43	543	0
C	45	123	0
D	65	324	1
E	43	543	1
F	44	67	0
G	32	567	1
H	28	876	1
I	34	1312	1
J	56	1716	1
K	65	342	0
L	46	299	1
M	53	2100	1



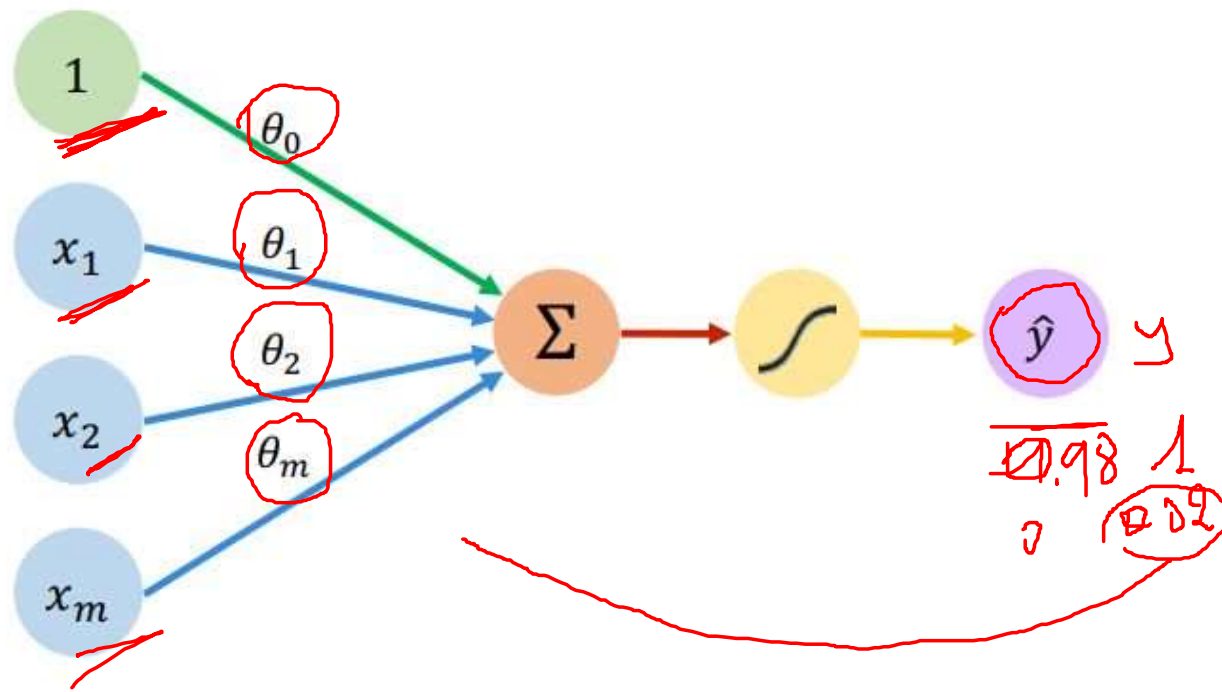


Diagram illustrating the mathematical representation of the neuron's output:

$$\hat{y} = g\left(\theta_0 + \sum_{i=1}^m x_i \theta_i\right)$$

Labels and annotations:

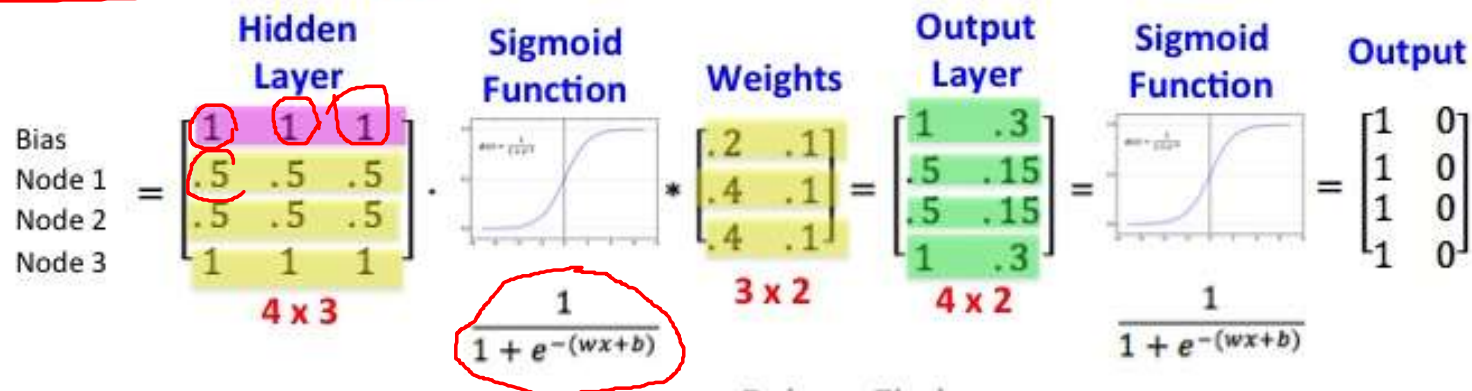
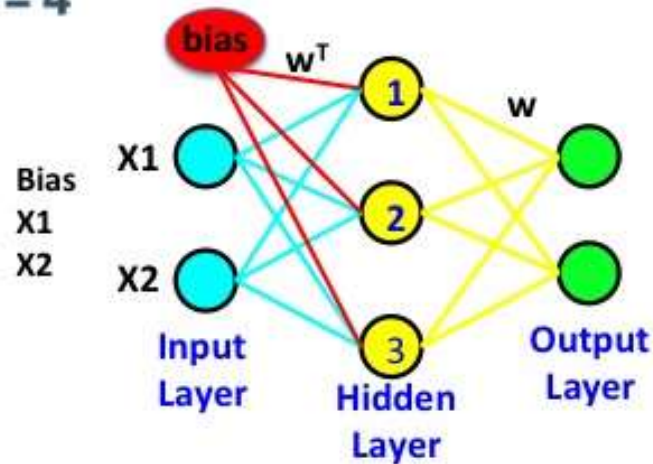
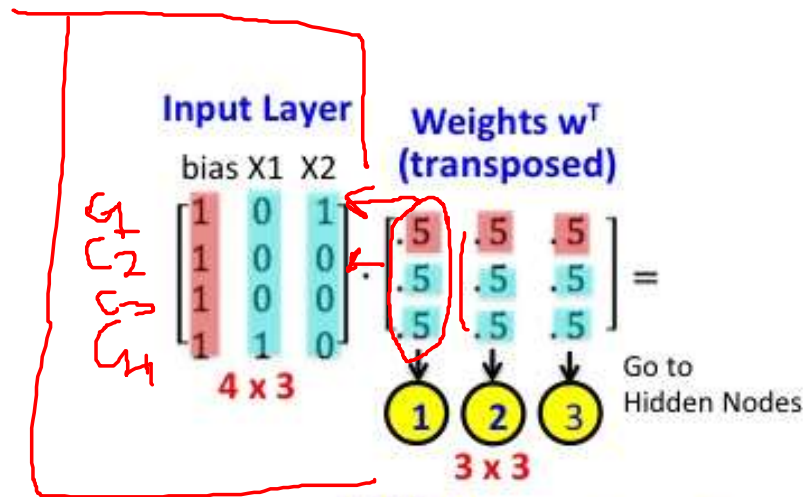
- Output:** Indicated by a purple arrow pointing to $\hat{y}$.
- Linear combination of inputs:** Indicated by a red arrow pointing to the term $\theta_0 + \sum_{i=1}^m x_i \theta_i$.
- Non-linear activation function:** Indicated by a yellow arrow pointing to g .
- Bias:** Indicated by a green arrow pointing to θ_0 .

Handwritten red annotations include a bracket on the left, a red line under θ_0 , and a red line under the summation term.

Inputs Weights Sum Non-Linearity Output

Neural Networks

Color Guided Matrix Multiplication for a Binary Classification Task with N = 4



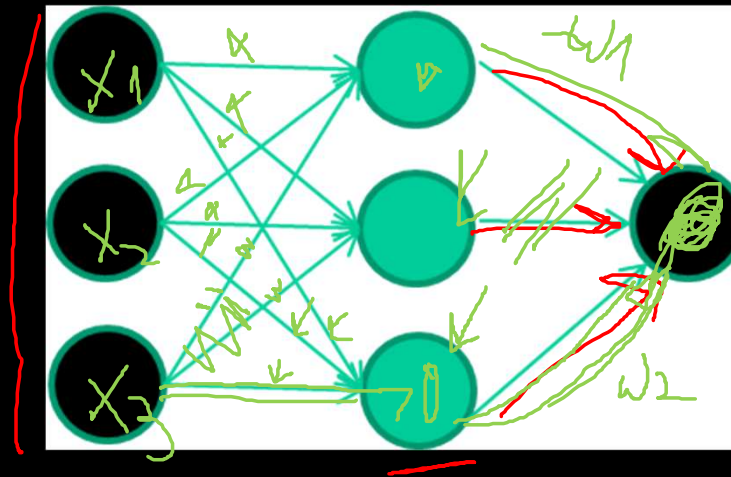
Rubens Zimbres

How do they learn?

Let us assume that we have a dataset with 3 predictors and one dependent variable or target

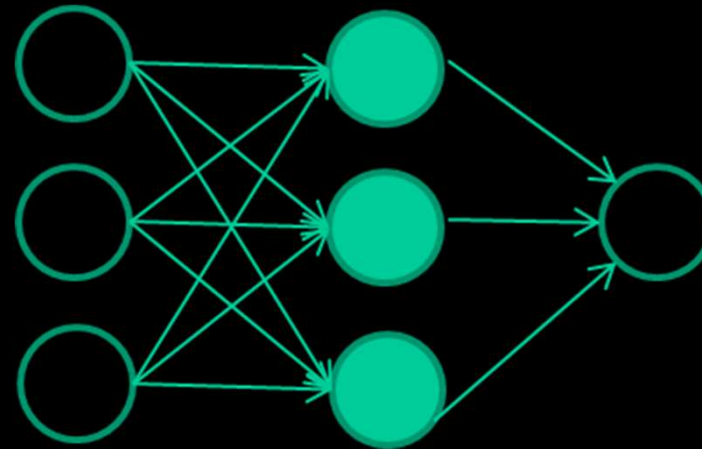
A dataset

<i>Fields</i>		<i>class</i>
1.4 2.7 1.9	→	0
3.8 3.4 3.2	→	0
6.4 2.8 1.7	→	1
4.1 0.1 0.2		0
etc ...		



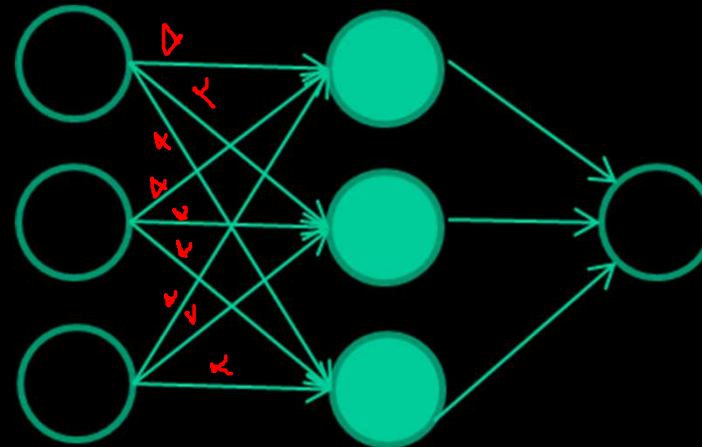
We have to train our neural network

<i>Fields</i>	<i>class</i>
1.4 2.7 1.9	0
3.8 3.4 3.2	0
6.4 2.8 1.7	1
4.1 0.1 0.2	0
etc ...	



Initialize with random weights

<i>Fields</i>	<i>class</i>
1.4 2.7 1.9	0
3.8 3.4 3.2	0
6.4 2.8 1.7	1
4.1 0.1 0.2	0
etc ...	



Training data

Fields *class*

1.4 2.7 1.9 0

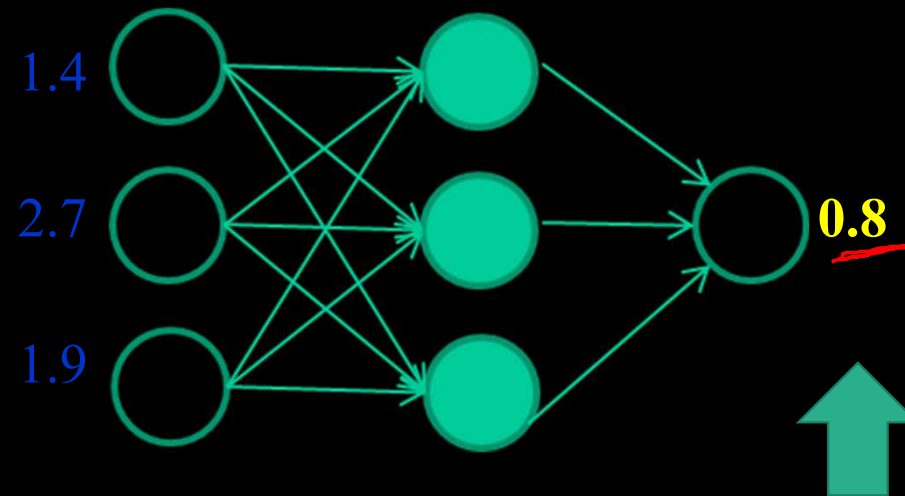
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Present a training pattern
& Feed it through to get output



Let us say that the weights lead to a probability of
0.8 for that individual.

Training data

Fields *class*

1.4 2.7 1.9 0

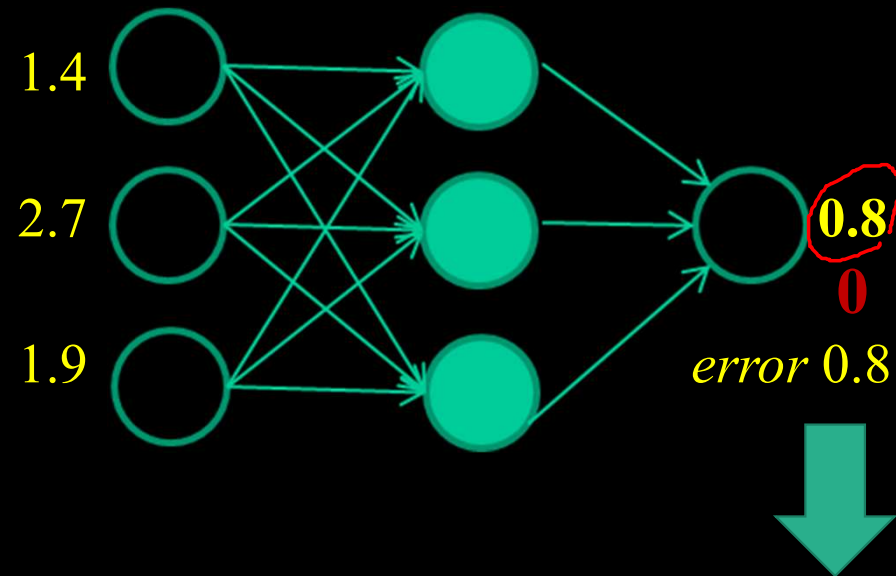
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Compare with target output



There is an error since this individual did not engage in the targeted outcome.

Training data

Fields *class*

1.4 2.7 1.9 0

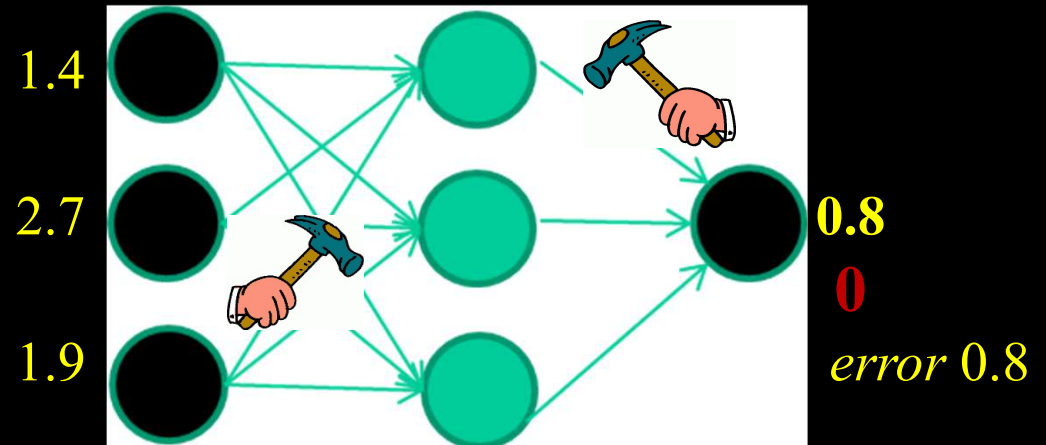
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Adjust weights based on error



Training data

Fields *class*

1.4 2.7 1.9 0

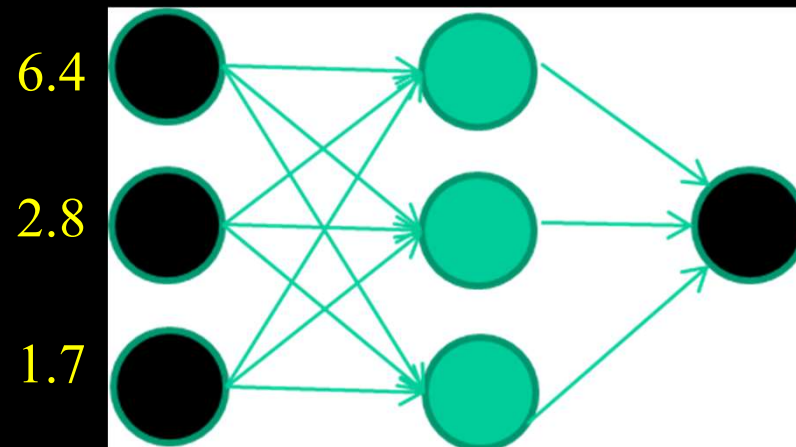
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Present a training pattern



Training data

Fields *class*

1.4 2.7 1.9 0

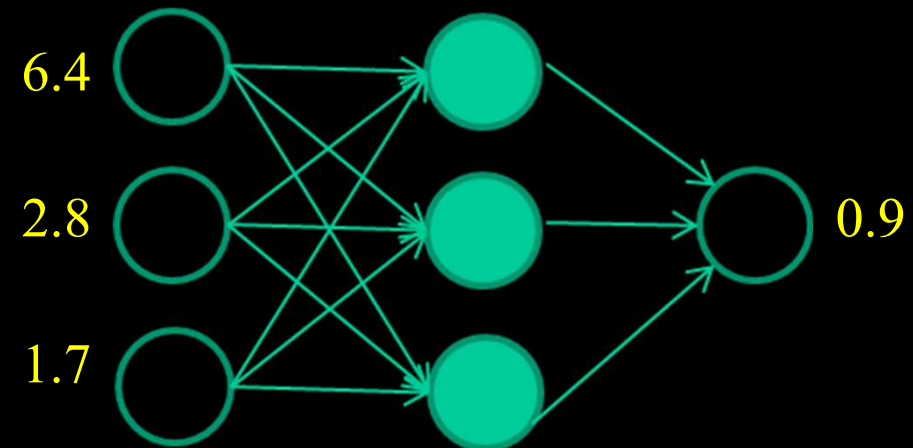
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Feed it through to get output



Training data

Fields *class*

1.4 2.7 1.9 0

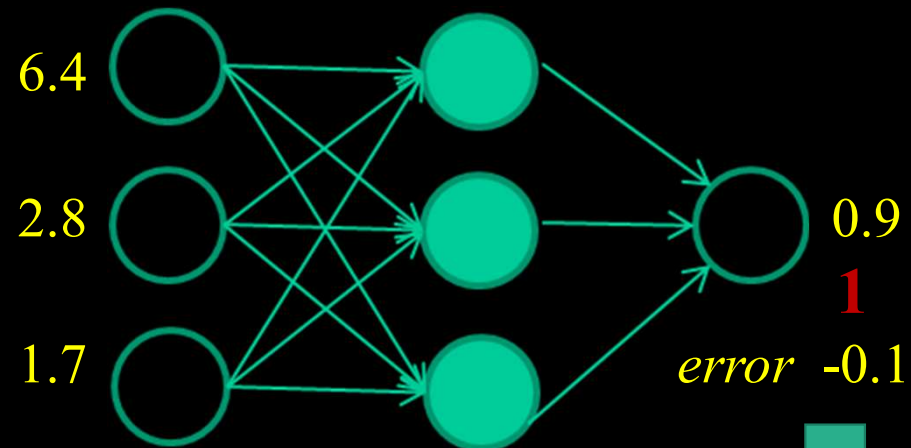
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Compare with target output



There is an error since this individual has a probability of 1

Training data

Fields *class*

1.4 2.7 1.9 0

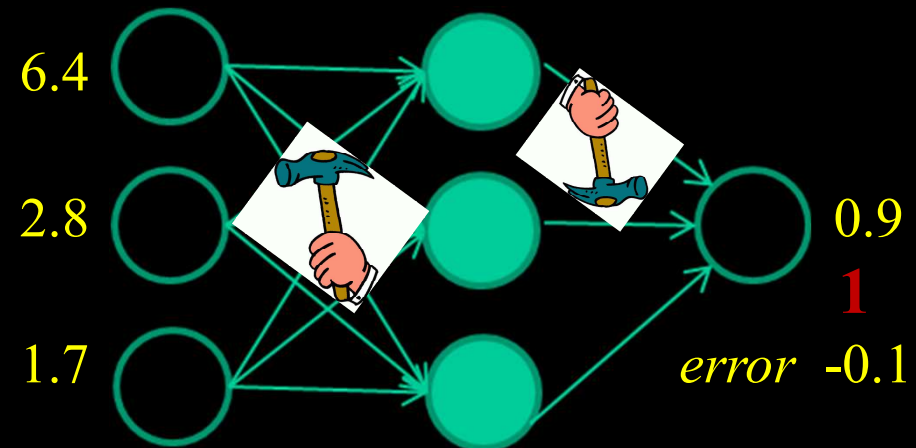
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

etc ...

Adjust weights based on error



Training data

Fields *class*

1.4 2.7 1.9 0

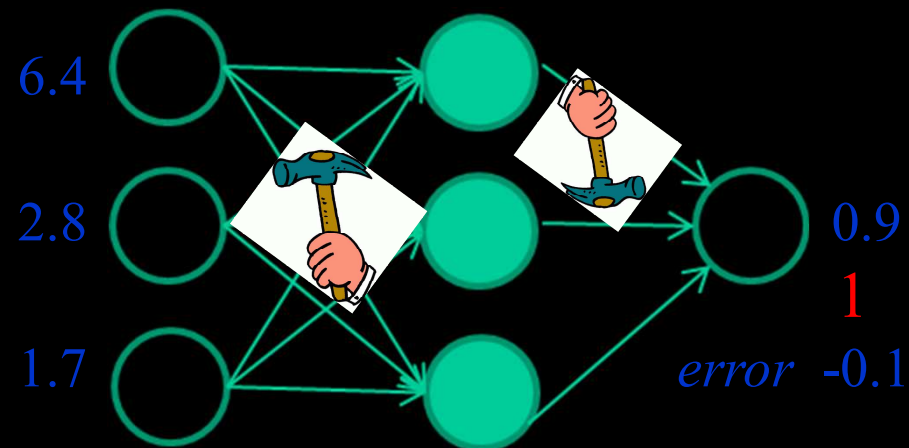
3.8 3.4 3.2 0

6.4 2.8 1.7 1

4.1 0.1 0.2 0

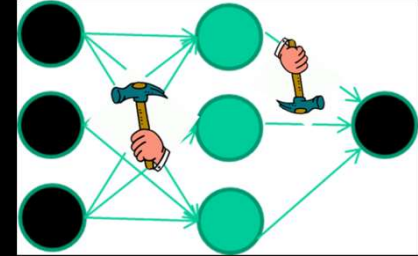
etc ...

And so on



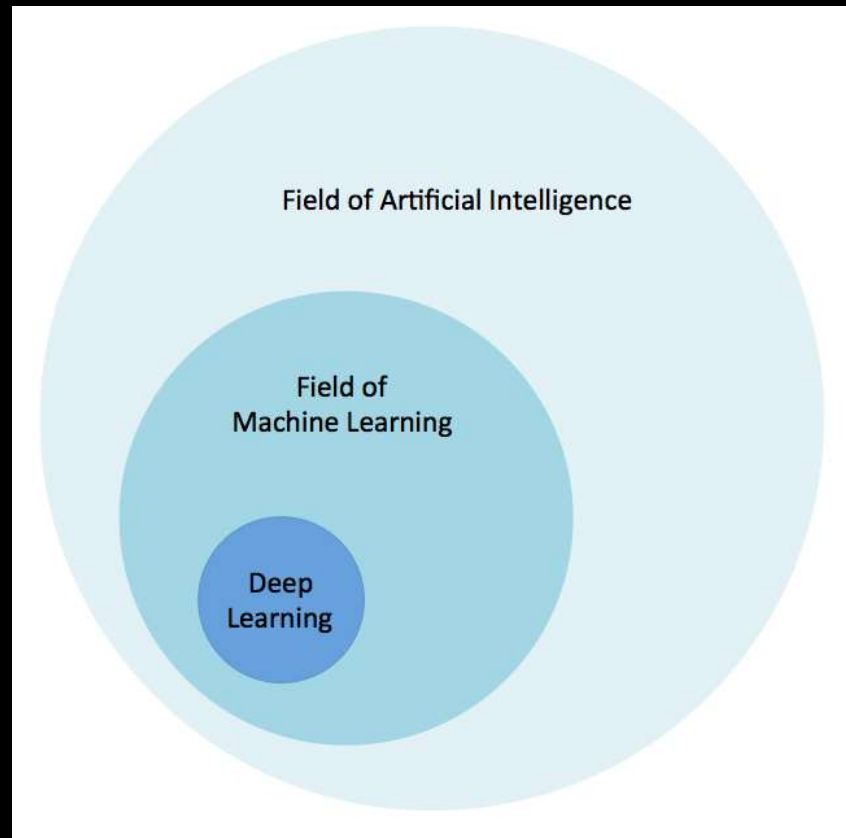
- Repeat this thousands, maybe millions of times – each time taking a random training instance, and making slight weight adjustments
- *Algorithms for weight adjustment are designed to make changes that will reduce the error*

In sum...



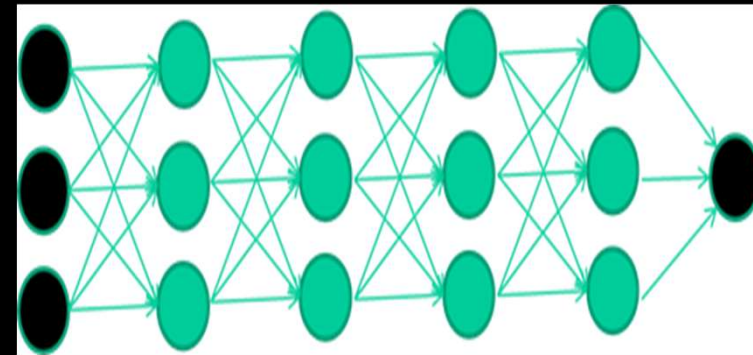
- Weight-learning algorithms for Neural Networks are dumb
- They work by making thousands and thousands of tiny adjustments, each making the network do better at the most recent pattern, but perhaps a little worse on many others
- But, by luck, eventually this tends to be good enough to learn effective classifiers for many real applications

And comes DEEP LEARNING

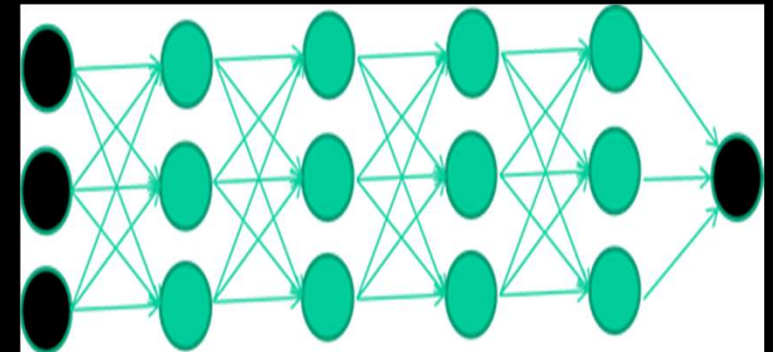
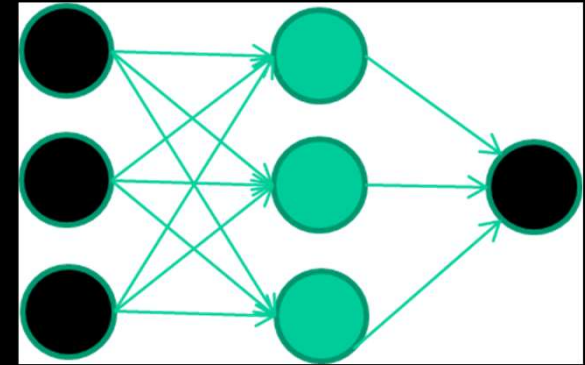


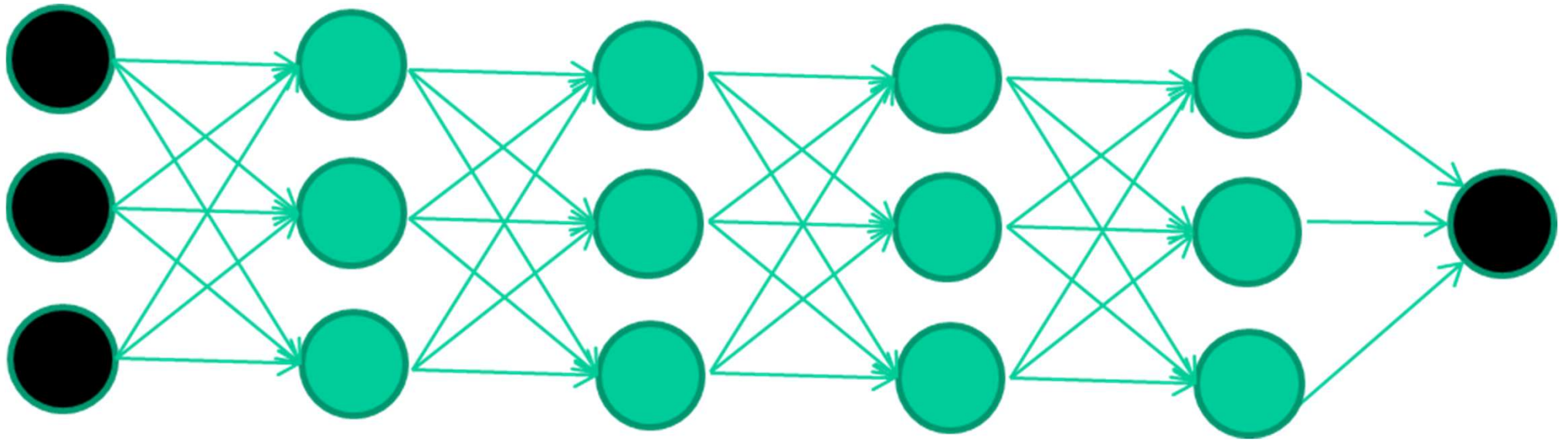
The short answers

1. 'Deep Learning' means using a neural network with several layers of nodes between input and output
2. the series of layers between input & output do feature identification and processing in a series of stages, just as our brains seem to.



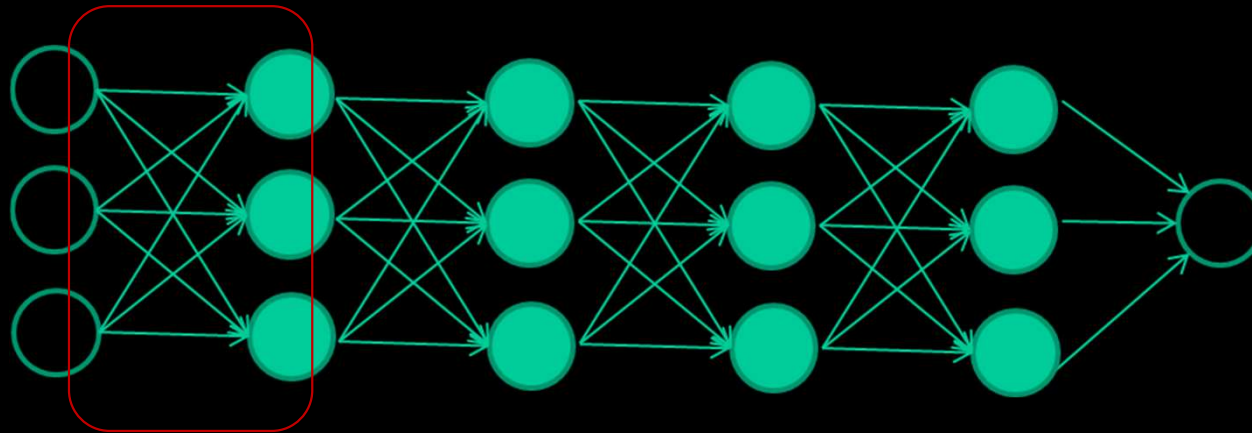
- Multilayer neural networks have been around for years. What's actually new?
- We have always had good algorithms for learning the weights in networks with 1 hidden layer
 - but these algorithms are not good at learning the weights for networks with more hidden layers
- What's new is: algorithms for training many-layer networks





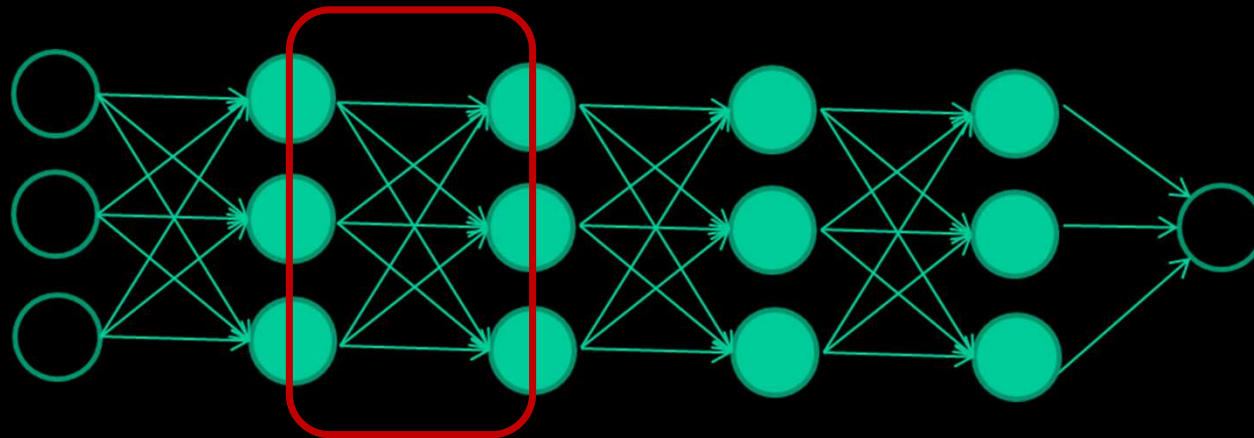
The new way to train multi-layer NNs...

The new way to train multi-layer NNs...



Train this layer first

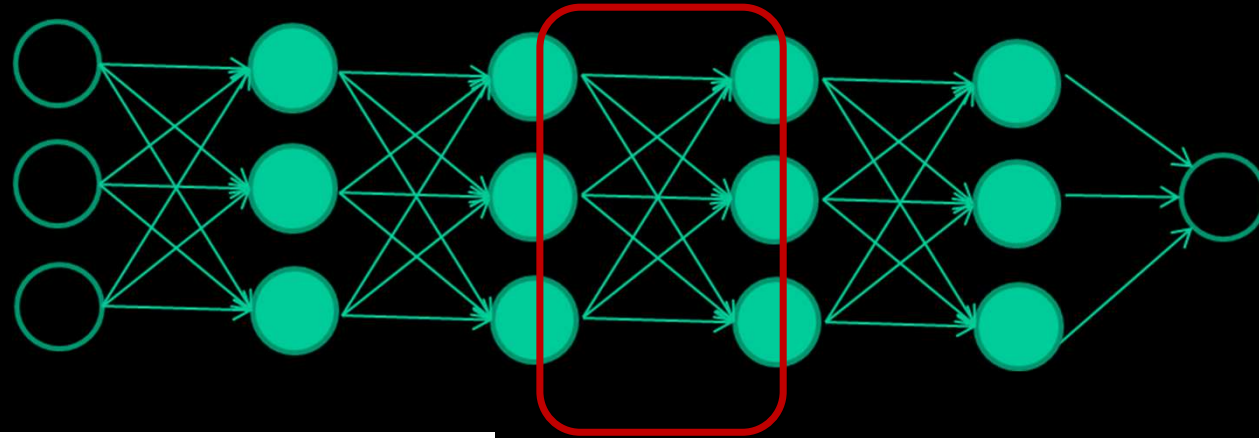
The new way to train multi-layer NNs...



Train this layer first

then this layer

The new way to train multi-layer NNs...

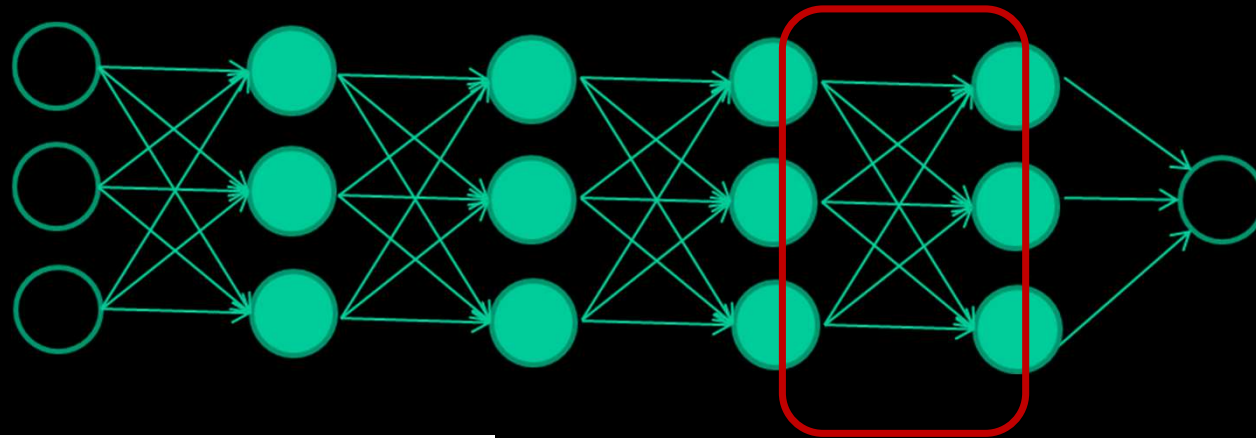


Train this layer first

then this layer

then this layer

The new way to train multi-layer NNs...



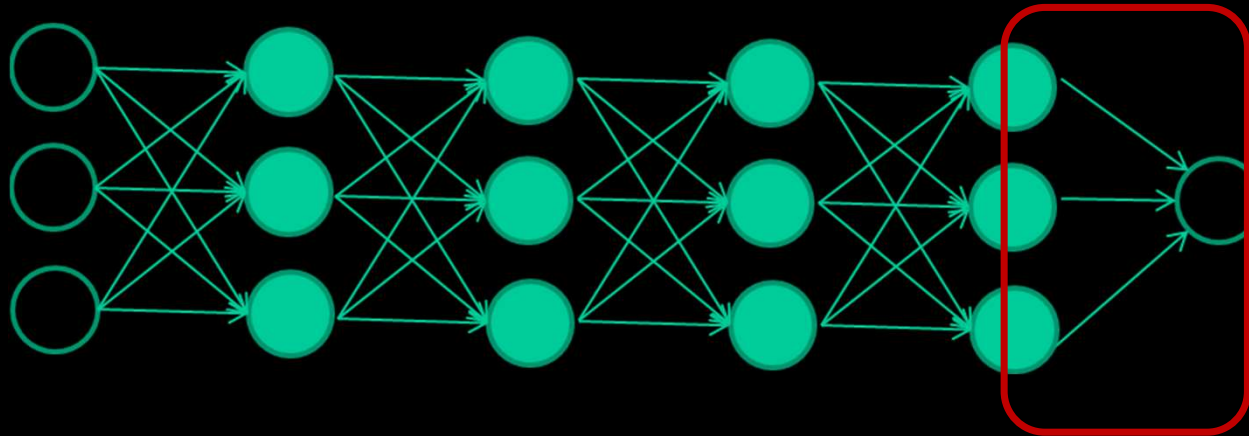
Train this layer first

then this layer

then this layer

then this layer

The new way to train multi-layer NNs...



Train this layer first

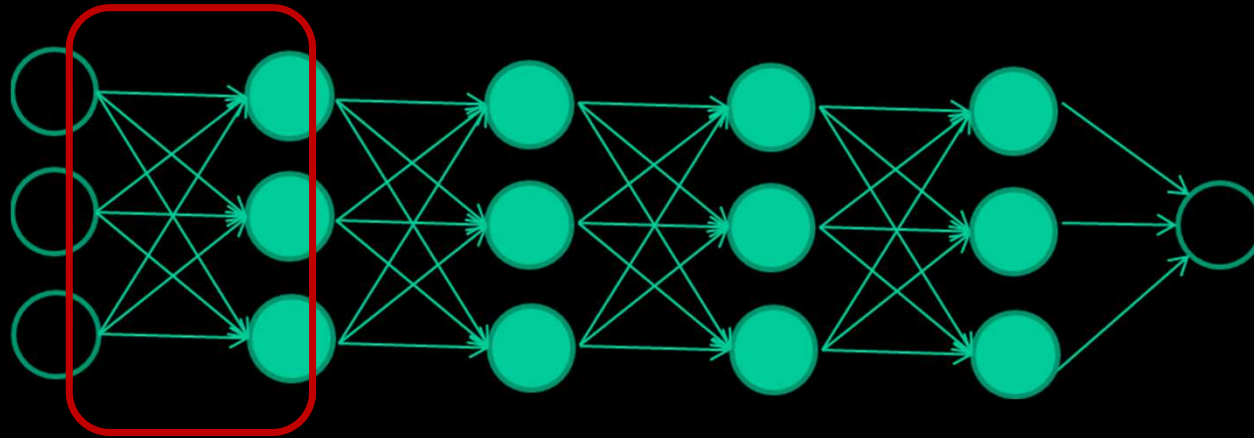
then this layer

then this layer

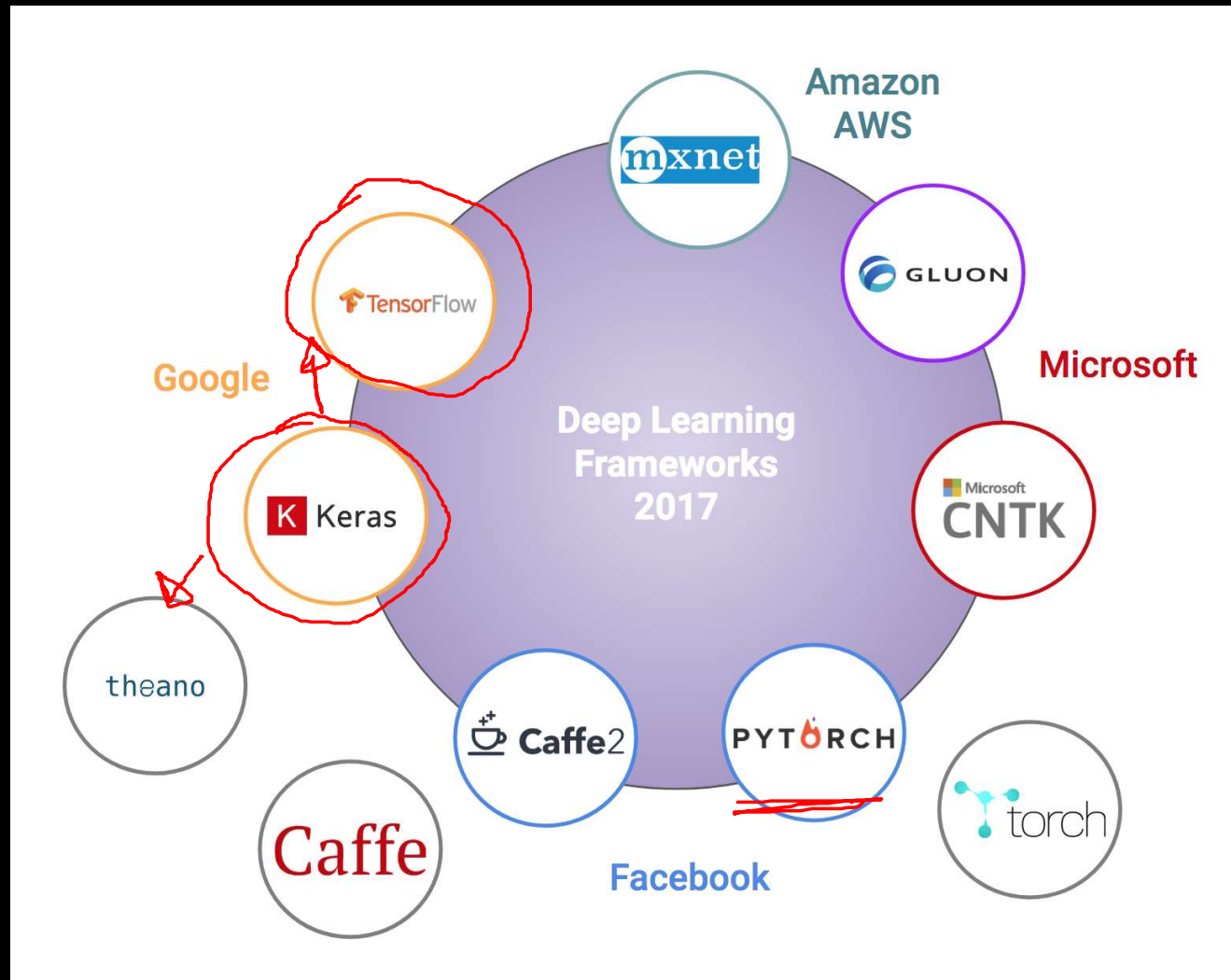
then this layer

finally this layer

The new way to train multi-layer NNs...



*EACH of the (non-output) layers is trained to be an **auto-encoder***

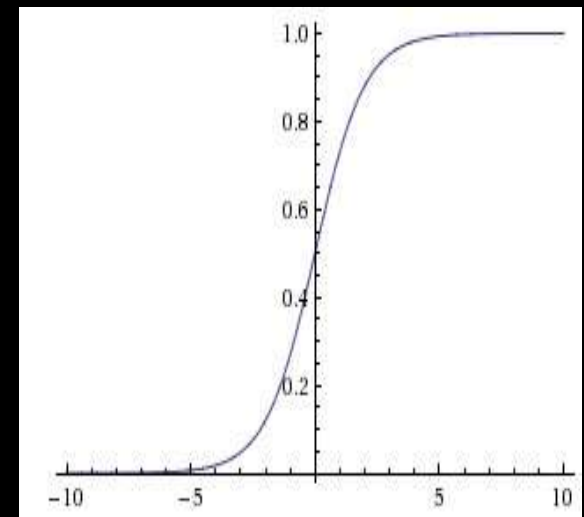
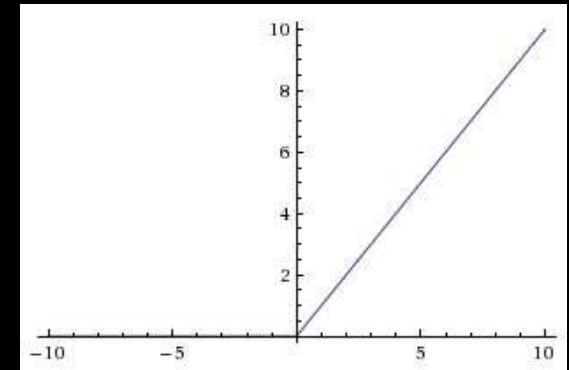


DEEP LEARNING WITH KERAS

- A sequential neural-network in Keras consists of a **sequence of layers**, starting from the inputs layer up to the output layer (also known as **Feedforward Neural Network**). The number and breadth of intermediate layers ("deep layers" or "hidden layers") depends on accuracy and performance requirements that we impose. **We can start with a single hidden layer and add more neurons to it or add more hidden layers in progressive stages until we meet our accuracy requirements.**
- The second factor we need to attend is **the connectivity type of our network**: which neurons from a given layer are connected to which neurons at the next layer? The most straightforward type is **full connectivity**, which means that every neuron at a given layer is connected to every neuron at the next layer. This type of connectivity is referred to by the **Dense method** of Keras layers module.

- The third factor is what kind of **activation function** are we going to use? Each non-input neuron receives many inputs from neurons at the layer below it. How do we determine its output? The function that determines the value that a neuron fires in response to its inputs vector is called the **activation function** of the model.
- Each connection must be assigned an initial weight, which is going to be tuned during the training phase. The usual arrangement is to assign small random weights when the model is defined. It usually has a negligible impact on the training speed.

- The most common activation function used is the **rectified linear unit (ReLU) function**.
 - This activation is thresholded at zero.
 - (+) It was found to greatly accelerate the convergence of stochastic gradient descent compared to the sigmoid/tanh functions. It is argued that this is due to its linear, non-saturating form.
 - (-) Unfortunately, ReLU units can be fragile during training and can “die”. It could cause the weights to update in such a way that the neuron will never activate on any datapoint again.
- For the output layer, the common practice is to use the sigmoid activation function as its output is always between 0 and 1 (which is useful for classifying).



Specify your model in
Keras ...

Define the model

- Models in Keras are defined as a sequence of layers.
 - create a Sequential model and add layers one at a time.
 - Set the number of predictors = # of inputs using the **input_dim** argument.
- How do we know the number of layers and their types?
 - The best network structure is found through a process of **trial and error**.

```
# create model
```

```
model1 = Sequential()
```

```
model1.add(Dense(16, input_dim=8, init='uniform',  
activation='relu'))
```

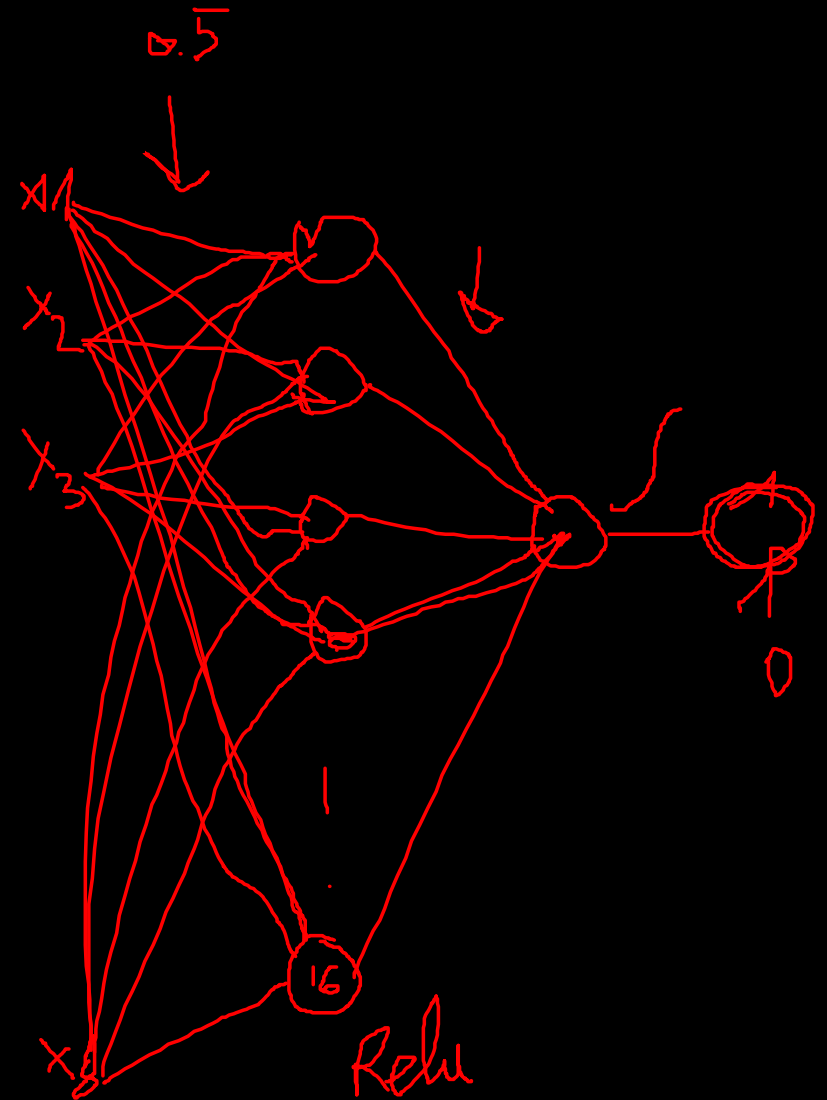
```
model1.add(Dense(1, init='uniform', activation='sigmoid'))
```

```
# We use a sigmoid on the output layer to ensure our network output is  
between 0 and 1
```

create model

```
model1 = Sequential()  
model1.add(Dense(16,  
input_dim=8, init='uniform',  
activation='relu'))  
model1.add(Dense(1,  
init='uniform',  
activation='sigmoid'))
```

We use a sigmoid on the output layer to ensure our network output is between 0 and 1



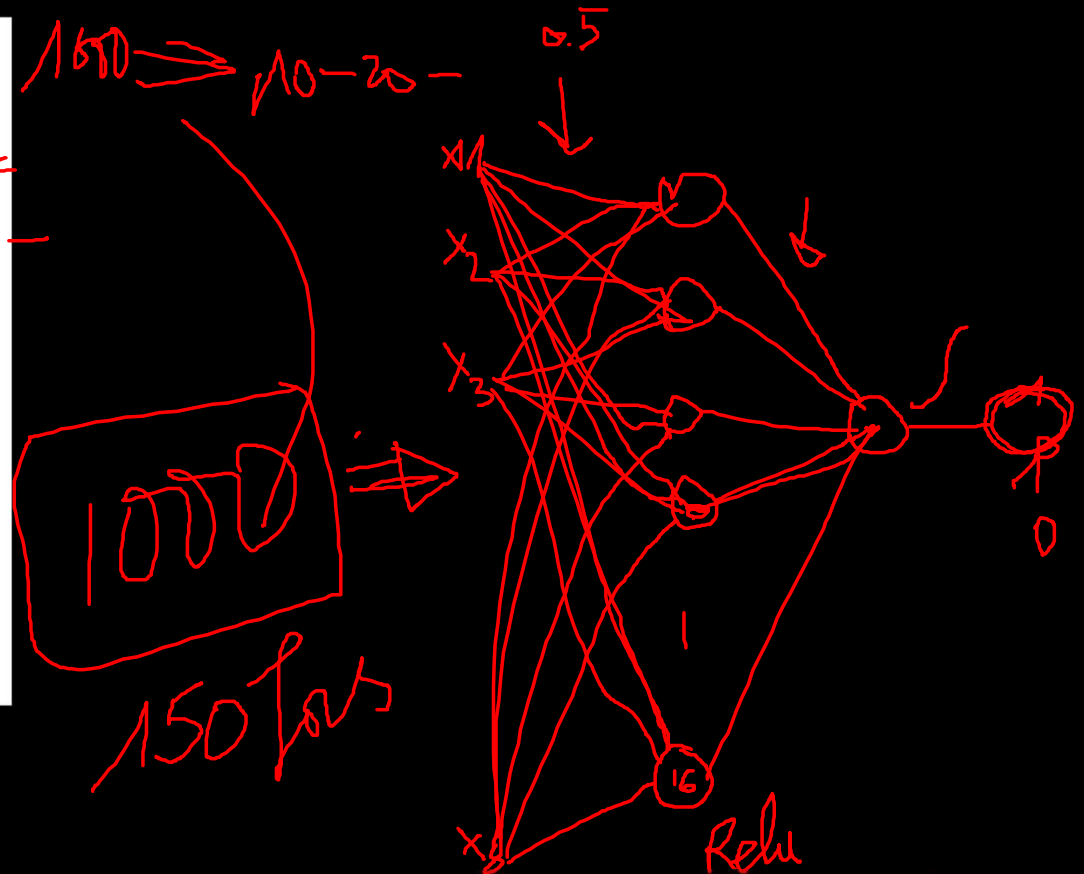
```
# create model
```

```
model1 = Sequential()
```

```
model1.add(Dense(16,  
input_dim=8, init = 'uniform',  
activation='relu'))
```

```
model1.add(Dense(1,  
init='uniform',  
activation='sigmoid'))
```

We use a sigmoid on the output layer to ensure our network output is between 0 and 1



```
# Compile model
```

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
# Fit the model
```

```
model.fit(X, Y, epochs=150, batch_size=10)
```

Define the model ...

- Fully connected layers are defined using the **Dense class**.
 - specify the number of neurons in the layer as the first argument,
 - the initialization method as the second argument as **init**
 - specify the activation function using the **activation** argument.
- We will use the rectifier (**relu**) activation function on the first two layers and the sigmoid function in the output layer.
 - The first layer has 16 neurons and expects 8 input variables.
 - The output layer has 1 neuron to predict the class (onset of diabetes or not).

```
# create model
model1 = Sequential()
model1.add(Dense(16,
input_dim=8, init = 'uniform',
activation='relu'))
model1.add(Dense(1,
init='uniform',
activation='sigmoid'))
# We use a sigmoid on the output layer to
ensure our network output is between 0 and
1
```


We need to specify

- (a) An optimizer. the optimizer used to search through different weights for the network.
 - Stochastic Gradient descent (SGD) : you update a set of parameters in an iterative manner to minimize an error function. SGD is more efficient for large scale problems because parameter updates are more frequent.
 - Adam: Adam updates its gradient by taking into account the previous gradient. Standard SGD requires careful tuning of learning rates, but this less true with Adam and related methods.

```
# Compile model
```

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

We need to specify ...

- (b) A loss function. It is the main source of feedback in the training process. Each time we run our model on a given input, the loss function measures the error of the output from the target objective.
- This is the objective that the model will try to minimize. For example, a **binary_crossentropy** or **mse for regression**.
- (c) A list of metrics. For any classification problem you will want to set this to `metrics=['accuracy']`.

```
# Compile model  
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Fit Model

- Now it is time to execute the model on some data by calling the **fit()** function on the model.
- The training phase consists of several cycles (called epochs in Keras jargon).
- **Epoch**: A full pass over all of your *training* data. One epoch ends when your model had **run the data through all nodes in your network**.
 - For example, if you have 1213 observations. So an epoch concludes when it has finished a training pass over all 1213 of your observations.
- The number of epochs is up to you, and we may experiment with several options.

```
# Fit the model  
model.fit(X, Y, epochs=150, batch_size=10)
```

Fit Model ...

- In the training phase, our model will tune the weights. Keras provides an option to decide at which points the weights update is done through the **batch_size** argument. The weights update will occur after every **batch_size** training instances. The model is updated each time a batch is processed, which means that it can be updated multiple times during one epoch.
 - Specify the number of iterations called **epochs**, using the **nepochs** argument.
 - Specify the number of instances using the **batch_size** argument.

```
# Fit the model  
model.fit(X, Y, epochs=150, batch_size=10)
```

Monitor the progress of the training process

- The **FitMonitor** callback is defined in the `kerutils` module and its purpose is to monitor the progress of the training process, report progress and some intermediate statistics, and save the best model in which the training accuracy and validations accuracy are maximal and are close enough

```
fmon = FitMonitor()
h = model1.fit(X_train, y_train, nb_epoch=100,
              batch_size=10, verbose=0, validation_data=(X_test,
                                                         y_test),
              callbacks=[fmon],)
```

Evaluate Model ...

- We have trained our neural network on the entire dataset and we can evaluate the performance of the network on the same dataset.
- This will only give us an idea of how well we have modeled the dataset (e.g. train accuracy), but no idea of how well the algorithm might perform on new data. We have done this for simplicity, but ideally, you could separate your data into train and test datasets for training and evaluation of your model.
- You can evaluate your model on your training dataset using the **evaluate()** function on your model and pass it the same input and output used to train the model.
- This will generate a prediction for each input and output pair and collect scores, including the average loss and any metrics you have configured, such as accuracy.

Use the conventional metrics

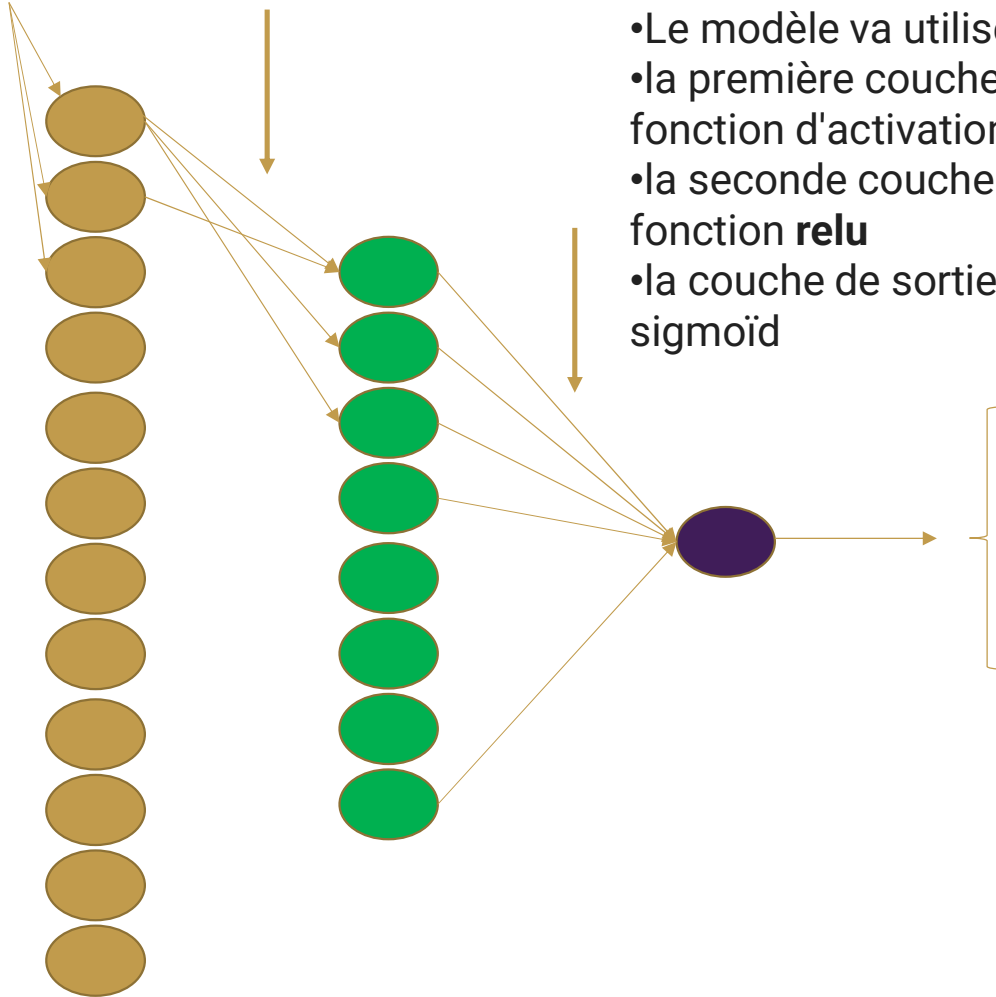
x1
x2

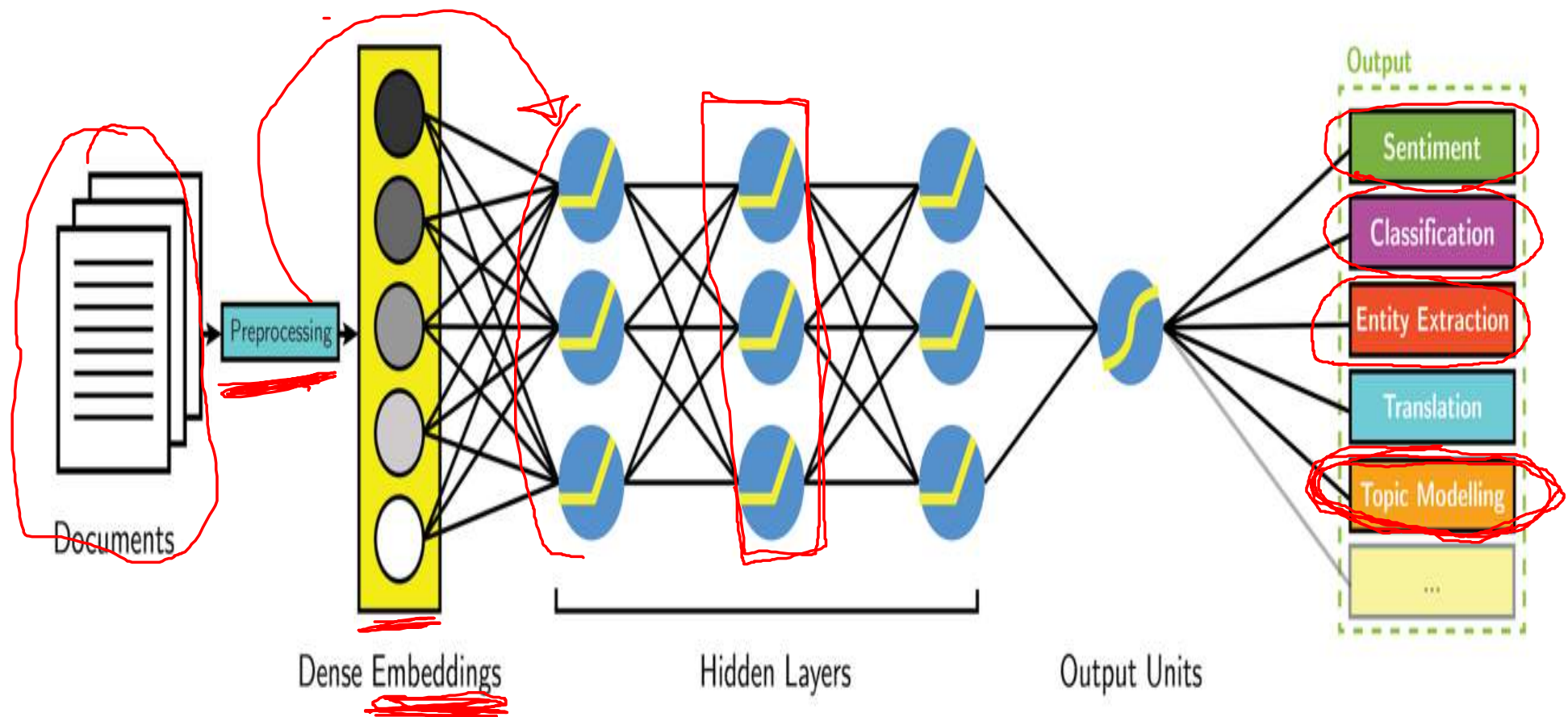
5943

4897

x5943

- Le modèle va utiliser **5943** variables (input_dim=**5943**)
- la première couche cachée a 12 noeuds et va utiliser la fonction d'activation **relu**
- la seconde couche cachée a 8 noeuds et utilise la fonction **relu**
- la couche de sortie a un noeud et utilise la fonction sigmoïd





Deep Neural Network - Test accuracy:

Deep Neural Network - Train Classification Report

	precision	<u>recall</u>	f1-score	support
0	0.62	0.56	0.59	383
1	0.90	0.92	0.91	1716
accuracy			0.86	2099
macro avg	0.76	0.74	0.75	2099
weighted avg	0.85	0.86	0.85	2099

TRAIN

$X_{\text{train}} \rightarrow \text{fit_transform}$

TEST

$X_{\text{train}} \rightarrow \text{transform}$

Modèle

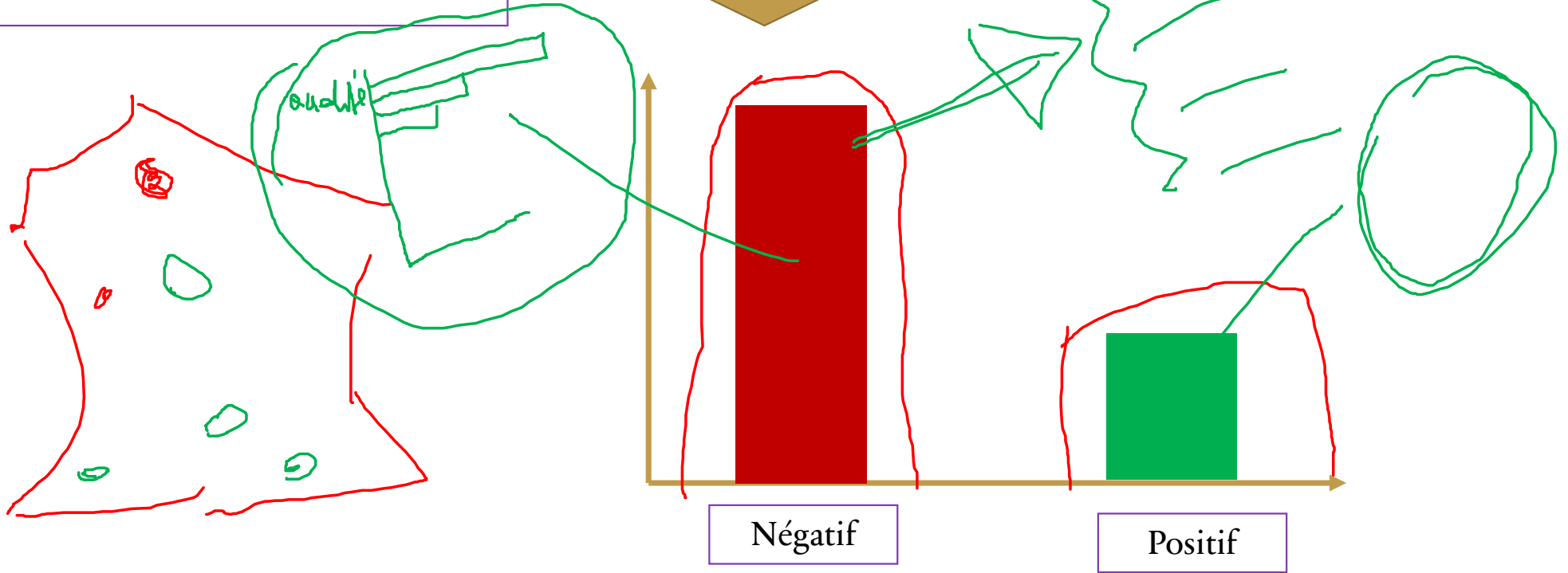
NOUVEAU CORPUS

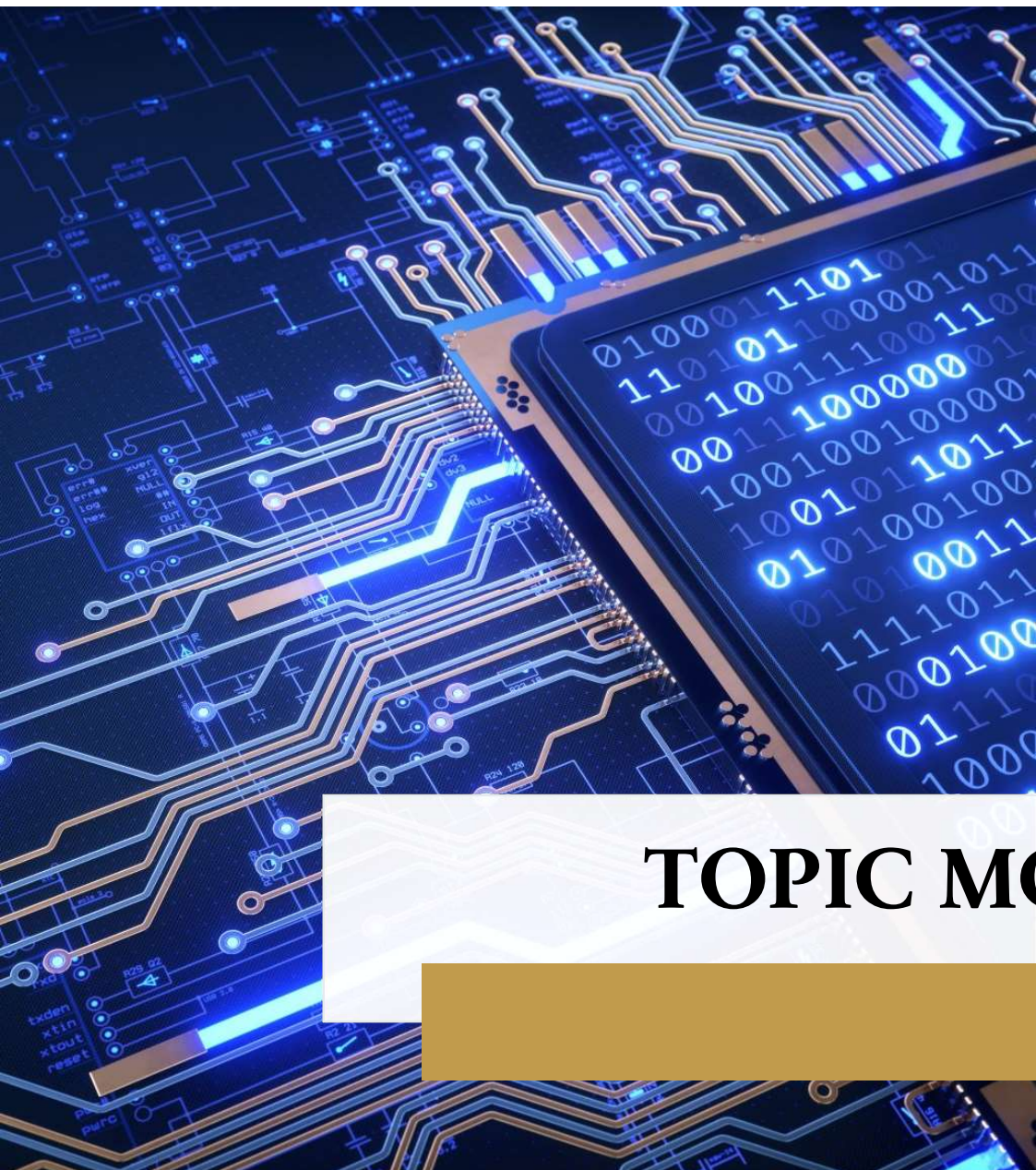
$X_{\text{nouveau}} \rightarrow \text{transform}$

DTM_nouveau

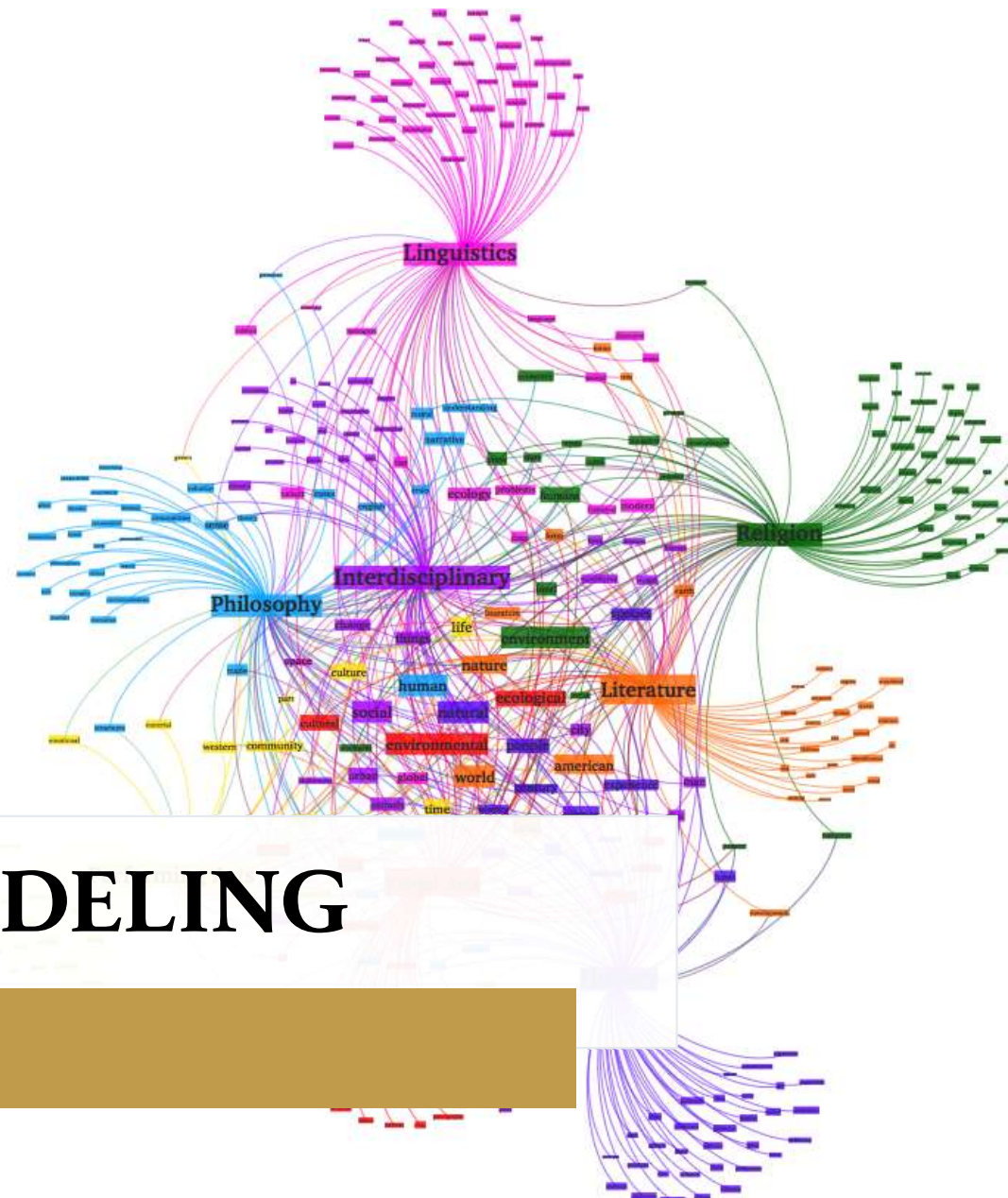
Commentaires

Modèle





TOPIC MODELING



Topic modeling is



a set of statistical models (Statistical modeling)

allowing to discover the “topics” hidden in a set of documents called Corpus.



It is about discovering the hidden structure of a corpus.

It is an unsupervised method based on word associations.



Topic modeling is a form of segmentation based on similarity of word usage.

Why?

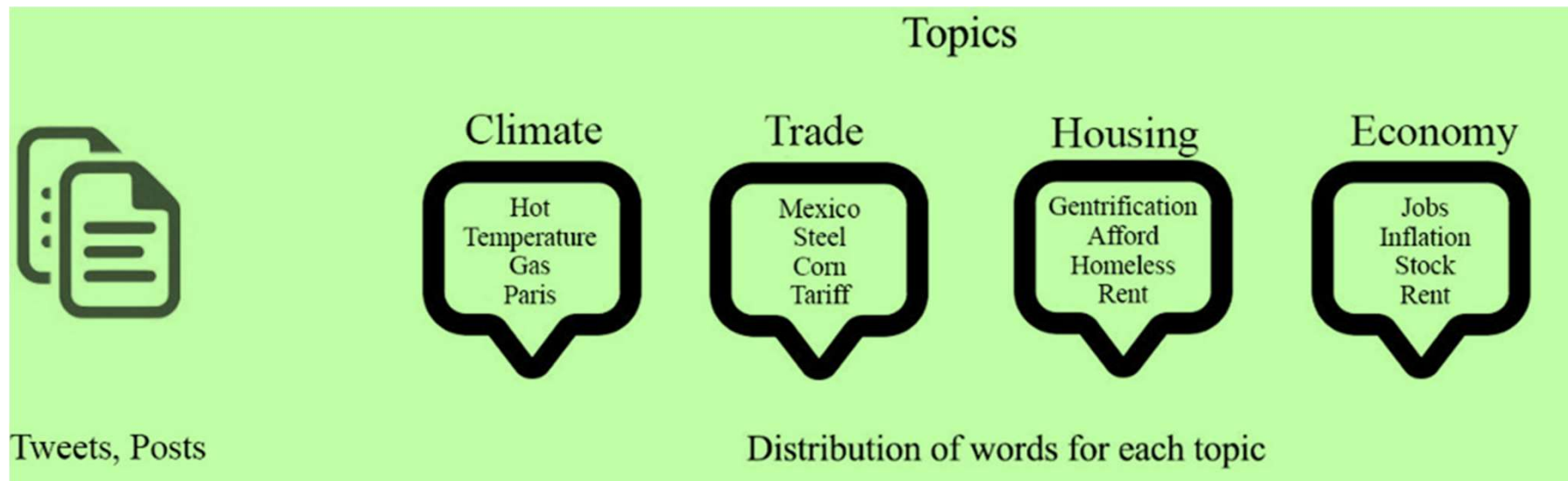
- We need to know what customers are saying about our products and services
- We could take these inferred topics and analyze the sentiment of their corresponding documents (reviews) to find out what customers are saying (or feeling) about specific products.
- We can extract representative statements or quotes, enabling us to summarize customers' opinions about products, perhaps even displaying them on the site.
- We use topic modeling to segment customers based on the topics
- We create features to feed into our ML models

How?

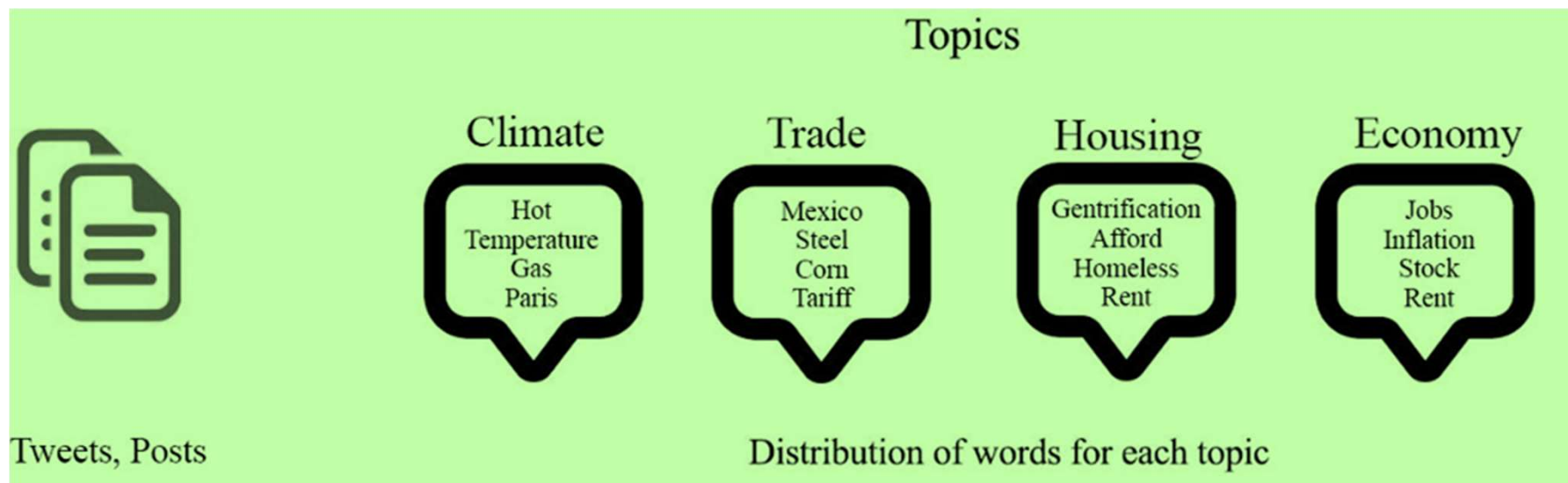
Work values	1	2	3	4	5	6	7	8
Pressure	0.74							
Authority	0.72							
Influence	0.69							
Impact	0.67							
Advancement	0.60							
Competence rewarded	0.50							
Continuous learning		0.67						
Fulfilling		0.66						
Interesting work		0.60						
Training		0.60						
Achievement		0.57						
Hours of work			0.80					
Balance			0.76					
Job security			0.60					
Benefits			0.55					
Salary			0.48					
Work environment			0.43					
Feedback				0.68				
Supportive supervisor				0.66				
Recognition				0.62				
Fairness				0.60				
Information				0.48				
Competent supervisor				0.40				
Contribution to society					0.84			
Helping people					0.83			
Moral values					0.62			
Fun						0.76		
Co-workers						0.76		
Social interaction						0.54		
Creativity							0.68	
Challenge							0.48	
Intellectually stimulating							0.51	
Use abilities							0.40	
Travel								0.57
Variety								0.47
Freedom								0.37
Independence								0.38
% explained variance	10.52	20.45	28.26	35.65	42.23	48.59	54.05	57.89

TABLE 1
w1
w2
w3

Note: Absolute values less than 0.40 were omitted. 1: Status; 2: Content; 3: Working conditions; 4: Organizational support; 5: Meaningful work; 6: Collegiality; 7: Competence; 8: Freedom.



- None of the tweets will explicitly say what topics they are.
 - A single tweet may be related to **multiple topics** and a **single word may imply different topics**.
- Our objective is grouping words in the tweets into K topics (say 4) and rates the likelihood of each word for each topic.
 - We group words based on the criteria that they should frequently appear in the same tweet. After the grouping, we review words that belong to the same group. From there, we may realize what the topic is.



- Each **topic** is simply a **distribution over words**. Each document contains a mixture of topics and words are drawn from these topics.

There are many techniques
for identifying topics ...

- Different methods:
 - NMF (Non-Negative Matrix Factorization)
 - LSA Latent Semantic Analysis
 - PLSA (Probabilistic latent Semantic Analysis)
 - LDA (Latent Dirichlet Allocation)
 - LDA in Deep Learning: lda2vec

Documents



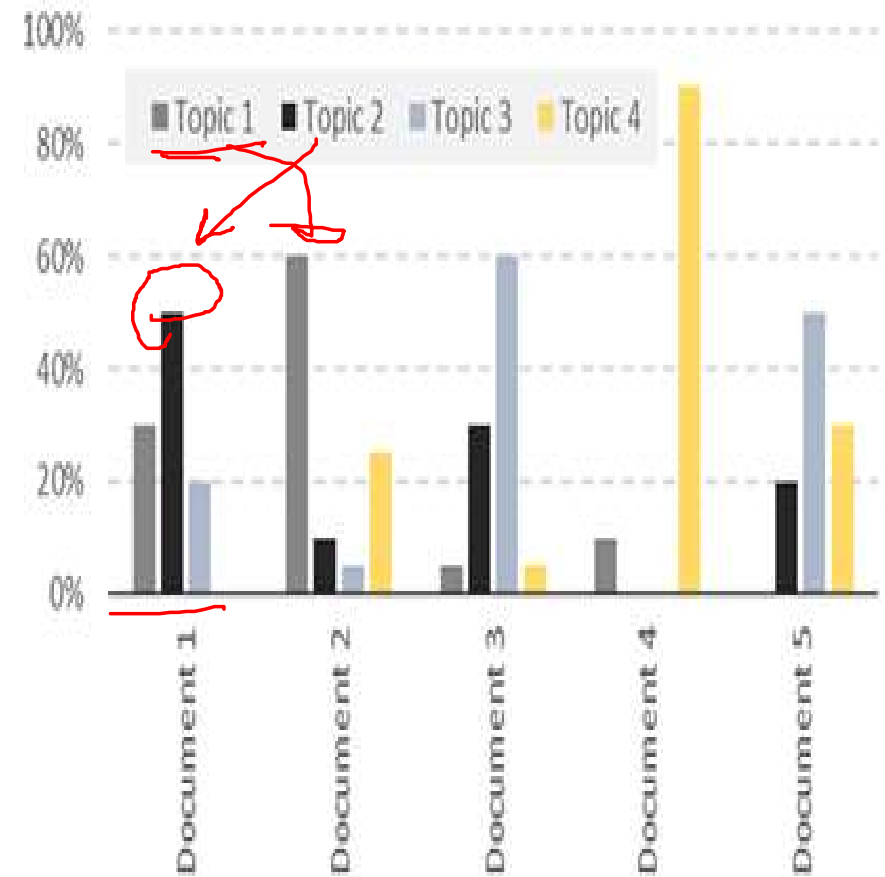
LDA

Creation of topics

	weight	words
<u>Topic 1</u>	3%	flower
	2%	rose
	1%	plant
...		
<u>Topic 2</u>	2%	company
	1%	wage
	1%	employee

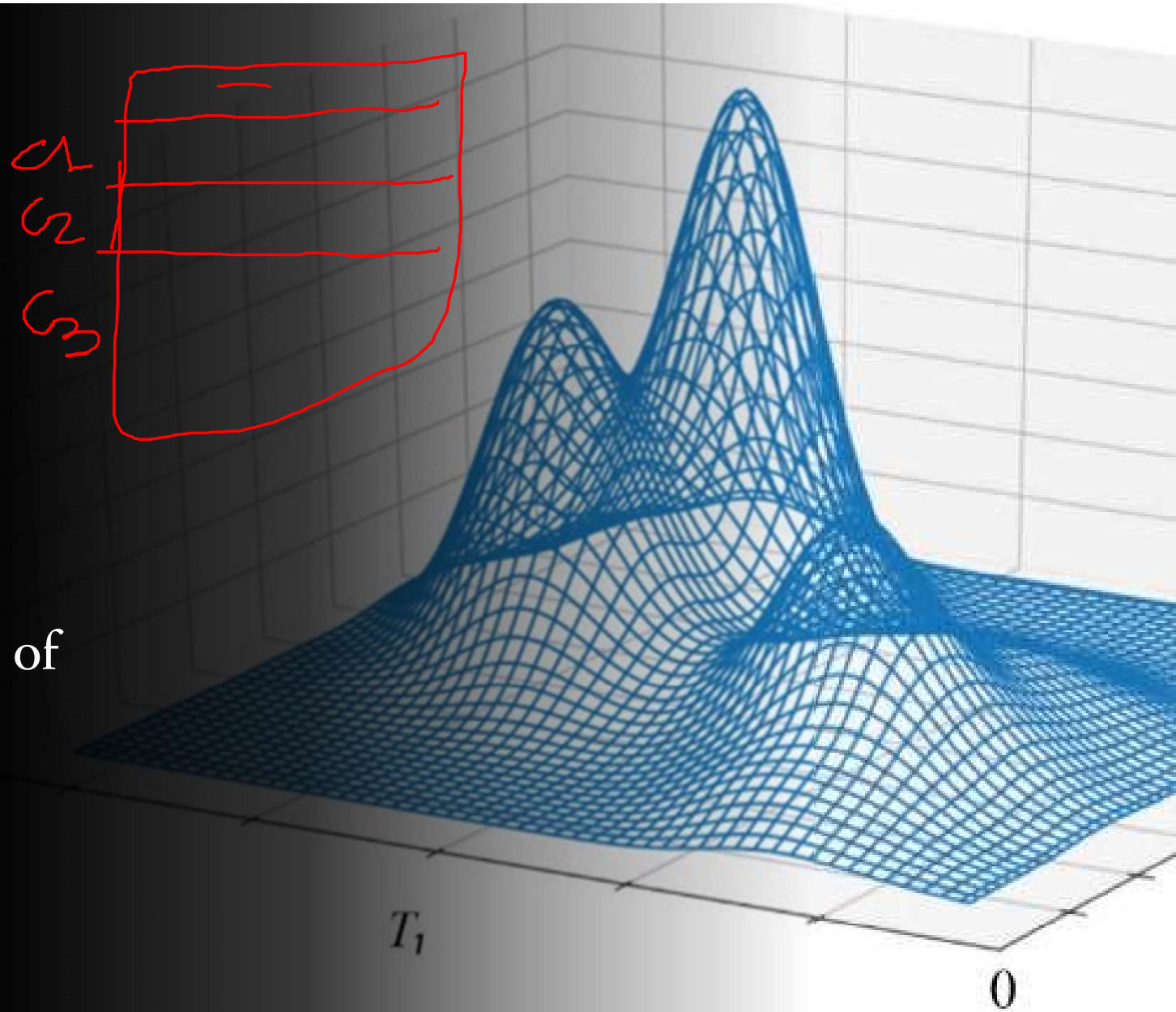
Topic 1

Topics allocation to documents



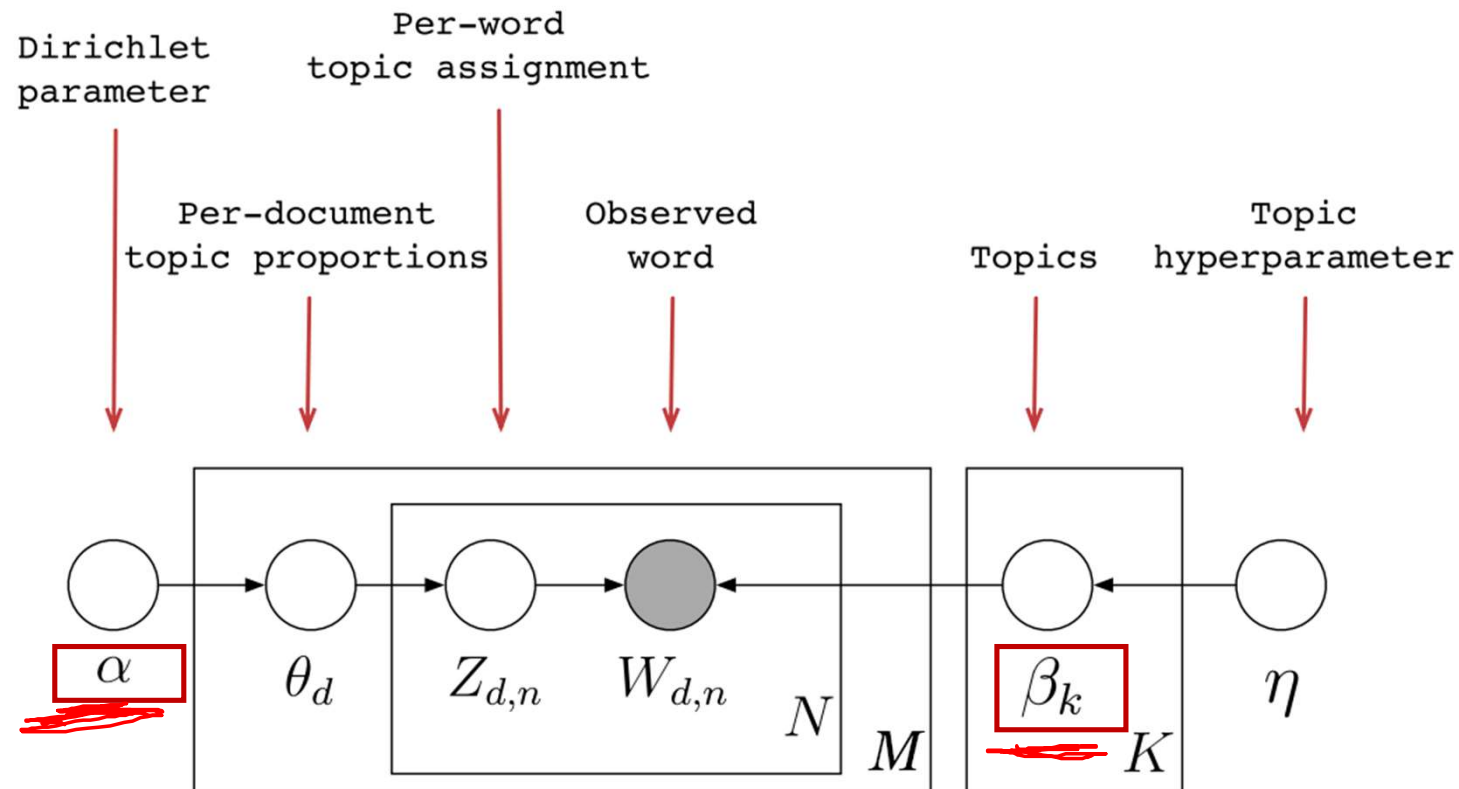
LDA Latent Dirichlet Allocation

- Latent → Hidden
- Dirichlet → A type of probability distribution
- Allocation : distribution of topics and words across documents topics



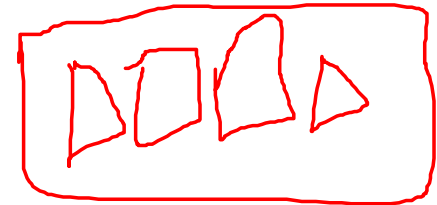


- LDA uses two Dirichlet distributions:
 - one for the distribution of topics over documents, and
 - one for the distribution of words over topics.
 - Both have their own alpha parameter, but in LDA they are given different names to avoid confusion.
- The parameter for the distribution of topics over documents is generally still referred to as the *alpha* parameter.
- The parameter for the distribution of words over topics, however, is often referred to as the *beta* parameter. These parameters are referred to as the hyperparameters of the LDA model.



- *Alpha* = Dirichlet's parameter on the distribution of topics by document
- *Beta* = Dirichlet's parameter on the distribution of words by topic
- M = the number of documents
- N = the number of words

• ALPHA AND BETA HYPERPARAMETERS



- *Alpha* represents the density of the topics in the document
 - The higher the alpha value, the more topics there are in the documents and the lower the alpha value, the fewer topics the documents contain.
- and
- *Beta* represents the density of words in the topic.
 - The higher the beta, the more words in the topics and the lower the beta, the fewer words in the topics.



Trade

Mexico	
Steel	
Com	
Tariff	

...

Climate

Hot	0.021
Temperature	0.015
Gas	0.008
Paris	0.005

Topic proportions for a document

[climate 0.7, trade 0.3, housing 0, economy 0]

$$p(\beta, \theta, \mathbf{z}, \mathbf{w}) = \left(\prod_{i=1}^K p(\beta_i | \eta) \right) \left(\prod_{d=1}^D p(\theta_d | \alpha) \prod_{n=1}^N p(z_{d,n} | \theta_d) p(w_{d,n} | \beta_{1:K}, z_{d,n}) \right)$$

the word "bond"

Topic assignment for a word
in a doc.
[trade]

Trade

Mexico	0.015
Steel	0.013
Com	0.008
Tariff	0.005

How it works?

1. Goal: you want LDA to learn the topic mix in each document (e.g. tweet, review), and the word mix in each topic
2. Choose the number of topics you think are in your corpus
 1. For example, $k=2$
3. Randomly assign each word in each document to one of the 2 topics
 1. For example: the word “banana” in document 1 assigned to topic B (“animals”)
4. Go through each word and its topic assignment in each document, examine how often a word occurs in a topic overall ... and how often the topic occurs in the document.
5. Go through multiple iterations

Topic Modeling Pipeline

